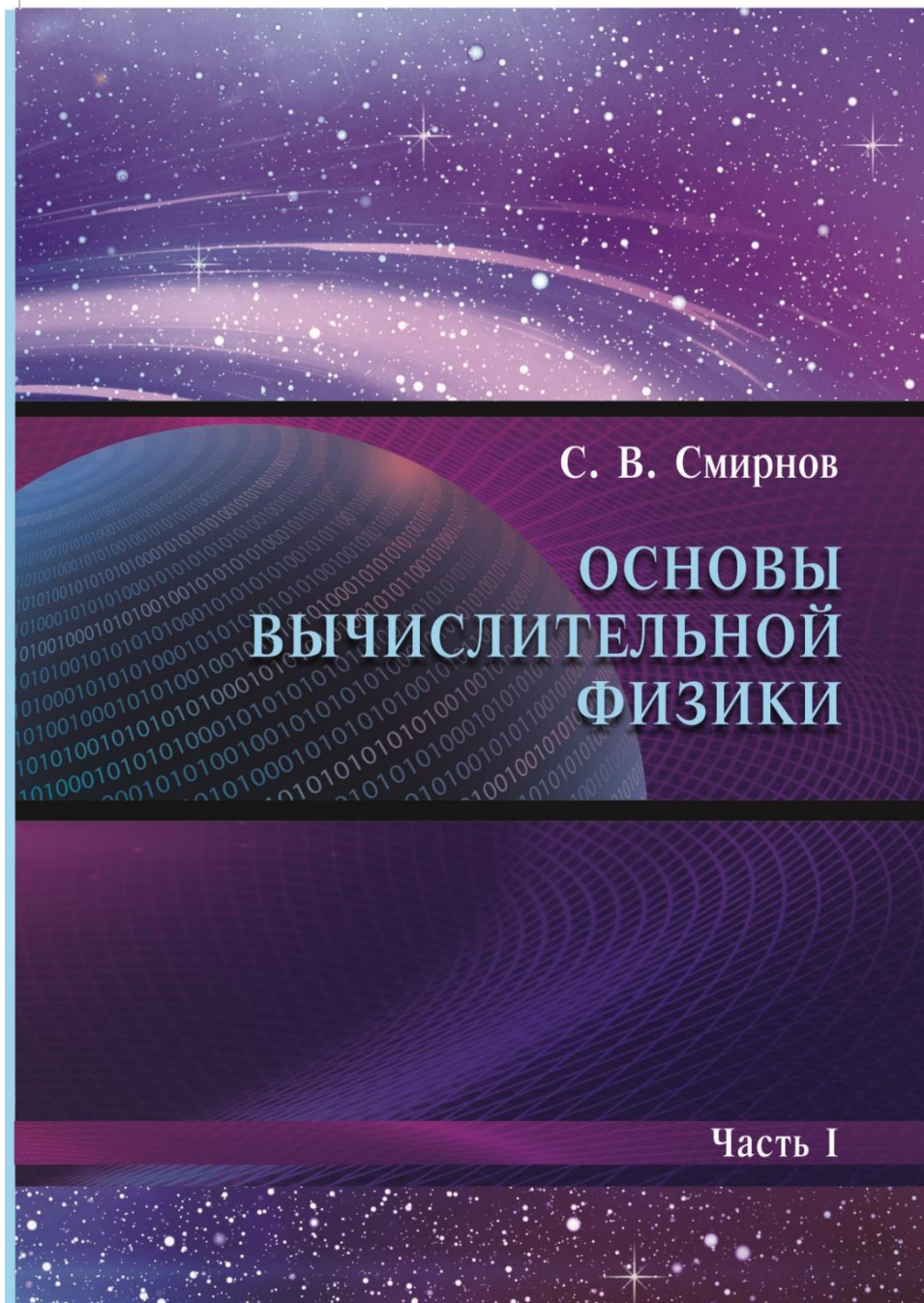


Введение в методы вычислительной математики

лекция 1

Чеховской Игорь Сергеевич

i.s.chekhovskoy@g.nsu.ru



Основа курса – методичка
С.В. Смирнова в двух частях

<http://www.phys.nsu.ru/smirnov/ovf.html>



ЗНАКОМСТВО



ЧТО? Методы вычислений, используемые при решении физических задач

ГДЕ? 5251 (лекции)
4214 (практикум)

КОГДА? Февраль-май, 1 лекция + 1 семинар в неделю
Май: *диф.зачёт*

КАК? Лекции:

- краткий обзор основных методов решения базовых задач
- примеры-демонстрации решения задач, обзор типичных ошибок
- 2 проверочные работы

Практикум:

- самостоятельное решение задач
- навык отладки программ, анализа и верификации результатов

Диф. зачёт:

Оценка 3: сдать до конца года 5 задач

Оценка 4: сдать до 2-го мая (включительно) 5 задач, а в сумме до конца года 6 задач

Оценка 5: сдать до 2-го мая (включительно) 5 задач, а в сумме до конца года 7 задач

Задачи сдаются **строго по порядку**.

Кроме того в конце апреля будет проверочная по материалам из лекций (проведем вместо одной из лекций), результаты которой также будут влиять на финальную оценку. При плохой сдаче проверочной возможно понижение итоговой оценки на 1 балл.

2:16 # 3

ЗНАКОМСТВО

ЗАЧЕМ?



*Американская атомная бомба
"Толстяк", уничтожившая
Нагасаки 9 августа 1945 года*



ЗНАКОМСТВО

Ленинградская АЭС –
активная зона реактора РБМК

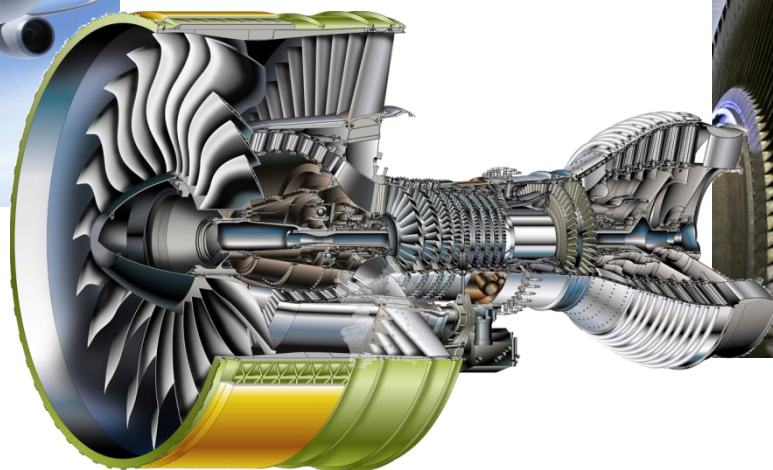
ЗАЧЕМ?



Американская атомная бомба



РДС-1

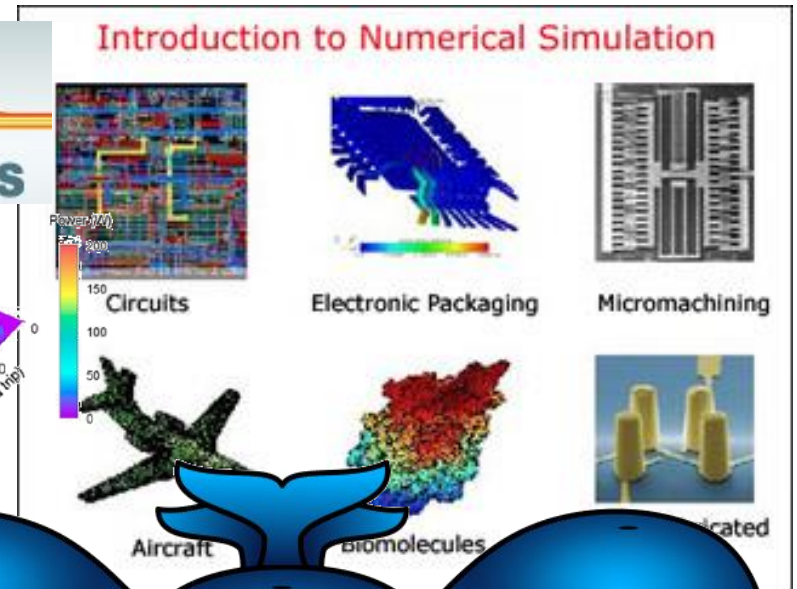
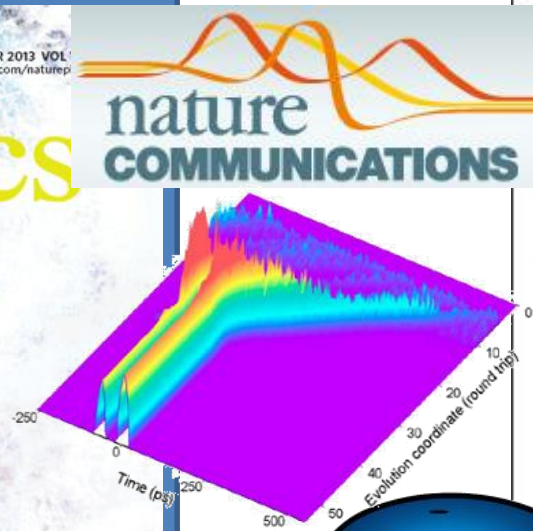
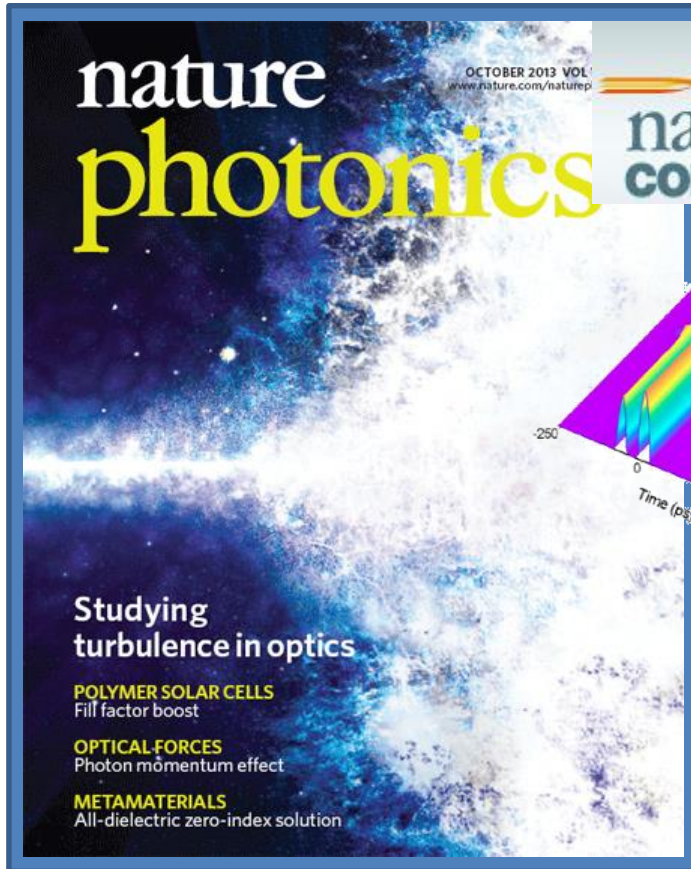


Турбина Балаковской АЭС

ЗНАКОМСТВО

ЗАЧЕМ?

MIT OpenCourseWare



Как правило, при решении достаточно сложной практической задачи используются и аналитические, и численные методы.

Первые позволяют понять качественную картину, приближенные зависимости от многих параметров. Вторые – уточнить полученные значения, особенно в промежуточных областях (в отсутствие малых и больших параметров), найти границы области применимости аналитических решений.

Аналитическое решение vs. численные методы

- Аналитическое решение позволяет увидеть зависимость от произвольного числа параметров.
- Аналитическое решение может быть получено для относительно простых задач (при наличии симметрии, малых параметров и т.п.) и почти всегда является приближенным.
- Численное решение дает значение искомой величины в одной точке. Чтобы получить зависимость от нескольких параметров, нужно выполнить большое число однотипных расчетов, что может потребовать много времени (ресурсов).
- Численное решение можно получить с меньшим количеством приближений – для несимметричных систем, при отсутствии малых параметров и т.п. Можно достичь более высокой точности решения / расширить область применимости.
- Численные методы решения задач мат. физики создают ощущение простоты (особенно при использовании готовых решений – MatLab, Mathematica и т.п.), но оно лишь отчасти соответствует действительности



J. R. Rice. *Numerical Methods, Software and Analysis*. 1983. p 343

The striking conclusion is reached that, for the general three-dimensional elliptic problem 10.3.2, **the progress made through better methods from 1945 to 1978 exceeds the progress made through faster computers.**

ТОЧНОСТЬ / ПОГРЕШНОСТЬ ВЫЧИСЛЕНИЙ



?

Может ли погрешность ответа быть меньше (много меньше?) погрешности входных данных?

??

Может ли погрешность ответа, полученного написанной «без ошибок» программой, быть одного порядка (больше, много больше) вычисляемой величины?

???

Можно ли оценить погрешность, не зная точного ответа?

$$\pi = 2\sqrt{3} \sum_{k=0}^{\infty} \frac{(-1/3)^k}{2k+1}$$

К вопросам точности вычислений мы будем возвращаться на протяжении всего курса, а для начала разберёмся в том, как компьютер работает с числами.

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

Мотивация

Зачем это инженерам?

Временами компьютер выдаёт неправильный результат. Нужно уметь
(а) понимать, когда ответ неправильный, (б) исправлять ошибки.

Примеры:

$$\frac{\sin \pi n}{\sin^n(\pi/n)} \equiv 0 \quad \forall n \geq 2$$

Разбиение отрезка L на n частей $h=L/n$ каждая и перебор while($L > 0$) $L=L-h$

$0.1+0.2-0.3=?$

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

Мотивация

Времен
(a) пони

Пример

$$\frac{\sin \pi}{\sin^n(\pi)}$$

Разбиен

0.1+0.2-



уметь

.

> 0) L=L-h

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

Мотивация

Особенно, если хотите запускать ракеты в космос!

Авария ракеты-носителя «Ариан-5» (4 июня 1996 года)

Одна из самых дорогостоящих компьютерных ошибок в истории — оценки только материальных потерь варьируются от 360 до 500 млн долларов

Возникшая ошибка в программном модуле ИСО произошла во время конвертации 64-битного вещественного числа в 16-битное знаковое целое, и при этом случилось арифметическое переполнение последней



ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

Мотивация

Или лечить людей!



Therac-25 — аппарат лучевой терапии, медицинский ускоритель

С июня 1985 года по январь 1987 года этот аппарат стал причиной как минимум шести передозировок радиации, некоторые пациенты получили дозы в десятки тысяч рад. Как минимум двое умерли непосредственно от передозировок.

В программном обеспечении Therac-25 были найдены как минимум четыре ошибки, которые могли привести к переоблучению (гонка данных, деление на ноль, переполнение)

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

целые числа

Сколько всего чисел?

В математике множество чисел *бесконечно*, однако *память ЭВМ ограничена*

Как следствие,

- количество различных чисел, которые могут быть записаны в ЭВМ, конечно;
- при любом способе кодирования существует *min* и *max* числа;
- аналог вещественных чисел в ЭВМ имеет конечную точность.

Как компьютер работает с числами?

(Для численного решения задач мы будем использовать языки программирования C/Python и x86-совместимые процессоры).

Все данные в компьютере хранятся и обрабатываются в двоичном виде, т.е. в виде последовательностей *битов*, каждый из которых может принимать всего два значения: 0 и 1.

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

целые числа

Как компьютер работает с числами?

(Для численного решения задач мы будем использовать язык программирования Си и x86-совместимые процессоры).

Все данные в компьютере хранятся и обрабатываются в двоичном виде, т.е. в виде последовательностей *битов*, каждый из которых может принимать всего два значения: 0 и 1.

Например, последовательность из трёх битов

101₂ соответствует десятичному числу $\underline{1} * 4 + \underline{0} * 2 + \underline{1} * 1 = 5_{10}$.

Последовательность 1101₂ соответствует $(\underline{1} * 8 + \underline{1} * 4 + \underline{0} * 2 + \underline{1} * 0)_{10} = 13_{10}$ и т.п.

Стандартный целочисленный тип данных в Си (`int`) обычно имеет разрядность 32 бита. В беззнаковом (`unsigned`) варианте это позволяет кодировать числа от 0 до $2^{32}-1 = 4\,294\,967\,295$, `signed int`: от $-2\,147\,483\,648$ до $+2\,147\,483\,647$.

Для тех случаев, когда этого недостаточно (кодирование даты, работа с памятью и файловой системой, финансы), предусмотрены 64-битные целые `std::int64_t`.

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

дробные числа



Можно ли, имея в распоряжении только целочисленную арифметику, заниматься физическими расчётами?

Можно, но это *неудобно*.

Можно было бы договориться, что последние пять десятичных разрядов числа являются дробной частью (формат чисел с *фиксированной точкой*). Например, число π представлялось бы при этом в виде 3**14159**.

✓ **Элементарная реализация четырёх арифметических действий**

- ❖ **Проблемы с переполнением:** 32-разрядный `int` не позволяет записать значение скорости света в СГС $c = 3 \cdot 10^{10}$ см/с
- ❖ **Проблемы с числами, близкими к нулю:** невозможно записать значение постоянной Планка $\hbar = 1.054 \cdot 10^{-27}$ эрг · с
- ❖ **Потеря точности при работе с относительно малыми числами** (0.0001)

ВЫВОД: для численных расчётов больше подходят числа с *плавающей точкой*
 $3.00 \cdot 10^{10}$ см/с, $1.05 \cdot 10^{-27}$ эрг · с, $1.38 \cdot 10^{-23}$ Дж/К, $6.02 \cdot 10^{23}$ моль⁻¹

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

числа с плавающей точкой

Например,

$$\hbar = \underbrace{1.054}_{\text{мантисса}} \cdot \overbrace{10^{-27}}^{\text{основание системы счисления (const, можно не хранить в памяти)}} \underbrace{\hspace{1cm}}_{\text{порядок числа (экспонента)}}$$

Неоднозначность: одно и то же число 3.14 можно записать по-разному:
 $3.140 \cdot 10^0$, $0.314 \cdot 10^1$, $31.400 \cdot 10^{-1}$ и т.д.

Основные проблемы, связанные с неоднозначностью:

- ❖ Более низкая эффективность (скорость) вычислений и обработки данных
- ❖ Избыточность кода → неэффективное использование памяти

Решение: использование *нормализованных чисел* ($1 \leq M_b < b$)

Договоримся, что первый разряд мантиссы нормализованного числа не равен нулю – **значит, в двоичной записи он всегда равен единице!**

В двоичном представлении нормализованное число x с плавающей точкой может быть записано с помощью трёх целых чисел S, E, M :

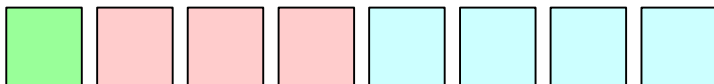
$$x = (-1)^S \cdot 1.M \cdot 2^E$$

Порядок E может быть разных знаков – в памяти его хранят в смещённом виде: $E \rightarrow E + bias$, $bias = 2^{w-1} - 1 \approx E_{\max}/2$

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

нормализованные числа с плавающей точкой

Для наглядности рассмотрим гипотетическое восьмибитовое число с плавающей точкой с $w=3$ (три бита для записи порядка), $t=4$ (четыре бита для мантииссы):



Как записать число 1? Нормализованный вид числа 1.00 есть $+1.00 \times 2^0$:



Опуская единицу нормализованного числа 1.0000, имеем мантииссу $M = 0$

Смещённый порядок $E_S = E + (2^{w-1} - 1) = 0 + (2^{3-1} - 1) = 3$

Знак числа $+1 = (-1)^0$, т.е. знаковый бит равен 0

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

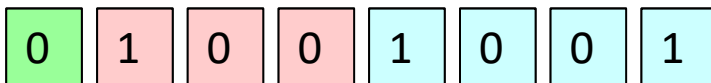
нормализованные числа с плавающей точкой

Запишем в том же восьмибитовом представлении число $\pi \approx 3.14159 \dots$

Ближайшая к π степень 2 есть 2^1 , поэтому в представлении с плавающей точкой $\pi = 1.5708_{10} \times 2^1$.

Мантисса имеет четыре двоичных разряда. Если бы разряды были десятичными, то мантисса кодировала бы *десятитысячные* доли. У нас двоичное счисление, поэтому знаменатель дроби равен $\dots 2^4 = 16$.

Мантисса $\pi/2 \approx 1.5708 \approx 1 \frac{9}{16}$, откуда получаем последние 4 бита (**M**):



Стандарт C99 (поддерживается gcc) вводит формат %a :
`printf("pi=%a\n", 3.14);`
`> pi=0x1.91eb86p+1`

$\left(1 \frac{9}{16}\right)_{10} = 1.1001_2$ (единицу в мантиссе нормализованного числа опускаем)

Смещённый порядок $E_S = E + (2^{w-1} - 1) = 1 + (2^{3-1} - 1) = 1 + 3 = 4$

Знак числа $+1 = (-1)^0$, т.е. знаковый бит равен 0

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

денормализованные числа с плавающей точкой

Какое самое маленькое по модулю нормализованное число?

(число вида $(-1)^S \cdot 1.M \cdot 2^E$) $n_1 = 1.00 \dots 00 \cdot 2^{E_{\min}}$

В нашем примере $w = 3$ и $n_1 = 1.0000 \cdot 2^{-3} = 1/8 = 0.125_{10}$.

Для $w = 8$ имеем $n_1 = 2^{-127} \approx 5.877 \cdot 10^{-39}$.

Число, следующее за минимальным: $n_2 = 1.00 \dots 01_2 \cdot n_1 = 0.1328_{10}$ для $w = 3$.

Для $w = 8$, $n = 23$: $n_2 = (1 + 2^{-23}) \cdot 2^{-127} \approx (1 + 1.192 \cdot 10^{-7}) \cdot n_1$.

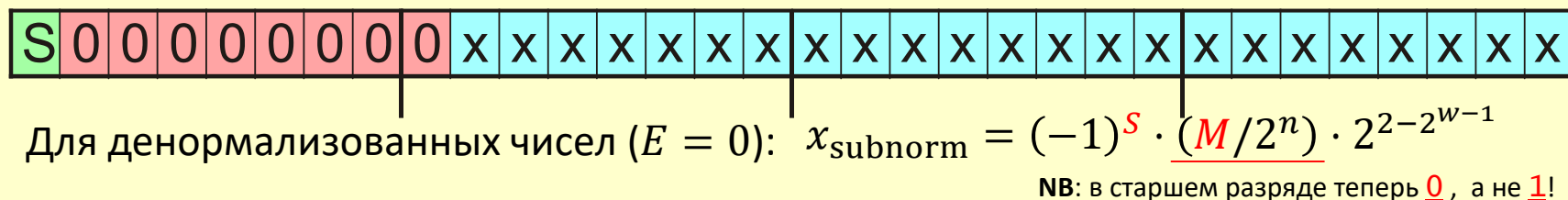
Видно, что интервал между минимальным (n_1) и следующим за ним (n_2) нормализованным числом в 2^n раз меньше минимального числа ($n_2 - n_1 \ll n_1$), т.е. в окрестности нуля наблюдается «дырка» (underflow gap). Ещё хуже то, что *ноль не представим в нормализованном виде*.



Для решения этих проблем введены *денормализованные* (subnormal) числа.

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

нормализованные и денормализованные числа с плавающей точкой IEEE 754



При $E = 0$ и $M = 0$ получаем ± 0 (есть два нуля, «+0» и «-0»!)

При $E = 2^w - 1 = \underline{11\dots1}_2$ и $M = 0$ получаем $\pm\infty$ (переполнение; $x/0$)

При $E = 2^w - 1 = \underline{11\dots1}_2$ и $M \neq 0$ – неопределенность, NaN (Not-a-Number)

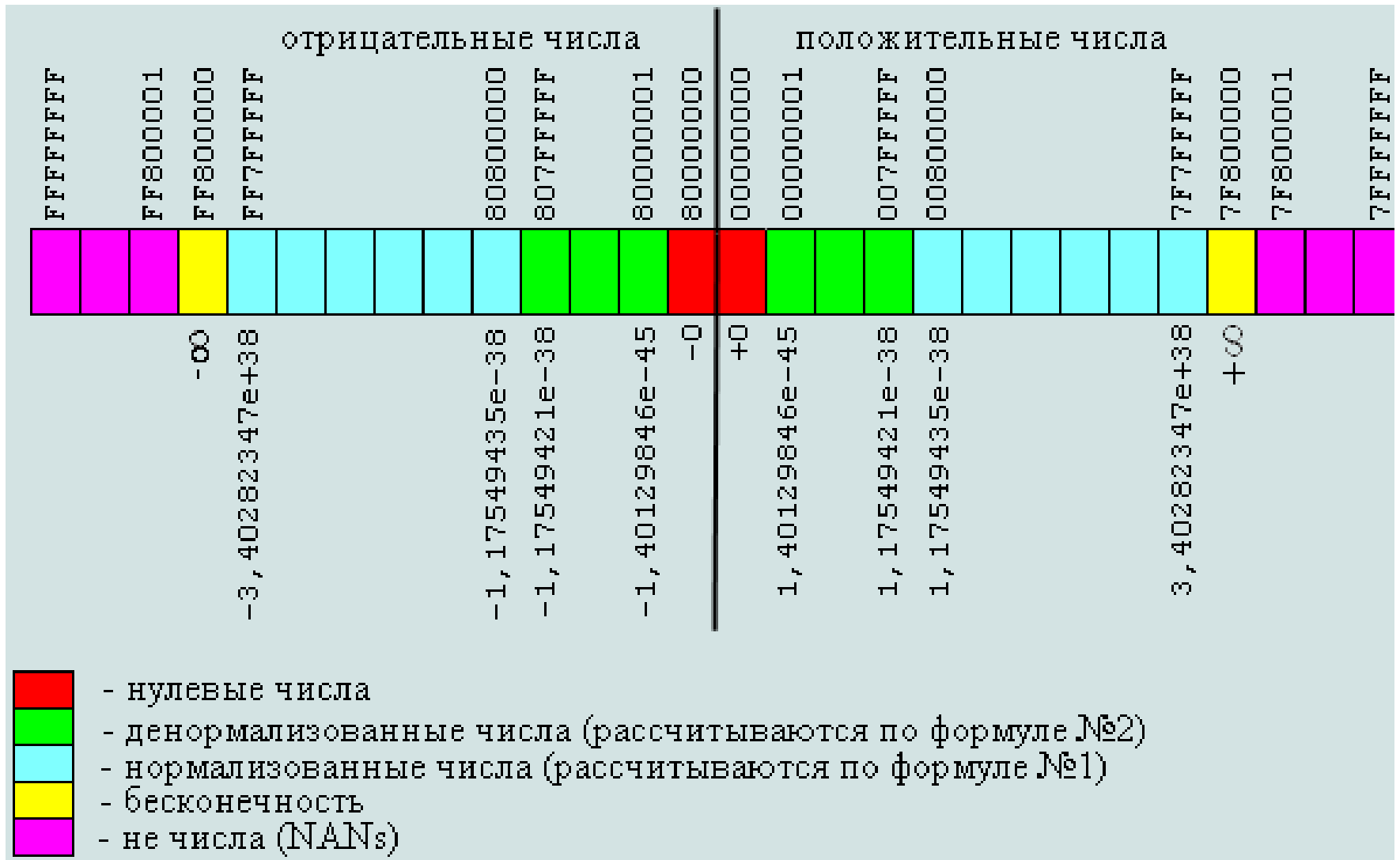
Неопределенность (NaN) получается в одном из следующих случаев:

- $\infty + (-\infty)$,
- $0 \times \infty$,
- $0/0$, ∞/∞ ,
- $\text{sqrt}(x)$ при $x < 0$

Особенности: • NaN $\neq$ NaN • избыточность кода NaN • quiet & signaling NaNs

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

нормализованные и денормализованные числа с плавающей точкой IEEE 754



ПРЕДСТАВЛЕНИЕ ЧИСЕЛ В ЭВМ

нормализованные и денормализованные числа с плавающей точкой IEEE 754

IEEE Std 754-2008
IEEE Standard for Floating-Point Arithmetic

3.4 Binary interchange format encodings

Each floating-point number has just one encoding in a binary interchange format. To make the encoding unique, in terms of the parameters in 3.3, the value of the significand m is maximized by decreasing e until either $e=e_{min}$ or $m \geq 1$. After this process is done, if $e=e_{min}$ and $0 < m < 1$, the floating-point number is subnormal. Subnormal numbers (and zero) are encoded with a reserved biased exponent value.

Representations of floating-point data in the binary interchange formats are encoded in k bits in the following three fields ordered as shown in Figure 3.1:

- a) 1-bit sign S
- b) w -bit biased exponent $E = e + bias$
- c) $(t = p - 1)$ -bit trailing significand field digit string $T = d_1 d_2 \dots d_{p-1}$; the leading bit of the significand, d_0 , is implicitly encoded in the biased exponent E .

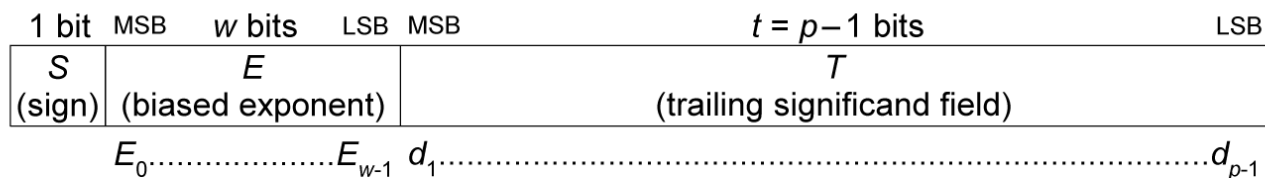


Figure 3.1—Binary interchange floating-point format

ПРЕДСТАВЛЕНИЕ ЧИСЕЛ в ЭВМ

нормализованные и денормализованные числа с плавающей точкой IEEE 754

IEEE Std 754-2008
IEEE Standard for Floating-Point Arithmetic

Table 3.5—Binary interchange format parameters

Parameter	binary16	binary32	binary64	binary128	binary{k} (k ≥ 128)	
k, storage	К счастью, работа с числами с плавающей точкой реализована в настоящее время настолько хорошо, что можно почти никогда не задумываться, как это работает. Исключения составляют лишь случаи возможного переполнения и работа на пределе точности вычислений, в том числе сравнение двух чисел с плавающей точкой на равенство / неравенство.					+ 13
p, precision						
emax, max						
bias, E−						
sign bit						
w, exponent field width in bits	5	8	11	15	round(4 × log2(k)) − 13	
t, trailing significand field width in bits	10	23	52	112	k − w − 1	
k, storage width in bits	16	32	64	128	1 + w + t	

ТОЧНОСТЬ / ПОГРЕШНОСТЬ ВЫЧИСЛЕНИЙ

Какова погрешность выполнения одной арифметической операции над числами с плавающей точкой?

Рассмотрим для примера умножение десятичных чисел **с плавающей точкой** с двумя знаками после запятой
(для простоты будем рассматривать десятичные числа):

$$7 \times \pi = ? \quad 7.00 \cdot 10^0 \times 3.14 \cdot 10^0 = 2.19\mathbf{9} \cdot 10^1 \approx 2.20 \cdot 10^1$$

Полученный ответ отличается от точного в третьем знаке мантииссы

Отбрасывание последней цифры за пределами точности вычислений называется *округлением*.

Округлите до целых следующие числа:

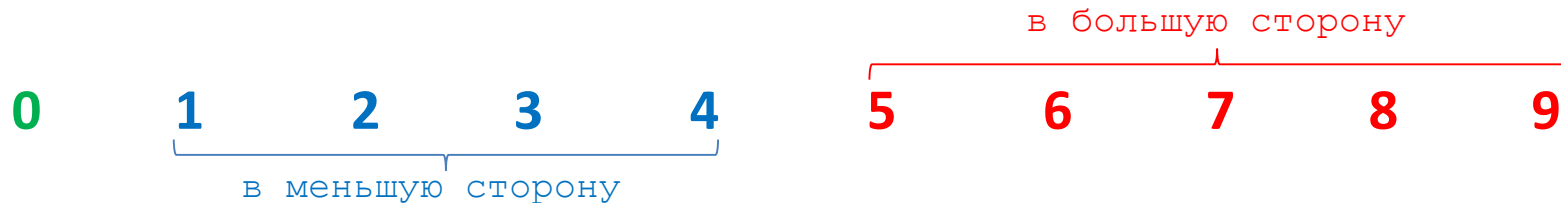
$$3.14159 \dots \approx 3$$

$$2.71828 \dots \approx 3$$

$$2.50000 \dots \approx \mathbf{2}$$

ОКРУГЛЕНИЕ

Что было бы, если бы компьютер округлял числа так, как учат в школе?



Полагая возникновение различных цифр 0..9 в отбрасываемом разряде равновероятным, легко оценить математическое ожидание ошибки, допускаемой при округлении:

$$M = \frac{1}{10} \cdot \varepsilon' \cdot (0 - 1 - 2 - 3 - 4 + 5 + 4 + 3 + 2 + 1) = \frac{\varepsilon'}{2}$$

(здесь ε' – единица в отбрасываемом разряде, ULP/B).

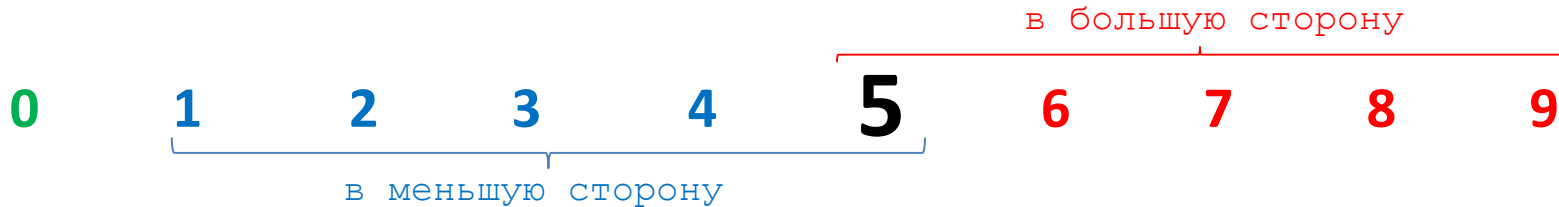
Очевидно, тот же самый ответ мы получим в любой системе счисления с чётным количеством цифр (в т.ч. и в двоичной системе).

$M > 0$ означает, что есть *систематическая погрешность (дрейф)*.

Суммарная ошибка вычислений будет возрастать *линейно* с числом операций.

ОКРУГЛЕНИЕ

Более правильный способ округления («банковский»):

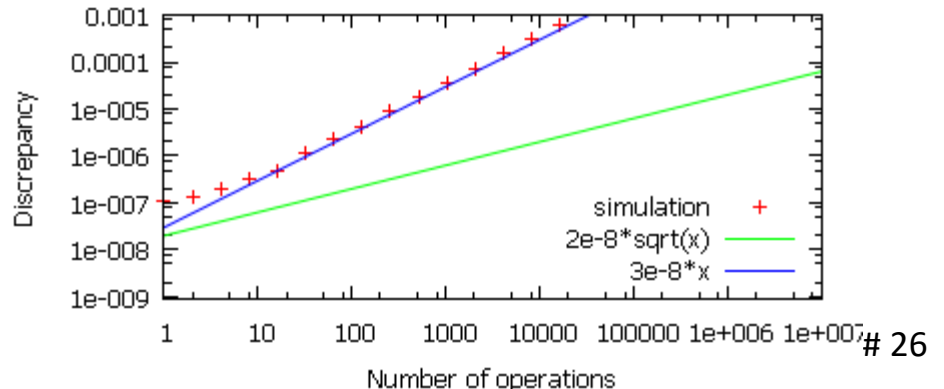
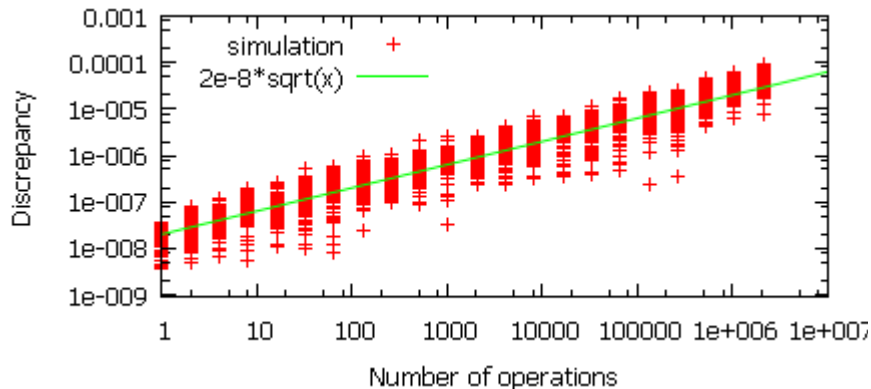


Если десятичную цифру 5_{10} (двоичную 1_2 и т.п.) округлять то в большую, то в меньшую в зависимости от чётности значения в предыдущем разряде, то математическое ожидание ошибки будет равно нулю:

$$M = \frac{1}{10} \cdot \varepsilon' \cdot \left(0 - 1 - 2 - 3 - 4 - 5 \cdot \frac{1}{2} + 5 \cdot \frac{1}{2} + 4 + 3 + 2 + 1 \right) = 0$$

При этом суммарная ошибка вычислений будет возрастать не линейно, а пропорционально корню квадратному от числа операций, т.е.

значительно медленнее.



ПОТЕРЯ ТОЧНОСТИ ПРИ ВЫЧИТАНИИ

Если $x \approx y$, то при вычислении $x - y$ совпадающие старшие разряды мантииссы взаимно уничтожаются. Остаются «хвостовые» разряды, где доминирует ошибка округления $\rightarrow$ результат может иметь очень мало значащих цифр.

Типичные «опасные» выражения и способы безопасного вычисления:

$1 - \cos x$ при $|x| \ll 1$. Оптимизация: $1 - \cos x = 2\sin^2\left(\frac{x}{2}\right)$

$\sqrt{1 + \delta} - 1$ при $|\delta| \ll 1$. Оптимизация: $\frac{\delta}{\sqrt{1+\delta}+1}$

Практика: избегать вычитания близких чисел
(переформулировать/рационализировать), подбирать порядок операций, при необходимости использовать повышенную точность

ТОЧНОСТЬ / ПОГРЕШНОСТЬ ВЫЧИСЛЕНИЙ

Точность вычислений удобно характеризовать т.н. *машинным эпсилон* – наименьшим числом, которое можно прибавить к единице, чтобы получилось число, отличное от единицы. Другими словами, ε – единица в младшем разряде (ULP, *unit in the last place*, или *unit of least precision*)

Задача 1

Машинным ε называется такое число, что $1 + \varepsilon/2 = 1$, но $1 + \varepsilon \neq 1$. (Также часто используется обозначение ULP – *unit in the last place*, или *unit of least precision*, единица в младшем разряде.) Найти машинное ε , число разрядов в мантиссе, максимальную и минимальную степени, при вычислениях с обычной и двойной точностью. Сравнить друг с другом четыре числа: 1 , $1 + \frac{\varepsilon}{2}$, $1 + \varepsilon$ и $1 + \varepsilon + \frac{\varepsilon}{2}$, объяснить результат.



ТОЧНОСТЬ / ПОГРЕШНОСТЬ ВЫЧИСЛЕНИЙ

РЕЗЮМЕ

- множество чисел, с которыми работает ЭВМ, конечно
- $\exists$ min и max число, остальные отображаются в $\pm\infty$
- помимо чисел и $\pm\infty$, есть неопределённость (NaN): $0/0$, ∞/∞ , $\infty + (-\infty)$
- компьютер работает с двоичными числами ($0.1_{10} + 0.2_{10} \approx 0.3_{10}$)
- точность вычислений на компьютере ограничена
- **относительную** погрешность удобно характеризовать машинным ε (ULP)
- числа с плавающей точкой, как правило, имеет смысл сравнивать лишь с точностью до некоторой погрешности
- арифметические операции на компьютере не обладают ассоциативностью, т.е. $(a + b) + c \neq a + (b + c)$. Например, $(10^{-20} + 1) - 1 \neq 10^{-20} + (1 - 1)$. Наибольшая потеря точности происходит при вычитании. Порядок операций нужно выбирать, помня о точности вычислений.
- для вывода чисел на экран удобно использовать `printf("%f", M_PI);`
`%e` - в формате с плавающей точкой, `%.10e` - 10 знаков после запятой

ЛИТЕРАТУРА

Смирнов С.В. Основы вычислительной физики. Часть I. НГУ, Новосибирск, 2015

Смирнов С.В. Основы вычислительной физики. Часть II. НГУ, Новосибирск, 2017

Калиткин Н.Н. Численные методы. Наука, М. 1978

Press W.H., Teukolsky S.A., Vetterling W.T., Flannery B.P.

Numerical recipes The art of scientific computing.

Cambridge University Press, New York, USA. 2007.

По языкам C/C++ :

Керниган Б.У., Ритчи Д.М. Язык программирования C. «Вильямс», М., 2007.

<http://cppreference.com/> и <http://www.cplusplus.com/>

Численное решение уравнений.

**Метод бисекции, метод простых итераций, метод
Ньютона**

Чеховской Игорь Сергеевич

Приближенное решение конечных уравнений

Конечное уравнение: $f(x) = 0$, $f(x)$ – некоторая функция (алгебраическая, тригонометрическая или их комбинация).

Не всякое конечное уравнение может быть решено точно:

- трансцендентные уравнения:

$$\operatorname{tg}(x) = x$$

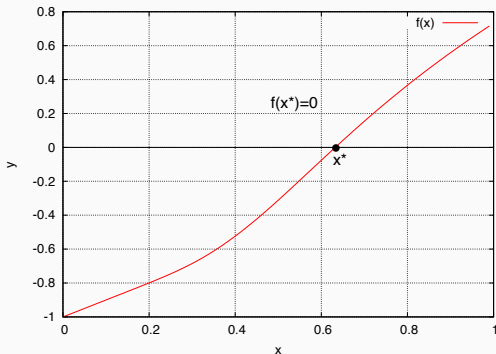
- алгебраические уравнения выше четвертой степени

Но задача может считаться решенной, если можно найти корень уравнения с необходимой точностью или найти отрезок $[a, b]$, в котором лежит уточняемый корень уравнения.

Нахождение примерных значений корня уравнений

Пример. Найти примерное значение корня уравнения:

$$f(x) = \frac{12}{e^{2/x} + 9} + x - 1 = 0$$



Примерное значение корня $x \approx 0.65$. Отрезок, в котором лежит корень $[0.6, 0.7]$.

Теорема (Больцано-Коши о промежуточном значении)

Пусть функция f непрерывна на отрезке $[a, b]$, причем $f(a) \neq f(b)$. Тогда для любого числа C , заключенного между $f(a)$ и $f(b)$ найдется такая точка $\gamma \in (a, b)$, что $f(\gamma) = C$.

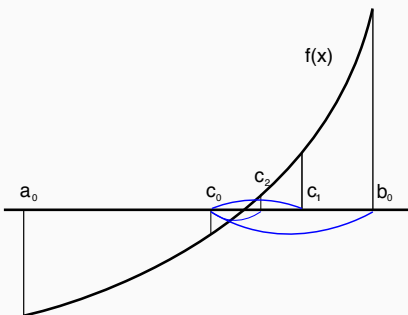
Следствие: Если функция f непрерывна на некотором отрезке $[a, b]$ и на концах этого отрезка принимает значения противоположных знаков $f(a) \cdot f(b) < 0$, то существует такая точка $\gamma \in (a, b)$ в которой $f(\gamma) = 0$.

Если отрезок $[a, b]$ настолько мал, что в нем лежит в точности один корень, то он является **интервалом изоляции корня**.

Достаточное условие того, что существует единственный корень уравнения $f(x) = 0$ на отрезке $[a, b]$:

1. Функция $f(x)$ непрерывна на отрезке $[a, b]$
2. Значения функции $f(x)$ на концах отрезка $[a, b]$ имеют разные знаки:
 $f(a) \cdot f(b) < 0$
3. Первая производная $f'(x)$ сохраняет определенный знак на всем отрезке, т.е. $f(x)$ – монотонная.

Метод деления отрезка пополам (метод бисекции, метод дихотомии)



1. Интервал изоляции корня $[a_0, b_0]$. Середина отрезка $c_0 = \frac{a_0 + b_0}{2}$.
Т.к. $f(c_0) \cdot f(b_0) < 0$, то $a_1 = c_0$, $b_1 = b_0$.
2. Интервал изоляции корня $[a_1, b_1]$. Середина отрезка $c_1 = \frac{a_1 + b_1}{2}$.
Т.к. $f(c_1) \cdot f(a_1) < 0$, то $a_2 = a_1$, $b_2 = c_1$.
3. Интервал изоляции корня $[a_2, b_2]$. Середина отрезка $c_2 = \frac{a_2 + b_2}{2}$.
Т.к. $f(c_2) \cdot f(a_2) < 0$, то $a_3 = a_2$, $b_3 = c_2$.
4. ...

Если требуется найти корень с точностью δ , деление продолжается пока длина отрезка $> 2\delta$. Проверка $|f(x)| < \text{const}$?

Метод является частным случаем бинарного (двоичного) поиска.

Максимальная погрешность на каждом шаге: $\frac{1}{2}(b_n - a_n)$.

Если $\delta_0 = b_0 - a_0$ – точность начального приближения, δ – требуемая точность, то необходимое число шагов

$$N \approx \log_2(\delta_0/\delta).$$

Если порядок корня определен (т.е. порядки a_n и b_n равны), то на каждом шаге метод уточняет очередной бит мантиссы.

Замечание. Для увеличения эффективности реализации метода на каждом шаге необходимо вычислять $f(x)$ один раз, а ранее найденные значения $f(a_n)$ и $f(b_n)$ сохранять в памяти.

Преимущества и недостатки метода бисекции

Преимущества:

- Метод прост и надежен: к простому корню сходится для любых непрерывных функций $f(x)$, в том числе недифференцируемых
- Точность метода гарантируется

Недостатки:

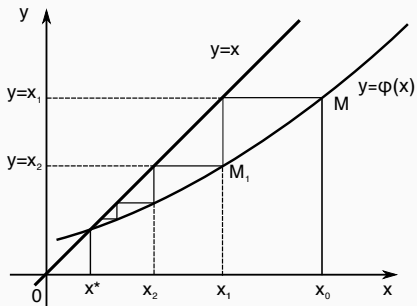
- Низкая скорость сходимости – за одну итерацию точность увеличивается примерно вдвое
- Если на отрезке несколько корней, то неизвестно, к какому корню сойдется процесс
- Метод неприменим для нахождения корней четной кратности
- Для корней нечетной высокой кратности метод сходится, но менее точен и хуже устойчив к ошибкам округления возникающих при вычислениях $f(x)$
- Метод не обобщается на функции нескольких переменных

Метод простых итераций (метод последовательных приближений)

$$f(x) = 0$$

→

$$\varphi(x) = x$$



Интервал изоляции корня $[a, b]$. $x_0 \in [a, b]$ – начальное (нулевое) приближение.

$$x_1 = \varphi(x_0)$$

$$x_2 = \varphi(x_1)$$

$$x_3 = \varphi(x_2)$$

...

$$x_n = \varphi(x_{n-1})$$

Если последовательность $x_1, x_2, \dots, x_n, \dots$, имеет предел $\lim_{n \rightarrow \infty} x_n = x^*$, то x^* является корнем уравнения:

$$x^* = \lim_{n \rightarrow \infty} x_n = \lim_{n \rightarrow \infty} \varphi(x_{n-1}) = \varphi\left(\lim_{n \rightarrow \infty} x_{n-1}\right) = \varphi(x^*)$$

Сходимость метода простых итераций

Сжимающее отображение – отображение метрического пространства $(\mathbb{M}, \rho)$ в себя, уменьшающее расстояние между любыми двумя точками не менее чем в $\alpha > 1$ раз:

$$\forall x, y \in \mathbb{M}: \alpha \cdot \rho(Tx, Ty) \leq \rho(x, y).$$

Полное метрическое пр-во – м. пр-во, в котором каждая фундаментальная последовательность сходится (к элементу того же пространства).

Теорема

(Банаха о неподвижной точке) Сжимающее отображение T на полном метрическом пространстве имеет, и при том единственную, неподвижную точку: $Tx^ = x^*$.*

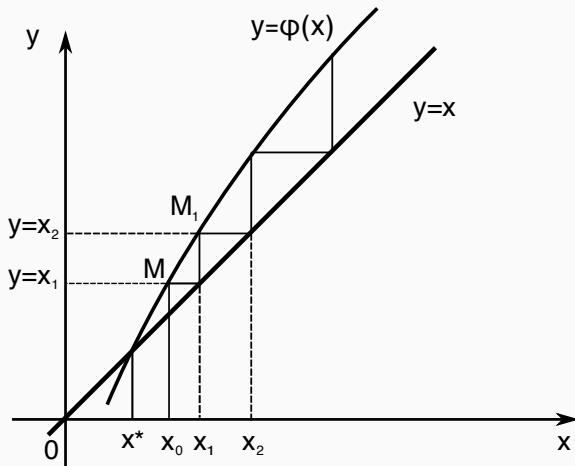
Теорема

Пусть отрезок $[a, b]$ является интервалом изоляции корня уравнения $x = \varphi(x)$ и во всех точках этого интервала производная $\varphi'(x)$ удовлетворяет неравенству

$$|\varphi'(x)| \leq q < 1.$$

При $a \leq \varphi(x) \leq b$ итерационный процесс сходится, причем за нулевое приближение x_0 можно брать любую точку отрезка $[a, b]$.

Пример расходящегося итерационного процесса



Модификации метода простых итераций

- Использование обратной функции $\varphi^{-1}(x)$ при $|\varphi'(x)| \geq q > 1$.
- Использование корректирующего множителя: $\varphi(x) = x - \lambda f(x)$, $\lambda \neq 0$.

$$x_{n+1} = x_n - \lambda f(x_n)$$

Достаточное условие сходимости:

$$\forall x : |\varphi'(x)| = |1 - \lambda f'(x)| \leq q < 1$$

Тогда

$$\forall x : 0 < 1 - q \leq \lambda f'(x) \leq 1 + q < 2$$

- $\lambda f'(x) > 0$,
- $|k| = \frac{1}{|\lambda|} > \frac{\max_x |f'(x)|}{2}$.

Скорость (порядок) сходимости

Погрешность:

$$R_n = x_* - x_n,$$

$$R_{n+1} = x_* - x_{n+1}.$$

Тогда

$$R_{n+1} \approx (1 - \lambda f'(x_*))R_n = qR_n^1, \quad q \approx \frac{x_n - x_{n-1}}{x_{n-1} - x_{n-2}}.$$

Порядок сходимости **первый** (погрешность уменьшается в геометрической прогрессии).

Если δ_0 – точность начального приближения, δ – требуемая точность, то необходимое число шагов

$$N \approx \frac{\log(\delta_0/\delta)}{-\log |\varphi'|},$$

т.е. при $|\varphi'(x_*)| < 0.5$ сходится быстрее метода деления отрезка пополам.

Преимущества и недостатки метода простых итераций

Преимущества:

- Не накапливаются ошибки вычислений
- Метод может быть обобщен на многомерный случай

Недостатки:

- Для сходимости метода необходимо выполнение условия $|\varphi'(x)| < 1$ как минимум вблизи корня.

Метод Ньютона (метод касательных)

Ищем решение уравнения $f(x) = 0$. Если x_n - некоторое приближение к корню x^* , а $f(x)$ имеет непрерывную производную, то:

$$0 = f(x^*) = f(x_n + (x^* - x_n)) = f(x_n) + (x^* - x_n)f'(\xi)$$

имеем следующий итерационный процесс:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

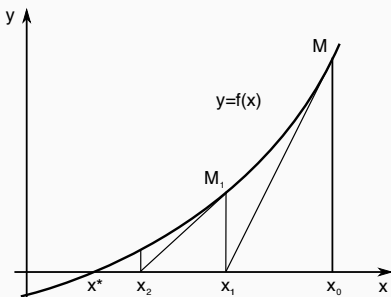
Геометрический смысл:

Уравнение касательной в точке x_0 :

$$y = f(x_0) + (x - x_0)f'(x_0)$$

В качестве приближенного корня уравнения $f(x) = 0$ выбирается абсцисса точки пересечения касательной с осью Ox :

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$



Условие и скорость сходимости метода Ньютона

Частный случай метода простых итераций с $\varphi(x) = x - \frac{f(x)}{f'(x)}$.

Если начальное приближение выбрано близко к корню, то итерации сходятся. При произвольном начальном приближении метод сходится при выполнении условия:

$$|f(x)f''(x)| < (f'(x))^2$$

Итерации сходятся к корню с той стороны, с которой $f(x)f''(x) > 0$. Для кратного корня скорость геометрической прогрессии.

Для простого корня (т.к. $x^* - x_n = \varphi(x^*) - \varphi(x_{n-1})$ и $\varphi'(x^*) = 0$):

$$x_n - x^* = \frac{1}{2}(x_{n-1} - x^*)^2 \varphi''(\xi), \quad \xi \in (x_{n-1}, x^*)$$

Погрешность очередного приближения $\approx$ квадрату погрешности предыдущего.

Критерий остановки метода Ньютона

Для надежного определения погрешности приближённого решения $\tilde{x}$ и контроля точности вычислений можно применять условие

$$|\tilde{x} - x^*| \leq \frac{|f(\tilde{x})|}{\min_{\xi \in [a, b]} |f'(\xi)|}.$$

Оно следует из теоремы Лагранжа о среднем (формулы конечных приращений):

$$f(\tilde{x}) - f(x^*) = f'(\xi) (\tilde{x} - x^*),$$

в которой x^* - точное решение уравнения, ξ - некоторая точка, заключённая между $\tilde{x}$ и x^* , т. е. $\xi \in (\tilde{x}, x^*)$. Ясно, что тогда

$$|f(\tilde{x}) - f(x^*)| \geq \min_{\xi} |f'(\xi)| \cdot |\tilde{x} - x^*|,$$

и при $\min_{\xi \in [a, b]} |f'(\xi)| \neq 0$ получаем оценку.

Преимущества и недостатки метода Ньютона

Преимущества:

- Быстрая сходимость метода (особенно для простых корней)
- Метод может быть обобщен на многомерный случай

Недостатки:

- Необходимо знать начальное приближение для корня.
- Необходимо знать первую производную $f'(x)$. (Устраняется применением метода секущих)

Метод секущих (метод хорд)

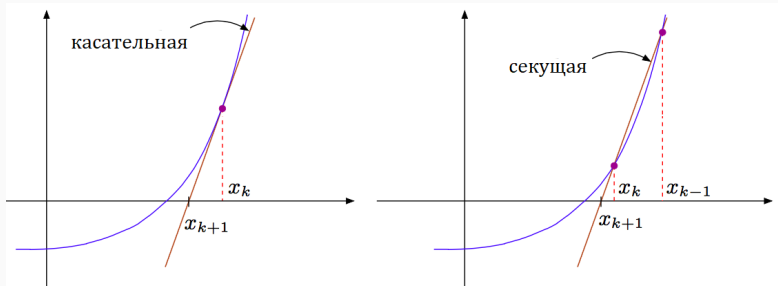
В общей формуле итерационного процесса

$x_{n+1} = x_n - \lambda f(x_n)$ вместо $\lambda = \frac{1}{f'}$ выберем

$$\lambda = \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \simeq \frac{1}{f'(x_n)},$$

т. е. возьмем обратное значение конечной разности.

Геометрический смысл:



Метод секущих (метод хорд)

Исследуем сходимость итерационного процесса вблизи корня. Для этого в формуле

$$x_{n+1} = x_n - \frac{(x_n - x_{n-1}) \cdot f(x_n)}{f(x_n) - f(x_{n-1})}$$

разложим все $f(x_k)$ в ряд в т. x_* :

$$x_{n+1} = \frac{x_n - (z_n - z_{n-1}) \cdot \left[f'(x_*) \cdot z_n + \frac{1}{2} f''(x_*) \cdot z_n^2 \right]}{f' z_n + f'' \frac{z_n^2}{2} - f' z_{n-1} - f'' \frac{z_{n-1}^2}{2}},$$

где $z_n \equiv x_n - x_* = -R_n$. Далее

$$z_{n+1} = z_n - \frac{(z_n - z_{n-1}) \cdot f' \cdot z_n \cdot \left[1 + \frac{f''}{2f'} z_n \right]}{(z_n - z_{n-1}) \cdot \left[f' + \frac{1}{2} f'' \cdot (z_n + z_{n-1}) \right]} = z_n - z_n \cdot \frac{1 + a z_n}{1 + a(z_n + z_{n-1})}, \quad a \equiv \frac{f''(x_*)}{2f'(x_*)},$$

$$z_{n+1} \simeq z_n - z_n \cdot (1 + a z_n) (1 - a(z_n + z_{n-1})) \simeq$$

$$z_n - z_n (1 + a z_n - a(z_n + z_{n-1})) = a z_n z_{n-1}.$$

Метод секущих (метод хорд)

Итого, получим

$$z_{n+1} = az_n z_{n-1}, \quad a \equiv \frac{f''(x_*)}{2f'(x_*)}, \quad z_n \equiv x_n - x_*$$

Для простых итераций имеем $z_{n+1} = q \cdot z_n^1$, для метода Ньютона $z_{n+1} = \gamma z_n^2$.

Попробуем найти аналогичную (степенную) зависимость между $x_{n+1} - x_*$ и $x_n - x_*$ для метода секущих в виде $z_{n+1} = a^\alpha z_n^\beta$:

$$a^\alpha z_n^\beta = az_n z_{n-1} \Rightarrow a^{\alpha-1} z_n^{\beta-1} = z_{n-1} \Rightarrow a^{\alpha-1} \left(a^\alpha z_{n-1}^\beta \right)^{\beta-1} = z_{n-1} \Rightarrow$$

$$a^{\alpha-1+\alpha(\beta-1)} z_{n-1}^{\beta^2-\beta-1} = 1 \Rightarrow \begin{cases} \alpha\beta - 1 = 0 \\ \beta^2 - \beta - 1 = 0 \end{cases} \Rightarrow \beta_{\pm} = \frac{1}{2} \pm \frac{\sqrt{5}}{2}$$

$\beta_- < 0$ – соответствует расходящемуся итерационному процессу, поэтому берем $\beta = \frac{1}{2} + \frac{\sqrt{5}}{2} \approx 1.618$.

Итерации в методе секущих сходятся быстрее, чем простые итерации при $\lambda = \text{const}$, но медленнее (по числу шагов), чем метод Ньютона.

Метод секущих (метод хорд)

Заметим, однако, что в методе Ньюtone на каждом шаге требуется вычисление $f(x_n)$ и $f'(x_n)$. Полагая время их вычисления приблизительно одинаковым и при том существенным по сравнению с временем выполнения одной арифм. операции, получим, что на каждую пару таких вычислений приходится один шаг метода Ньютона и увеличение точности $z_{n+1} = \gamma z_n^2$.

За то же время можно выполнить две итерации м. секущих, получив $z_{n+2} = a^\alpha z_{n+1}^\beta = a^\alpha (a^\alpha z_n^\beta)^\beta = a^{\alpha+\alpha\beta} z_n^{\beta^2}$, $\beta^2 \approx 2.618$.

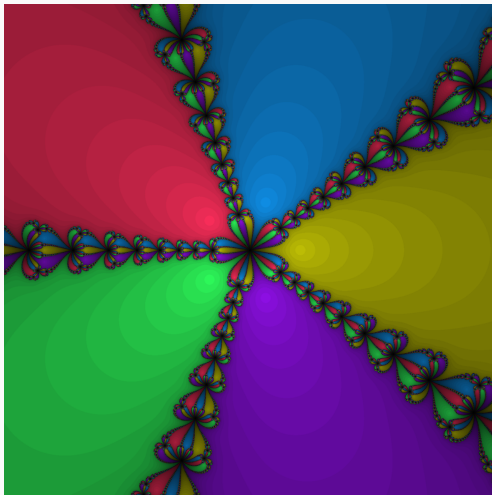
Т. е. метод секущих может сходиться быстрее, чем метод Ньютона!

Условие остановки итераций

Формула м. секущих включает в себя разделенную разность, погрешность вычисления которой м. б. велика при малых $|x_n - x_{n-1}|$.

Метод Гарвика:

- выбираем δ и выполняем итерации, пока $|x_{n+1} - x_n| > \delta$ (до попадания в окрестность корня, где хорошо работает разложение в ряд Тейлора)
- продолжаем выполнять итерации, пока $|x_{n+1} - x_n|$ убывает. Рост свидетельствует о начале "разболтки"



$$p(z) = z^5 - 1$$

<https://arxiv.org/pdf/1708.01144.pdf>

Задача 2

Решить уравнение $\operatorname{tg} x = x$ методами деления пополам, простых итераций, Ньютона и секущих. С помощью метода Ньютона найти все корни комплекснозначного полинома $z^3 - 1$, определить области притяжения к этим корням.

Метод Ньютона для систем уравнений

Необходимо найти решение нелинейной системы уравнений $f(x) = 0$ или

$$f_k(x_1, x_2, \dots, x_n) = 0 \quad 1 \leq k \leq n$$

Если известно некоторое приближение $x^{(n)}$ к корню x^* , то вводя приращение $\delta x = x^* - x^{(n)}$

$$f(x^{(n)} + \delta x) = 0 \approx f(x^{(n)}) + J^{(n)} \delta x \Rightarrow \sum_{i=1}^n \frac{\partial f_k(x^{(n)})}{\partial x_i} \Delta x_i^{(n)} = -f_k(x^{(n)})$$

$1 \leq k \leq n$, т.е. система линейных уравнений на $\delta x^{(n)} = x^{(n+1)} - x^{(n)}$.

$$x_i^{(n+1)} = x_i^{(n)} - \left[\left(J^{(n)} \right)^{-1} \right]_{ij} f_j(x^{(n)})$$

В малой окрестности корня итерации сходятся, если $\det J = \det \left[\frac{\partial f}{\partial x} \right] \neq 0$, причем сходимость квадратичная.

Пример решения системы двух уравнений

$$\begin{cases} f(x, y) = 0 \\ \varphi(x, y) = 0 \end{cases}$$

Пусть x_0, y_0 - начальные приближения, поправки h и k : $x^* = x_0 + h$, $y^* = y_0 + k$.

Разлагая в ряд Тейлора, имеем систему линейных уравнений на поправки $h_1 = x_1 - x_0$ и $k_1 = y_1 - y_0$:

$$\begin{cases} f(x_0, y_0) + h_1 \left(\frac{\partial f}{\partial x} \right)_0 + k_1 \left(\frac{\partial f}{\partial y} \right)_0 = 0 \\ \varphi(x_0, y_0) + h_1 \left(\frac{\partial \varphi}{\partial x} \right)_0 + k_1 \left(\frac{\partial \varphi}{\partial y} \right)_0 = 0 \end{cases}$$

Поправка на n -ом шаге:

$$\begin{cases} h_{n+1} = x_{n+1} - x_n = \frac{f'_y(n) \varphi(n) - \varphi'_y(n) f(n)}{f'_x(n) \varphi'_y(n) - f'_y(n) \varphi'_x(n)} \\ k_{n+1} = y_{n+1} - y_n = \frac{\varphi'_x(n) f(n) - f'_x(n) \varphi(n)}{f'_x(n) \varphi'_y(n) - f'_y(n) \varphi'_x(n)} \end{cases}$$

Калиткин Н. Н.: Численные методы

Смирнов С. В.: ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ, Часть I
<http://www.phys.nsu.ru/smirnov/ovf1.pdf>

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ
<http://www.ict.nsc.ru/matmod/files/textbooks/SharyNuMeth.pdf>

Методы Мюллера, Стеффенсена:
<https://arxiv.org/pdf/1708.01144.pdf>

Интерполирование. Интерполяционные формулы
Лагранжа и Ньютона.

Точность интерполяционных формул. Интерполяция
кубическими сплайнами

Чеховской Игорь Сергеевич

Расстояния в функциональных пространствах

На множестве Y , образованном элементами произвольной природы, **расстоянием (метрикой)** называется функция dist с неотрицательными вещественными значениями, удовлетворяющая для любых $f, g, h \in Y$ условиям:

1. $\text{dist}(f, g) = 0 \Leftrightarrow f = g$,
2. $\text{dist}(f, g) = \text{dist}(g, f)$ - симметричность,
3. $\text{dist}(f, h) \leq \text{dist}(f, g) + \text{dist}(g, h)$ - неравенство треугольника.

Примеры:

- равномерное (чебышёвское) расстояние: $\max_{x \in [a, b]} |f(x) - g(x)|$
- интегральное расстояние: $\int_a^b |f(x) - g(x)| dx$
- среднеквадратичное расстояние: $\sqrt{\int_a^b (f(x) - g(x))^2 dx}$

Отметим, что разные расстояния не вполне эквивалентны друг другу в том смысле, что сходимость последовательности функций к какому-то пределу относительно одного из этих расстояний не обязательно влечёт сходимость относительно другого.

Понятие интерполяции

Интерполяция – построение аналитического выражения функции по ее таблице значений и нахождение промежуточных значений (восстановление или доопределение функции).

Частный случай **аппроксимации** – задачи построения функции, близкой к заданной функции в некоторой норме.

Пусть $y(x)$ – некоторая функция, для которой известна лишь таблица ее значений:

$$\begin{cases} y(x_0) = y_0 \\ y(x_1) = y_1 \\ \dots\dots\dots \\ y(x_n) = y_n \end{cases}$$

$P_n(x)$ - любая функция, принимающая в заданных точках заданные значения. В общем случае бесконечное количество функций $P_n(x)$.

Примеры применения: экспериментальные данные, аналитическое выражение $y(x)$ очень сложное.

Задача алгебраической интерполяции:

Для данных **различных** значений $x = x_0, x_1, \dots, x_n$ и $y = y_0, y_1, \dots, y_n$ найти алгебраический полином $P_n(x)$ степени n , удовлетворяющий условиям:

$$\begin{cases} P_n(x_0) = y_0 \\ P_n(x_1) = y_1 \\ \dots\dots\dots \\ P_n(x_n) = y_n \end{cases}$$

Точки $x = x_0, x_1, \dots, x_n$ – узлы интерполяции,

$P_n(x)$ – интерполяционный полином,

формулы для нахождения $P_n(x)$ – интерполяционные формулы.

Формулировка задачи

Ищем $P_n(x)$ в виде полинома n -ой степени с неизвестными коэффициентами $a_0, \dots, a_n$:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

Используя условия

$$\begin{cases} P_n(x_0) = a_0 + a_1x_0 + a_2x_0^2 + \dots + a_nx_0^n = y_0 \\ P_n(x_1) = a_0 + a_1x_1 + a_2x_1^2 + \dots + a_nx_1^n = y_1 \\ \dots\dots\dots \\ P_n(x_n) = a_0 + a_1x_n + a_2x_n^2 + \dots + a_nx_n^n = y_n, \end{cases}$$

получим систему $n + 1$ линейных алгебраических уравнений (СЛАУ) с $n + 1$ неизвестными. Решение СЛАУ существует и единственно, так как матрица системы – матрица Вандермонда, ее определитель $\prod_{0 \leq i, j \leq n} (x_j - x_i)$ – ненулевой при $x_i \neq x_j, i \neq j$.

Решив СЛАУ, получим интерполяционный полином $P_n(x)$.

Пример:

$x_0 = 0, x_1 = 1, x_2 = 2$ и $y_0 = 1, y_1 = 1, y_2 = 3$.

Интерполяционный полином $P_n(x) = 1 - x + x^2$

Проблемы:

- нужно решать СЛАУ специального вида
- полученная СЛАУ чувствительна к ошибкам округления при решении
 - **плохо обусловлена.**

Интерполяционная формула Лагранжа

Ищем

$$P_n(x) = \sum_{i=0}^n y_i \varphi_i(x),$$

где $\varphi_i(x)$ – полином степени n такой, что $\varphi_i(x_j) = \delta_{ij}$. Каковы $\varphi_i(x)$? Имеет место

$$\varphi_i(x) = \Phi_i(x - x_0)(x - x_1) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n).$$

Подставим $x = x_i$:

$$\Phi_i(x) = \frac{1}{(x_i - x_0)(x_i - x_1) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)}.$$

Интерполяционный полином в форме Лагранжа:

$$P_n(x) = \sum_{i=0}^n y_i \frac{\prod_{j \neq i} (x - x_j)}{\prod_{j \neq i} (x_i - x_j)}$$

$$P_n(x) = \sum_{i=0}^n y_i \frac{\omega_n(x)}{(x - x_i) \omega'_n(x_i)}, \quad \omega_n(x) = \prod_j (x - x_j), \quad \omega'_n(x_i) = \prod_{j \neq i} (x_i - x_j).$$

$n = 1$ (прямая):

$$P_n(x) = \frac{x-b}{a-b}y_0 + \frac{x-a}{a-b}y_1$$

$n = 2$ (парабола):

$$P_n(x) = \frac{(x-b)(x-c)}{(a-b)(a-c)}y_0 + \frac{(x-a)(x-c)}{(b-a)(b-c)}y_1 + \frac{(x-a)(x-b)}{(c-a)(c-b)}y_2$$

Недостаток формулы Лагранжа: необходимость перевычисления всего полинома при смене набора узлов интерполяции (например, при добавлении узлов).

Как преодолеть этот недостаток?

Определим *разделенные разности* следующим образом:

$$\begin{aligned}y(x_i, x_j) &= \frac{y_i - y_j}{x_i - x_j}, \\y(x_i, x_j, x_k) &= \frac{y(x_i, x_j) - y(x_j, x_k)}{x_i - x_k}, \\y(x_i, x_j, x_k, x_m) &= \frac{y(x_i, x_j, x_k) - y(x_j, x_k, x_m)}{x_i - x_m} \\&\dots \quad \dots \quad \dots\end{aligned}$$

Если функция y является n раз дифференцируемой, то

$$y[x_0, \dots, x_n] = \frac{y^{(n)}(\xi)}{n!}$$

для некоторой точки ξ в интервале (x_0, x_n) .

Пример с 4 узлами (разделенная разность 3-го порядка):

x_0	y_0	$y(x_0, x_1)$		
x_1	y_1	$y(x_1, x_2)$	$y(x_0, x_1, x_2)$	
x_2	y_2	$y(x_2, x_3)$	$y(x_1, x_2, x_3)$	$y(x_0, x_1, x_2, x_3)$
x_3	y_3			

Интерполяционный полином в форме Ньютона

Ищем полином n -ой степени в виде:

$$P_n(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \dots + a_n(x-x_0)(x-x_1)\dots(x-x_{n-1})$$

Определим коэффициенты:

$$y_0 = P_n(x_0) = a_0,$$

$$y_1 = P_n(x_1) = y_0 + a_1(x_1 - x_0) \Rightarrow a_1 = \frac{y_1 - y_0}{x_1 - x_0} = y(x_0, x_1)$$

$$y_2 = P_n(x_2) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) \Rightarrow$$

$$\Rightarrow a_2 = \frac{1}{x_2 - x_0} \left(\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0} \right) = y(x_0, x_1, x_2)$$

... ..

Формула Ньютона:

$$\begin{aligned} P_n(x) &= y_0 + y(x_0, x_1)(x - x_0) + y(x_0, x_1, x_2)(x - x_0)(x - x_1) + \dots \\ &\dots + y(x_0, x_1, \dots, x_n)(x - x_0)(x - x_1)\dots(x - x_{n-1}) \end{aligned}$$

Вычисление по схеме Горнера:

$$\begin{aligned} P_n(x) &= y_0 + \dots \\ &\dots + (x - x_0) \cdot [y(x_0, x_1) + \dots \\ &\dots + (x - x_1) \cdot [y(x_0, x_1, x_2) + \dots \\ &\dots \\ &\dots + (x - x_{n-1}) \cdot y(x_0, x_1, \dots, x_n)] \dots] \end{aligned}$$

Удобнее, если узлы не меняются.

Погрешность полинома Ньютона

$P_n(x)$ - полином степени n . Представим погрешность в виде $y(x) - P_n(x) = \omega_n(x)r(x)$, где $\omega_n(x) = \prod_{i=0}^n (x - x_i)$.

Вспомогательная функция $q(\xi) = y(\xi) - P_n(\xi) - \omega_n(\xi)r(x)$ имеет $n + 2$ нуля: $\xi = x_0, x_1, \dots, x_n, x$.

Пусть $y(x)$ имеет $n + 1$ непрерывную производную, тогда

$$q^{(n+1)}(\xi) = y^{(n+1)}(\xi) - (n + 1)!r(x).$$

Между двумя нулями гладкой функции лежит нуль ее производной.

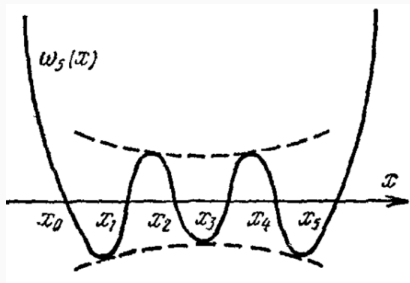
Поэтому между крайними из $n + 2$ нулей функции лежит нуль $n + 1$ -й производной, т. е. $q^{(n+1)}(\xi^*) = 0$, тогда $r(x) = \frac{y^{(n+1)}(\xi^*)}{(n+1)!}$

Итоговая оценка погрешности:

$$|y(x) - P_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega_n(x)|,$$

где $M_{n+1} = \max |y^{(n+1)}(\xi)|$, а ξ лежит между наименьшим и наибольшим из значений $x, x_0, x_1, \dots, x_n$.

Погрешность при равномерном расположении узлов



Пусть $n = 2k + 1$. Оценим погрешность в центральном интервале. Экстремум $\omega_n(x)$:

$$\begin{aligned} \left[\frac{h}{2} \frac{3h}{2} \cdots \frac{(2k+1)h}{2} \right]^2 &= \\ &= \left[\frac{(2k+1)! h^{k+1}}{k! 2^{2k+1}} \right]^2 \end{aligned}$$

Пользуясь формулой Стирлинга $p! \approx \sqrt{2\pi p} (p/e)^p$ имеем:

$$|y(x) - P_n(x)| < \sqrt{2/\pi n} M_{n+1} (h/2)^{n+1} = C h^{n+1}$$

Т.е. полином Ньютона имеет погрешность $O(h^{n+1})$, если $y(x)$ имеет $n+1$ непрерывную производную.

Формула Лагранжа:

$$2n^2 + 2n = O(n^2)$$

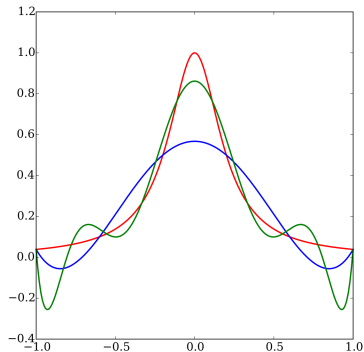
Формула Ньютона:

$$n(n+1)/2 = O(n^2)$$

– подготовительный этап с построением таблицы конечных разностей, который выполняется один раз.

Вычисление формулы Ньютона по схеме Горнера:

$$3n = O(n)$$



$$f(x) = \frac{1}{1 + 25x^2}$$
$$x_i = \frac{2i}{n} - 1, \quad i \in \{0, 1, \dots, n\}$$

Погрешность растет неограниченно с увеличением числа узлов интерполяции!

Почему так происходит?

Как этого можно избежать?

Полиномы Чебышёва

Полиномы Чебышёва – это семейство алгебраических полиномов, обозначаемых $T_n(x)$. Тригонометрическое представление:

$$T_n(x) = \cos(n \arccos x),$$

$$x \in [-1, 1], n = 0, 1, 2, \dots$$

Свойство:

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).$$

Явные алгебраические выражения
для полиномов Чебышёва:

$$T_0(x) = 1,$$

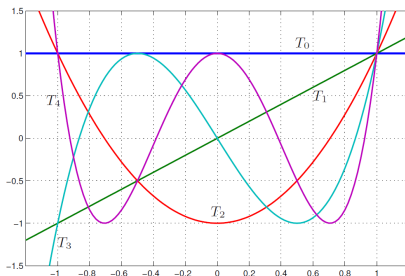
$$T_1(x) = x,$$

$$T_2(x) = 2x^2 - 1,$$

$$T_3(x) = 4x^3 - 3x,$$

$$T_4(x) = 8x^4 - 8x^2 + 1,$$

$$T_5(x) = 16x^5 - 20x^3 + 5x,$$



Графики первых полиномов Чебышёва на интервале $[-1.2, 1.2]$.

Предложение. Среди полиномов степени $n, n \geq 1$, со старшим коэффициентом, равным 1, полином $\tilde{T}_n(x) := 2^{1-n} T_n(x)$ имеет на интервале $[-1, 1]$ наименьшее равномерное отклонение от нуля. Иными словами, если $Q_n(x)$ – полином степени n со старшим коэффициентом 1, то

$$\max_{x \in [-1, 1]} |Q_n(x)| \geq \max_{x \in [-1, 1]} |\tilde{T}_n(x)| = 2^{1-n}.$$

Полиномы $\tilde{T}_n(x) = 2^{1-n} T_n(x)$ имеющие единичный старший коэффициент, называют приведёнными полиномами Чебышёва.

Предложение. Если $T_n(x)$ — n -ый полином Чебышёва, то полином переменной y , задаваемый как

$$2^{1-2n}(b-a)^n \cdot T_n\left(\frac{2y-(b+a)}{b-a}\right),$$

имеет старший коэффициент 1 и на интервале $[a, b]$ равномерно наименее уклоняется от нуля среди всех полиномов степени n со старшим коэффициентом 1.

Каково наиболее выгодного расположения узлов $\{x_0, x_1, \dots, x_n\}$ алгебраического интерполянта степени n на интервале $[a, b]$?

Возьмём эти узлы как

$$x_k = \frac{1}{2}(b+a) + \frac{1}{2}(b-a) \cos \left(\frac{(2k+1)\pi}{2(n+1)} \right), \quad k = 0, 1, \dots, n,$$

т. е. корнями полинома, который получается в результате замены переменных из полинома Чебышёва $(n+1)$ -ой степени $T_{n+1}(x)$.

Тогда соответствующий полином

$$\omega_n(x) = (x - x_0)(x - x_1) \dots (x - x_n),$$

который фигурирует в формуле для остаточного члена интерполяции, совпадёт с полиномом $(n+1)$ -ой степени из предложения. При этом $\omega_n(x)$ будет иметь наименьшее отклонение от нуля на $[a, b]$ в равномерной (чебышёвской) метрике, и в смысле этой метрики погрешность интерполирования при прочих равных условиях будет наименьшей возможной. Узлы интерполяции x_k называют чебышёвскими узлами на интервале $[a, b]$.

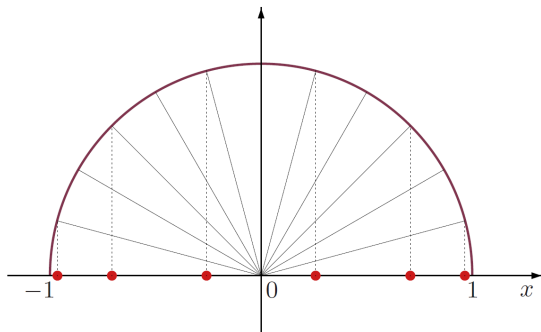


Иллюстрация расположения корней полинома Чебышёва шестой степени.

Провести интерполяционный полином $P_n(x)$ через точки

$$x_i = \frac{2i}{n} - 1, \quad f(x_i) = \frac{1}{1 + 25x_i^2}, \quad i \in \{0, 1, \dots, n\}$$

при $n = 3, \dots, 10$ (пример Рунге). Нарисовать графики $|P_n(x) - f(x)|$, объяснить рост ошибки интерполяции. Использовать корни полиномов Чебышёва в качестве узлов интерполяции, чтобы уменьшить ошибку интерполяции.

Пусть интервал $[a, b]$ разбит на подынтервалы $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$, причем $a = x_0$, $x_n = b$.

Сплайн на $[a, b]$ – это функция, непрерывная вместе со своими производными вплоть до некоторого порядка на $[a, b]$, и которая на каждом $[x_{i-1}, x_i]$ является некоторым полиномом.

Степень сплайна – максимальная из степеней полиномов, задающих сплайн на подынтервалах $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$.

Дефект сплайна – разность между степенью сплайна и наивысшим порядком производной сплайна, непрерывной на $[a, b]$ (то, сколько сплайну не хватает до “полноценного полинома”).

Простейшие сплайны – степень 1, дефект 1 (кусочно-линейная функция).

Исторически сплайн (spline) – гибкая металлическая линейка, которая применялась для решения задачи геометрической интерполяции.



Сплайны степени 1, дефекта 1 однозначно строятся по $x_0, x_1, \dots, x_n$ и $f_0, f_1, \dots, f_n$.

Далее рассматриваем лишь сплайны дефекта 1, их вполне достаточно во многих приложениях.

Пусть p – степень сплайна, дефект равен 1. Чтобы построить сплайн, нужно знать $(p + 1)n$ коэффициентов полиномов на n подынтервалах.

Имеем

- $p(n - 1)$ условий непрерывности в узлах для сплайна и его производных до $(p - 1)$ -го порядка;
- $(n + 1)$ условие интерполяции.

Не хватает $(p + 1)n - (p(n - 1) + n + 1) = p - 1$ условий!

Обычно задают их на границах отрезка $[a, b]$. Если p - четное, то требуется $(p - 1)$ - нечетное число доп. условий, что приводит к асимметрии задачи. Наиболее популярны сплайны нечетной степени, особенно кубические.

Кубический интерполяционный сплайн на отрезке $[a, b]$ с сеткой $a = x_0 < x_1 < \dots < x_n = b$ – это функция $S(x)$, удовлетворяющая условиям:

- $S(x)$ – полином 3-й степени на каждом из $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$;
- $S(x) \in C^2[a, b]$;
- $S(x_i) = f_i$ для $i = 0, 1, \dots, n$.

Для однозначного построения нужны еще $p - 1 = 3 - 1 = 2$ доп. условия.
Возможные способы задания:

- даны $S'(a) = f'_0$ и $S'(b) = f'_n$;
- даны $S''(a) = f''_0$ и $S''(b) = f''_n$;
- условие периодичности $S'(a) = S'(b)$, $S''(a) = S''(b)$.

Рассмотрим

$$S''(a) = S''(x_0) = c_0, \quad S''(b) = S''(x_n) = c_n.$$

Будем искать сплайн в виде

$$S(x) = a_{i-1} + b_{i-1}(x - x_{i-1}) + c_{i-1} \frac{(x - x_{i-1})^2}{2} + d_{i-1} \frac{(x - x_{i-1})^3}{6}$$

для $x \in [x_{i-1}, x_i]$, $i = 1, 2, \dots, n$. При этом $S''(x_i) = c_i$, $i=0, 1, \dots, n-1$.

$S''(x)$ – линейная функция на $[x_{i-1}, x_i]$, определяется однозначно:

$$S''(x) = c_{i-1} \frac{x_i - x}{h_i} + c_i \frac{x - x_{i-1}}{h_i}, \quad x \in [x_{i-1}, x_i], \quad i = 1, 2, \dots, n$$

С учетом $S(x_{i-1}) = f_{i-1}$, $S(x_i) = f_i$, $i = 1, 2, \dots, n$, дважды проинтегрировав, получим:

$$\begin{aligned} S(x) &= f_{i-1} \frac{x_i - x}{h_i} + f_i \frac{x - x_{i-1}}{h_i} + \\ &+ c_{i-1} \frac{(x_i - x)^3 - h_i^2(x_i - x)}{6h_i} + c_i \frac{(x - x_{i-1})^3 - h_i^2(x - x_{i-1})}{6h_i}. \end{aligned}$$

Используя условие непрерывности

$$S'(x_i - 0) = S'(x_i + 0), \quad i = 1, 2, \dots, n - 1,$$

получим СЛАУ:

$$\begin{cases} \frac{h_i}{6} c_{i-1} + \frac{h_i + h_{i+1}}{3} c_i + \frac{h_{i+1}}{6} c_{i+1} = \frac{f_{i+1} - f_i}{h_{i+1}} - \frac{f_i - f_{i-1}}{h_i}, & i = 1, 2, \dots, n - 1, \\ c_0, c_n - \text{заданы.} \end{cases}$$

Матрица системы – **трехдиагональная**, является матрицей с **диагональным преобладанием**, т.е.

$$\forall i = 1, 2, \dots, n : |a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}|.$$

Признак Адамара неособенности матриц: матрица с диагональным преобладанием неособенна ($\text{Det} \neq 0$).

Таким образом, решение СЛАУ на c_i существует и единственно.

Может быть найдено **методом прогонки**, что требует $\mathcal{O}(n)$ арифметических операций.

Теорема. Пусть $f(x) \in C^p[a, b]$, $p = 1, 2, 3$, а $S(x)$ – ее интерполяционный кубический сплайн с краевыми условиями на S'' . Тогда

$$\max_{x \in [a, b]} |f^{(k)}(x) - S^{(k)}(x)| = \mathcal{O}(h^{p-k}), \quad k = 0, \dots, p.$$

Этот результат также справедлив для других типов краевых условий на S .

Интерполяционный кубический сплайн $S(x)$, удовлетворяющий условию

$$S''(a) = S''(b) = 0,$$

называется **естественным сплайном**.

Он минимизирует функционал

$$\mathcal{E}(\phi) = \int_a^b (\phi''(x))^2 dx,$$

задающий энергию упругой деформации гибкой балки (линейки), закрепленной в $x_0, x_1, \dots, x_n$ и принимающий форму $\phi(x) \in C^2[a, b]$.

Калиткин Н. Н.: Численные методы

Смирнов С. В.: ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ, Часть I

<http://www.phys.nsu.ru/smirnov/ovf1.pdf>

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

<http://www.ict.nsc.ru/matmod/files/textbooks/SharyNuMeth.pdf>

Численное интегрирование

Чеховской Игорь Сергеевич

Формулировка задачи

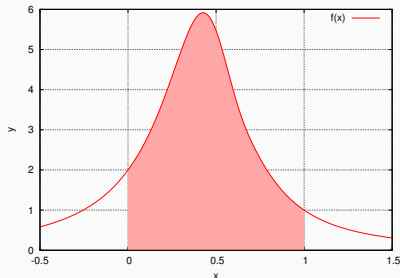
Необходимо вычислить определенный интеграл:

$$I = \int_a^b f(x) dx$$

- Первообразная функции $f(x)$ не выражается через элементарные функции
- Выражение для первообразной слишком сложное.
- Функция $f(x)$ задана таблицей.

Пример: Вычислить интеграл

$$\int_0^1 \frac{1 + e^{-\left(\frac{x}{1-x}\right)^2}}{3x^2 - 3x + 1} dx$$



Интерполяционные квадратурные формулы

$$I = \int_a^b f(x) dx$$

Заменим $f(x)$ на аппроксимирующую функцию $\varphi(x)$, для которой интеграл легко вычисляется.

$\varphi(x)$ - интерполяционный полином Лагранжа,

$\varphi_i(x)$ - интерполяционный коэффициент полинома Лагранжа:

$$f(x) = \varphi(x) + r(x) = \sum_{i=0}^n f(x_i) \varphi_i(x) + r(x)$$

x_i - узлы сетки, $r(x)$ – остаточный член аппроксимации.

$$I = \sum_{i=0}^n c_i f(x_i) + R, \quad c_i = \int_a^b \varphi_i(x) dx, \quad R = \int_a^b r(x) dx$$

c_i – веса, R – погрешность или остаточный член формулы (остаток).

Интерполяционные квадратурные формулы на равномерной сетке – **формулы Ньютона-Котеса**.

Формула левых (правых) прямоугольников

Заменим подынтегральную функцию на константу $f(\bar{x})$ (интерполяционный полином нулевой степени) на интервале $[a, b]$, где $\bar{x} = a$:

$$I = \int_a^b f(x)dx \approx (b - a)f(a)$$

Оценка погрешности: $f(x) = f(a) + (x - a)f'(a) + \dots$

$$R \approx \frac{1}{2}(b - a)^2 f'(\bar{x})$$

При $\bar{x} = b$ – формула правых прямоугольников.

Совпадает с определением интеграла Римана.

Порядок аппроксимации “простой” формулы – 2.

Обобщенная (составная) формула левых (правых) прямоугольников

На подробной неравномерной сетке $a = x_0 < x_1 < \dots < x_n = b$
с известными значениями $f(x)$ в узлах сетки:

$$I \approx \sum_{i=1}^n (x_i - x_{i-1}) f(x_{i-1}), \quad R \approx \frac{1}{2} \sum_{i=1}^n (x_i - x_{i-1})^2 f'(x_{i-1}),$$

Равномерная сетка: $h = (b - a)/n$, узлы $x_i = a + ih$, $i = 0, 1, \dots, n$:

$$I \approx h \sum_{i=1}^n f_i, \quad R \approx \frac{h}{2} \int_a^b f'(x) dx, \quad f_i = f(x_i)$$

Порядок аппроксимации (точности) составной формулы – 1.

Алгебраический порядок точности – максимальная степень полинома, для которого квадратурная формула является точной – равняется 0.

Формула средних прямоугольников

Заменим подынтегральную функцию на константу $f(\bar{x})$ на интервале $[a, b]$, где $\bar{x} = (a + b)/2$:

$$I = \int_a^b f(x) dx \approx (b - a) f\left(\frac{a + b}{2}\right)$$

Оценка погрешности: $f(x) = f(\bar{x}) + (x - \bar{x})f'(\bar{x}) + (x - \bar{x})^2 f''(\bar{x})/2 + \dots$

$$R = \int_a^b f(x) dx - (b - a) f(\bar{x}) \approx \frac{1}{24} (b - a)^3 f''(\bar{x}),$$

поскольку

$$\int_a^b \left(x - \frac{a + b}{2}\right) dx = \int_{-\frac{b-a}{2}}^{\frac{b-a}{2}} t \, dt = 0.$$

Порядок аппроксимации “простой” формулы – 3.

Обобщенная (составная) формула средних прямоугольников

Неравномерная сетка: $a = x_0 < x_1 < \dots < x_n = b$ с известными значениями $f(x)$ в середине каждого интервала:

$$I \approx \sum_{i=1}^n (x_i - x_{i-1}) f(\bar{x}_i), \quad R \approx \frac{1}{24} \sum_{i=1}^n (x_i - x_{i-1})^3 f''(\bar{x}_i), \quad \bar{x}_i = \frac{x_i + x_{i-1}}{2}$$

Равномерная сетка: $h = (b - a)/n$, узлы $x_i = a + ih$, $i = 0, 1, \dots, n$:

$$I \approx h \sum_{i=1}^n f_i, \quad R \approx \frac{h^2}{24} \int_a^b f''(x) dx, \quad f_i = f(\bar{x}_i)$$

Порядок аппроксимации составной формулы – 2.

Алгебраический порядок – 1.

Формула трапеций

Заменяем $f(x)$ на полином Лагранжа первой степени.

Узлы $x_0 = a$, $x_1 = b$, $y_0 = f(a)$, $y_1 = f(b)$.

$$\varphi(x) = \frac{x-b}{a-b}f(a) + \frac{x-a}{b-a}f(b)$$

Тогда

$$I = \int_a^b f(x)dx \approx \frac{f(a) + f(b)}{2} [b - a]$$

Остаточный член интерполяции $r(x)$:

$$r(x) = \frac{f''(\xi)}{2}(x-a)(x-b)$$

где $\xi \in [a, b]$. Погрешность формулы трапеций:

$$R = \int_a^b r(x)dx = -\frac{f''(\xi)}{12}(b-a)^3$$

Порядок аппроксимации "простой" формулы – 3.

Обобщенная формула трапеций

Неравномерная сетка: $a = x_0 < x_1 < \dots < x_n = b$.

$$I = \int_a^b f(x) dx \approx \frac{1}{2} \sum_{i=1}^n (x_i - x_{i-1})(f_i + f_{i-1})$$

$$R \approx -\frac{1}{12} \sum_{i=1}^n (x_i - x_{i-1})^3 f''(\xi_i)$$

Равномерная сетка: $h = (b - a)/n$, узлы $x_i = a + ih$, $i = 0, 1, \dots, n$:

$$I = \int_a^b f(x) dx \approx h \left(\frac{1}{2} f_0 + f_1 + \dots + \frac{1}{2} f_n \right)$$

$$R \approx -\frac{1}{12} \sum_{i=1}^n h^3 f''(\xi_i) \approx -\frac{1}{12} h^2 \int_a^b f''(x) dx$$

Мажорантная оценка погрешности:

$$|R| \leq \frac{b-a}{12} h^2 M_2, \quad M_2 = \max |f''(\xi)|, \quad \xi \in [a, b]$$

Порядок аппроксимации составной формулы – 2.

Алгебраический порядок – 1.

Формула Симпсона (формула парабол)

Заменяем $f(x)$ на интервале $[a, b]$ на полином Лагранжа второй степени.

Узлы $x_0 = a$, $x_1 = \frac{a+b}{2}$, $x_2 = b$, $y_0 = f(a)$, $y_1 = f\left(\frac{a+b}{2}\right)$, $y_2 = f(b)$.

Оценка интеграла по формуле Симпсона:

$$I = \int_a^b f(x)dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

Оценка погрешности формулы Симпсона:

$$R \leq \frac{(b-a)^5}{2880} M_4, \quad M_4 = \max |f^{(IV)}(\xi)|, \quad \xi \in [a, b]$$

Порядок аппроксимации "простой" формулы – 5 (повышенный порядок).

Обобщенная формула Симпсона

Для более точного вычисления интеграла разобьем отрезок $[a, b]$ на $N = 2n$ частей. $h = (b - a)/N$, $x_i = a + ih$, $i = 0, 1, \dots, N = 2n$.

$$\begin{aligned} I = \int_a^b f(x) dx &\approx \frac{h}{3} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_{2i}) + 4 \sum_{i=1}^n f(x_{2i-1}) + f(x_N) \right] = \\ &= \frac{h}{3} \sum_{i=1,2}^{N-1} [f(x_{i-1}) + 4f(x_i) + f(x_{i+1})] \end{aligned}$$

Здесь индекс $i = 1, 2$ означает, что i меняется от 1 с шагом 2.

Погрешность метода Симпсона:

$$R \leq \frac{(b-a)h^4}{180} M_4, \quad M_4 = \max |f^{(IV)}(\xi)|, \quad \xi \in [a, b]$$

Порядок аппроксимации составной формулы – 4.

Алгебраический порядок – 3.

Контрольный вопрос.

Квадратурные формулы Гаусса

Полиномы Лежандра (через формулу Родрига):

$$L_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n, \quad n = 0, 1, 2, \dots$$

$$L_0(x) = 1,$$

$$L_1(x) = x,$$

$$L_2(x) = \frac{1}{2} (3x^2 - 1),$$

$$L_3(x) = \frac{1}{2} (5x^3 - 3x),$$

$$L_4(x) = \frac{1}{8} (35x^4 - 30x^2 + 3),$$

$$L_5(x) = \frac{1}{8} (63x^5 - 70x^3 + 15x),$$

....

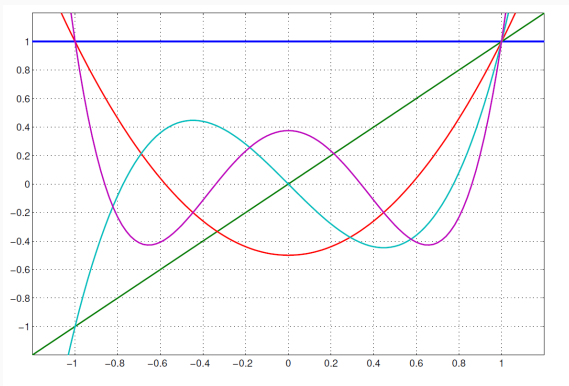
Эти полиномы с точностью до множителя совпадают с результатом ортогонализации Грама-Шмидта на интервале $[-1, 1]$, если последовательно применять процесс ортогонализации к степеням $1, x, x^2, x^3, \dots$.

Т. о., полиномы Лежандра образуют ортонормированный базис.

Квадратурные формулы Гаусса

Аналогично полиномам Чебышёва, нули полиномов Лежандра тоже сгущаются к концам интервала $[-1, 1]$. Кроме того, нули полинома Лежандра $L_n(x)$ перемежаются с нулями полинома $L_{n+1}(x)$. Наконец, справедливо рекуррентное представление

$$(n + 1)L_{n+1}(x) = (2n + 1)xL_n(x) - nL_{n-1}(x).$$



Графики первых полиномов Лежандра на интервале $[-1.2, 1.2]$.

Предложение 1. Все нули полиномов Лежандра $L_n(x)$ вещественные, простые и находятся на интервале $[-1, 1]$.

Предложение 2. Среди всех полиномов степени n , $n \geq 1$, со старшим коэффициентом, равным 1, полином $\tilde{L}_n(x)$ имеет на интервале $[-1, 1]$ наименьшее *среднеквадратичное* отклонение от нуля. Иными словами, если $Q_n(x)$ - полином степени n со старшим коэффициентом 1, то

$$\int_{-1}^1 (Q_n(x))^2 dx \geq \int_{-1}^1 (\tilde{L}_n(x))^2 dx.$$

Из Предложения 2 следует, что для достижения наименьшей погрешности алгебраической интерполяции в среднеквадратичном смысле узлы интерполяции следует брать корнями соответствующего полинома Лежандра или же полинома, который получается из него линейным преобразованием на необходимый нам интервал.

Квадратурные формулы Гаусса

$$\int_a^b f(x)dx \approx \sum_{i=1}^n A_i f(t_i)$$

A_i , t_i – величины, получаемые из условия максимизации алгебраического порядка квадратурной формулы, причем t_i – корни полиномов Лежандра.

Элементы формулы Гаусса

n	I	t_i	A_i
1	1	0	2
2	1;2	$\mp 0,57735027$	1
3	1;3 2	$\mp 0,77459667$ 0	0,55555556 0,88888889
4	1;4 2;3	$\mp 0,86113631$ $\mp 0,33998104$	0,34785484 0,65214516
5	1;5 2;4 3	$\mp 0,906113631$ $\mp 0,53846931$ 0	0,23692688 0,47862868 0,56888889
6	1;6 2;5 3;4	$\mp 0,93246951$ $\mp 0,66120939$ $\mp 0,23861919$	0,17132450 0,36076158 0,46791394
7	1;7 2;6 3;5 4	$\mp 0,94910791$ $\mp 0,74153119$ $\mp 0,40584515$ 0	0,12948496 0,27970540 0,38183006 0,41795918
8	1;8 2;7 3;6 4;5	$\mp 0,96028986$ $\mp 0,79666648$ $\mp 0,52553242$ $\mp 0,18343464$	0,10122854 0,22238104 0,31370664 0,36268379

https://dep_vipm.pnzgu.ru/files/dep_vipm.pnzgu.ru/books/dobrynina_kvadraturnye_i_kubaturnye_formuly.pdf

- способ повышения порядка точности квадратурных формул.

$$I = F(h) + R(h) = F(h) + Ch^p + \mathcal{O}(h^{p+1})$$

h – шаг равномерной сетки, $F(h)$ – квадратурная формула, $R(h)$ – погрешность, p – порядок точности квадратурной формулы.

Введем новую сетку с шагом rh :

$$I = F(rh) + C(rh)^p + \mathcal{O}(h^{p+1})$$

Тогда

$$Ch^p = R = \frac{F(h) - F(rh)}{r^p - 1} + \mathcal{O}(h^{p+1})$$

- **апостериорная** оценка точности,

$$I = \frac{F(h)r^p - F(rh)}{r^p - 1} + \mathcal{O}(h^{p+1})$$

- формула для повышения порядка точности. Если порядок точности неизвестен *a priori*, добавляем уравнение (**процесс Эйткена**)

$$I = F(r^2h) + C(r^2h)^p.$$

Интегралы по бесконечным областям

Подинтегральная функция убывает по экспоненциальному закону (или быстрее) – можем заменить бесконечную область интегрирования на конечную. Для этого оценим отбрасываемый “хвост”:

$$|f(x)| \leq A \exp(-kx) \quad \forall x > x_0 \Rightarrow$$
$$|R| = \left| \int_{x_0}^{\infty} f(x) dx \right| \leq \int_{x_0}^{\infty} A \exp(-kx) = A \exp(-kx_0)$$

Подинтегральная функция убывает по степенному закону – замена, чтобы свести бесконечную область интегрирования к конечной:

$$x = t/(1 - t)$$
$$I = \int_0^{\infty} f(x) dx = \int_0^1 \frac{f(x(t)) dt}{(1 - t)^2}$$

Написать программу для вычисления интеграла произвольной функции на заданном отрезке методами Симпсона и каким-либо методом Гаусса (на выбор). Исследовать порядок аппроксимации на тестовом примере.

Вычислить интегралы с погрешностью не выше 10^{-8} :

$$I_a = \int_0^1 \frac{\sin(\pi x^5)}{x^5(1-x)} dx, \quad I_b = \int_0^\infty e^{-\sqrt{x} + \sin(\frac{x}{10})} dx.$$

https://dep_vipm.pnzgu.ru/files/dep_vipm.pnzgu.ru/books/dobrynina_kvadraturnye_i_kubaturnye_formuly.pdf

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

<http://www.ict.nsc.ru/matmod/files/textbooks/SharyNuMeth.pdf>

Численное дифференцирование

Чеховской Игорь Сергеевич

Необходимо вычислить k -ую производную $y^{(k)}(x)$.

Возможные трудности:

- Выражение для производной $y^{(k)}(x)$ слишком сложное.
- Функция $y(x)$ задана таблицей.
- Решение дифференциальных уравнений при помощи разностных методов.

Выход – численное дифференцирование.

Понятие о численном дифференцировании

Аппроксимация дифференцируемой функции $y(x)$ легко вычисляемой функцией $\varphi(x)$.

Рассмотрим аппроксимацию интерполяционным полиномом в форме Ньютона.

Пусть $\xi_i = x - x_i$, x_i – узлы интерполяции, $y(x_0, x_1, \dots, x_k)$ – разделенные разности.

$$\varphi(x) = y(x_0) + \xi_0 y(x_0, x_1) + \xi_0 \xi_1 y(x_0, x_1, x_2) + \xi_0 \xi_1 \xi_2 y(x_0, x_1, x_2, x_3) + \dots$$

Вычислим производные функции $\varphi(x)$:

$$\begin{aligned}\varphi'(x) &= y(x_0, x_1) + (\xi_0 + \xi_1) y(x_0, x_1, x_2) + \\ &\quad + (\xi_0 \xi_1 + \xi_0 \xi_2 + \xi_1 \xi_2) y(x_0, x_1, x_2, x_3) + \dots \\ \varphi''(x) &= 2y(x_0, x_1, x_2) + 2(\xi_0 + \xi_1 + \xi_2) y(x_0, x_1, x_2, x_3) + \dots\end{aligned}$$

Общий вид k -ой производной функции $\varphi(x)$:

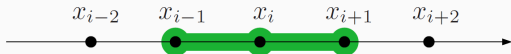
$$\begin{aligned}\varphi^{(k)}(x) = & k! \left[y(x_0, x_1, \dots, x_k) + \left(\sum_{i=0}^k \xi_i \right) y(x_0, x_1, \dots, x_{k+1}) + \right. \\ & + \left(\sum_{\substack{i=0 \\ i > j \geq 0}}^{i=k+1} \xi_i \xi_j \right) y(x_0, x_1, \dots, x_{k+2}) + \\ & + \left. \left(\sum_{\substack{i=0 \\ i > j > l \geq 0}}^{i=k+2} \xi_i \xi_j \xi_l \right) y(x_0, x_1, \dots, x_{k+3}) \dots \right]\end{aligned}$$

Простые приближенные выражения для производных

В рассмотренных выше рядах можно оставлять только несколько слагаемых. Оставляя только первое слагаемое в ряде имеем:

$$\begin{aligned}y'(x) &\approx y(x_0, x_1) = \frac{y(x_0) - y(x_1)}{x_0 - x_1} \\ \frac{1}{2}y''(x) &\approx y(x_0, x_1, x_2) = \frac{1}{x_0 - x_2} \left[\frac{y_0 - y_1}{x_0 - x_1} - \frac{y_1 - y_2}{x_1 - x_2} \right] \\ \frac{1}{k!}y^{(k)}(x) &\approx y(x_0, x_1, \dots, x_k) = \sum_{p=0}^k y_p \prod_{i=0; i \neq p}^k \frac{1}{(x_p - x_i)}\end{aligned}$$

Шаблон формулы численного дифф-я – это множество узлов сетки, входящих в правую часть этой формулы, явным образом либо в качестве аргументов используемых значений функции.



Шаблон формулы второй разностной производной

Примеры формул для производных на равномерной сетке

$$f'(x_i) \approx f_{\bar{x},i} := \frac{f_i - f_{i-1}}{x_i - x_{i-1}} = \frac{f_i - f_{i-1}}{h_i}$$

– разделённая разность назад,

$$f'(x_i) \approx f_{x,i} := \frac{f_{i+1} - f_i}{x_{i+1} - x_i} = \frac{f_{i+1} - f_i}{h_{i+1}}$$

– разделённая разность вперёд.

$$f'(x_i) \approx f_{\bar{x},i} = \frac{f_{i+1} - f_{i-1}}{2h}$$

– формула центральной разности.

$$f'(x_i) \approx \frac{-3f_i + 4f_{i+1} - f_{i+2}}{2h}.$$

$$f'(x_i) \approx \frac{f_{i-2} - 4f_{i-1} + 3f_i}{2h}.$$

Примеры формул для производных на равномерной сетке

Формулы, полученные по четырём точкам, т. е. с применением интерполяционного полинома третьей степени.

Для первой производной:

$$f'(x_i) \approx \frac{1}{6h} (-11f_i + 18f_{i+1} - 9f_{i+2} + 2f_{i+3})$$

$$f'(x_i) \approx \frac{1}{6h} (-2f_{i-1} - 3f_i + 6f_{i+1} - f_{i+2})$$

$$f'(x_i) \approx \frac{1}{6h} (f_{i-2} - 6f_{i-1} + 3f_i + 2f_{i+1})$$

$$f'(x_i) \approx \frac{1}{6h} (-2f_{i-3} + 9f_{i-2} - 18f_{i-1} + 11f_i)$$

Для второй производной:

$$f''(x_i) \approx \frac{1}{h^2} (2f_i - 5f_{i+1} + 4f_{i+2} - f_{i+3})$$

$$f''(x_i) \approx \frac{1}{h^2} (f_{i-1} - 2f_i + f_{i+1})$$

$$f''(x_i) \approx \frac{1}{h^2} (-f_{i-3} + 4f_{i-2} - 5f_{i-1} + 2f_i).$$

Исследуем точность выражений по оценке скорости убывания членов ряда. Пусть x_i – интерполяционные узлы, $0 \leq i \leq n$, и шаг сетки мал. Погрешность вычисления k -ой производной $\approx$ первому отброшенному члену ряда. Т.е.

$$R_n^{(k)}(x) \sim k!y(x_0, x_1, \dots, x_{n+1}) \sim \frac{k!}{(n+1)!}y^{(n+1)}(x)$$

и пропорциональна сумме произведений различных сомножителей ξ_i . Каждое произведение содержит $n+1-k$ сомножителей и в сумме C_{n+1}^k слагаемых. Таким образом,

$$R_n^{(k)}(x) \leq \frac{M_{n+1}}{(n+1-k)!} \max_i |\xi_i|^{n+1-k}, \quad M_{n+1} = \max |y^{(n+1)}|$$

При равномерной сетке $\max_i |\xi_i| < nh$

$$R_n^{(k)}(x) \leq M_{n+1} \left(\frac{enh}{n+1-k} \right)^{n+1-k} = O(h^{n+1-k})$$

Порядок точности формул по отношению к шагу сетки равен числу оставленных в ней слагаемых или равен числу узлов интерполяции минус порядок производной.

Минимальное число узлов сетки для вычисления k -ой производной равно $k + 1$, при этом точность первого порядка по h .

Оценка точности формул с помощью разложения в ряд Тейлора–Маклорена

x_i – узлы равноотстоящие на h . Проверим точность формулы:

$$y_i'' = y''(x_i) = \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + O(h^2)$$

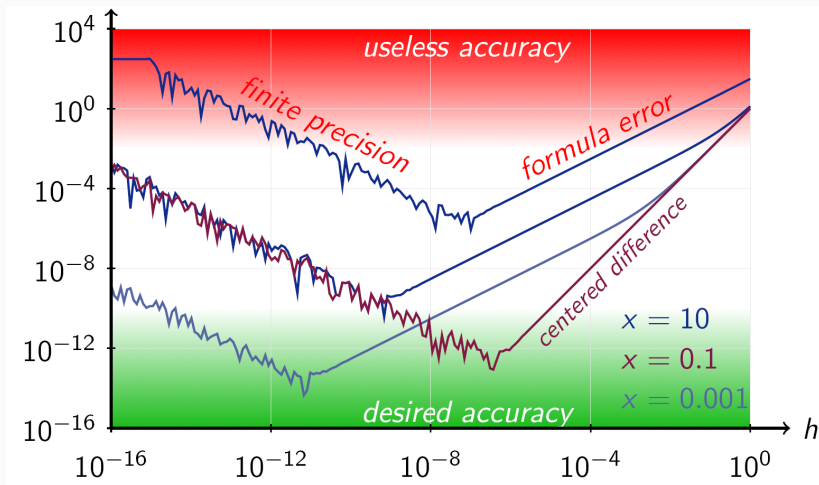
Пусть $y(x)$ имеет непрерывную четвертую производную. Тогда

$$y(x_i \pm h) = y(x_{i\pm 1}) = y_i \pm hy_i' + \frac{h^2}{2}y_i'' \pm \frac{h^3}{6}y_i''' + \frac{h^4}{24}y^{IV}(\eta_{\pm})$$

Подставляя в формулу, имеем:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = y_i'' + \frac{h^2}{24}[y^{IV}(\eta_+) + y^{IV}(\eta_-)]$$

Оценка точности формул с помощью разложения в ряд Тейлора–Маклорена



Почему так происходит?

Полная вычислительная погрешность численного дифференцирования

Предположим, что первая производная функции вычисляется по формуле «разность вперёд»

$$f'(x_i) \approx f_{x,i} = \frac{f_{i+1} - f_i}{h}.$$

Как мы уже знаем, её погрешность

$$|f_{x,i} - f'(x_i)| \leq \frac{M_2 h}{2},$$

где $M_2 = \max_{\xi \in [a,b]} |f''(\xi)|$. Если значения функции вычисляются с ошибками, то вместо точных f_i и f_{i+1} мы получаем их приближённые значения $\tilde{f}_i$ и $\tilde{f}_{i+1}$, такие что

$$|f_i - \tilde{f}_i| \leq \delta \quad \text{и} \quad |f_{i+1} - \tilde{f}_{i+1}| \leq \delta,$$

где δ – предельная абсолютная погрешность вычисления значений функции. Тогда в качестве прикл. значения мы должны взять

$$f'(x_i) \approx \frac{\tilde{f}_{i+1} - \tilde{f}_i}{h}.$$

Предельная полная вычислительная погрешность $E(h, \delta)$:

$$\begin{aligned} E(h, \delta) &= \left| \frac{\tilde{f}_{i+1} - \tilde{f}_i}{h} - f'(x_i) \right| \leq \left| \frac{\tilde{f}_{i+1} - \tilde{f}_i}{h} - \frac{f_{i+1} - f_i}{h} \right| + \left| \frac{f_{i+1} - f_i}{h} - f'(x_i) \right| \\ &\leq \left| \frac{(\tilde{f}_{i+1} - f_{i+1}) + (f_i - \tilde{f}_i)}{h} \right| + \frac{M_2 h}{2} \\ &\leq \frac{|f_{i+1} - \tilde{f}_{i+1}| + |f_i - \tilde{f}_i|}{h} + \frac{M_2 h}{2} = \frac{2\delta}{h} + \frac{M_2 h}{2}. \end{aligned}$$

Отметим, во-первых, что эта оценка, достижима при подходящем сочетании знаков фигурирующих в неравенствах величин. Во-вторых, оценка не стремится к нулю при уменьшении шага h , так как первое слагаемое неограниченно увеличивается при $h \rightarrow 0$.

Полная вычислительная погрешность численного дифференцирования

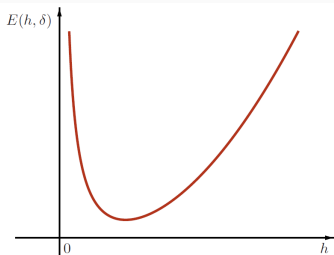
$E(h, \delta)$ при фиксированном δ имеет минимум:

$$\frac{\partial E(h, \delta)}{\partial h} = \frac{\partial}{\partial h} \left(\frac{2\delta}{h} + \frac{M_2 h}{2} \right) = -\frac{2\delta}{h^2} + \frac{M_2}{2} = 0.$$

То есть, оптимальное значение шага численного дифференцирования, при котором достигается минимальная полная погрешность, равно

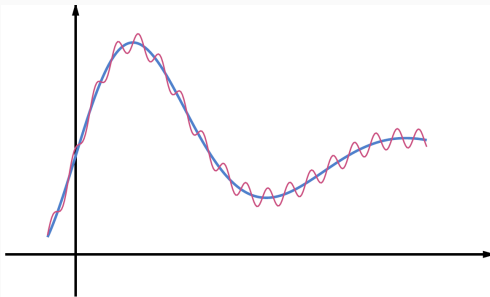
$$h^* = 2\sqrt{\delta/M_2},$$

и брать меньший шаг численного дифференцирования смысла нет. Значение достигаемой при этом полной погрешности есть $E(h^*, \delta) = 2\sqrt{\delta M_2}$.



Типичный график полной погрешности
численного дифференцирования

Некорректность задачи численного дифференцирования



Возмущение функции добавкой $\frac{1}{n} \sin(nx)$.

Сколь угодно большое возрастание погрешности числ. дифф-я, в действительности, является отражением более глубокого факта некорректности задачи дифф-я. Её решение не зависит непрерывно от входных данных. Если $f(x)$ – исходная функция, то возмущённая функция $f(x) + \frac{1}{n} \sin(nx)$ при $n \rightarrow \infty$ будет равномерно сходиться к исходной, тогда как её производная $f'(x) + \cos(nx)$ не сходится к $f'(x)$. При возмущении исходной функции слагаемым $\frac{1}{n} \sin(n^2x)$ производная вообще может сколь угодно сильно отличаться от производной исходной функции.

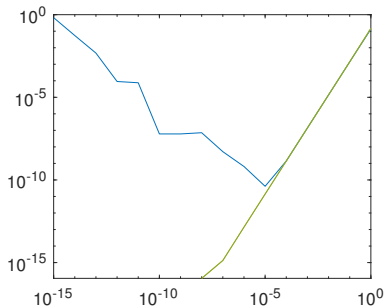
Способ исправления

Численное дифференцирование плохо обусловлено!

Пусть f – аналитическая функция, $f(x) \in \mathbb{R}$ при $x \in \mathbb{R}$, и $f(x)$ может быть вычислена в комплексных точках вблизи вещественной оси. Тогда численное дифференцирование устойчиво. Вычисление первой производной:

$$f(x + ih) = f(x) + ihf'(x) - h^2 f''(x)/2! - ih^3 f^{(3)}(x)/3! + \dots$$

$$f'(x) = \frac{\Im(f(x + ih))}{h} + O(h^2), \quad i^2 = -1.$$



Первая производная от $\sin(x)$ при $x = 1$. Синяя линия – формула центральной разности, зеленая – способ через выход в комплексную плоскость.

С помощью Фурье

Удобен в случае, когда построение интерполяционного полинома становится трудоемким.

Пусть задан шаблон из $p + 1$ точек $x_0, x_1, \dots, x_p$. Дифференцирование любого порядка является операцией, линейной по значениям функции. Поэтому будем искать приближенное выражение для производной в виде суммы

$$f^{(k)}(x) \approx \sum_{i=0}^p c_i f(x_i)$$

Коэффициенты c_i данной линейной формы постараемся подобрать так, чтобы эта формула являлась точной формулой для какого-то «достаточно представительного» набора функций. Например, в качестве таких «пробных функций» можно взять все полиномы степени не выше заданной, либо тригонометрические полиномы какого-то фиксированного порядка и т. п. Рассмотрим ниже подробно случай алгебраических полиномов.

Возьмём $f(x)$ равной последовательным степеням переменной x , т. е. $1, x, x^2, \dots, x^q$ для некоторого фиксированного q . Если линейная форма обращается в точное равенство на этих пробных функциях, то с учётом её линейности можно утверждать, что она будет точной для любого алгебраического полинома степени не выше q .

Каждое условие, выписанное для какой-то определённой степени x^j , $j = 0, 1, \dots, q$, является линейным соотношением для неизвестных коэффициентов c_i , и в целом мы приходим к системе линейных уравнений относительно c_i , $i = 0, 1, \dots, p$.

Для разрешимости этой системы естественно взять число неизвестных равным числу уравнений, т. е. $q = p$.

Метод неопределенных коэффициентов

$$\left\{ \begin{array}{lcl} c_0 & + & c_1 + \dots + c_p = 0, \\ c_0 x_0 & + & c_1 x_1 + \dots + c_p x_p = 0, \\ \vdots & & \ddots \quad \vdots \\ c_0 x_0^{k-1} & + & c_1 x_1^{k-1} + \dots + c_p x_p^{k-1} = 0, \\ c_0 x_0^k & + & c_1 x_1^k + \dots + c_p x_p^k = k!, \\ c_0 x_0^{k+1} & + & c_1 x_1^{k+1} + \dots + c_p x_p^{k+1} = (k+1)!x, \\ \vdots & & \ddots \quad \vdots \quad \vdots \\ c_0 x_0^p & + & c_1 x_1^p + \dots + c_p x_p^p = p(p-1) \dots (p-k+1) x^{p-k}. \end{array} \right.$$

В правых частях этой системы стоят k -е производные от $1, x, x^2, \dots, x^q$, а матрицей системы является матрица Вандермонда, которая неособенна для несовпадающих узлов $x_0, x_1, \dots, x_p$.

Несмотря на плохую обусловленность матрицы Вандермонда, использование метода не приводит к большим ошибкам, поскольку обычно размер системы уравнений небольшой.

Метод конструктивно проще и «технологичнее» в применении.

https://en.wikipedia.org/wiki/Numerical_differentiation

Таблицы с различными формулами:

https://en.wikipedia.org/wiki/Finite_difference_coefficient

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

<http://www.ict.nsc.ru/matmod/files/textbooks/SharyNuMeth.pdf>

Вычислительные методы линейной алгебры. Прямые методы решения систем линейных алгебраических уравнений

Чеховской Игорь Сергеевич

Задачи вычислительной линейной алгебры

1. решение систем линейных алгебраических уравнений (СЛАУ)
2. вычисление определителей матриц
3. нахождение обратной матрицы
4. нахождение собственных значений и собственных векторов матриц
5. нахождение сингулярных чисел и сингулярных векторов матриц
6. линейная задача наименьших квадратов: найти вектор x , минимизирующий $\langle Ax - b, Ax - b \rangle$ для заданных $m \times n$ -матрицы A и m -вектора b (решение переопределенных СЛАУ).

Задачи вычислительной линейной алгебры

Решение этих задач с точки зрения классической математики не встречает трудностей:

1. Метод Крамера

4, 5. Составить характеристический многочлен, решить уравнение

На практике эти правила неприменимы уже при $n \geq 10$: раскрытие определителя требует $n \cdot n!$ умножений и сложений.

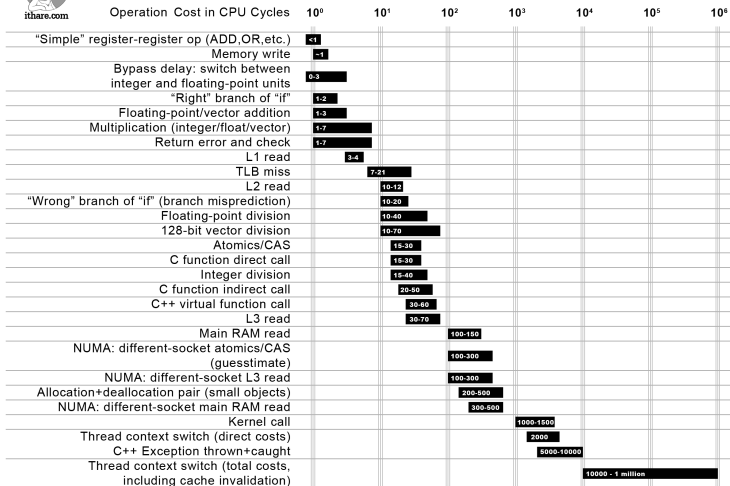
Флоп (англ., floating point operation) – одна усредненная арифметическая операция в арифметике с плавающей точкой в секунду.

Топ-500 суперкомпьютеров: <https://top500.org/>

Задачи вычислительной линейной алгебры



Not all CPU operations are created equal



Distance which light travels while the operation is performed



Задачи вычислительной линейной алгебры

Сколько времени нужно решать СЛАУ 30×30 по методу Крамера с раскрытием определителей по явной комбинаторной формуле на суперкомпьютере?

Пусть производительность нашего гипотетического компьютера равна 1 экзафлоп = 10^{18} операций с плавающей точкой в секунду.

30 компонент решения:

$30 \cdot 30!$ операций / (10^{18} флоп $\cdot$ 3600 сек/час $\cdot$ 24 час/сутки $\cdot$ 365 сутки/год), т. е. примерно $7.57 \cdot 10^9$ лет.

Для сравнения, возраст Земли оценивается в $4.5 \cdot 10^9$ лет.

Другая причина неуспешности классических алгоритмов – их большая чувствительность к ошибкам в данных и в вычислениях.

Матрицы и нормы

Множество матриц является *линейным векторным пространством*, а матрица - это составной многомерный объект.

Введем расстояние между матрицами как

$$\text{dist}(A, B) := \|A - B\|,$$

где A, B - вещ. или компл. $m \times n$ матрицы. Введем понятие матричной нормы и потребуем от нее тех же свойств, что и для векторной нормы.

Аксиомы нормы:

(МН1) $\|A\| \geq 0 \ \forall A$, причём $\|A\| = 0 \Leftrightarrow A = 0$ - неотрицательность,

(МН2) $\|\alpha A\| = |\alpha| \cdot \|A\| \ \forall A$ и $\forall \alpha \in \mathbb{R}$ или $\forall \alpha \in \mathbb{C}$ - абсолютная однородность,

(МН3) $\|A + B\| \leq \|A\| + \|B\| \ \forall A, B$ - «неравенство треугольника».

Связь нормы матриц с операцией их умножения отражает четвёртая аксиома матричной нормы:

(МН4) $\|AB\| \leq \|A\| \cdot \|B\| \ \forall A, B$ - «субмультипликативность».

Определение. Норма матрицы A называется *подчиненной* норме вектора x , если она определяется по правилу:

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sup_{\|x\|=1} \|Ax\|$$

Наиболее употребительными на практике являются следующие три векторные нормы:

$$\|x\|_1 = \sum_{i=1}^n |x_i|, \quad \|x\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2}, \quad \|x\|_\infty = \max_i |x_i|,$$

которым соответствуют матричные нормы:

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}| \text{ – первая норма (столбцовая норма, или 1-норма),}$$

$$\|A\|_2 = \sqrt{\max_{1 \leq i \leq n} |\lambda_i(A^T A)|} = \sqrt{\max_{1 \leq j \leq n} |\lambda_j(AA^T)|}$$

– спектральная норма (вторая норма, или норма Гильберта),

$$\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}| \text{ – бесконечная норма (строчная норма, или } \infty\text{-норма).}$$

Также на практике часто используется сферическая норма (норма Фробениуса, или евклидова норма):

$$\begin{aligned}\|A\|_F = \|A\|_E = N(A) &= \left(\sum_{i=1}^n \sum_{j=1}^m |a_{ij}|^2 \right)^{1/2} = \\ &= \left(\sum_{j=1}^m \sigma_j^2 \right)^{1/2} = \left(\sum_{j=1}^m \lambda_j (A^T A) \right)^{1/2} = \sqrt{\text{tr}(A^T A)}.\end{aligned}\tag{1}$$

Фробениусова матричная норма согласована с евклидовой векторной нормой $\|\cdot\|_2$, с которой она совпадает для векторов.

Замечание. В конечномерном линейном пространстве все нормы эквивалентны в том смысле, что, если имеет место $\|x_n\|_\alpha \rightarrow 0$ в норме α , то в любой другой норме $\beta : \|x_n\|_\beta \rightarrow 0$.

Число обусловленности матриц: определение и оценка ошибок

Определение. Для неособенной матрицы A её *числом обусловленности* (относительно выбранной подчинённой нормы) называется

$$\text{cond}(A) = \|A\| \|A^{-1}\|.$$

Пусть $(A + \Delta A)\tilde{x} = b + \Delta b$, $Ax = b$, $x = A^{-1}b$. Тогда при $\|A^{-1}\| \|\Delta A\| < 1$ справедлива оценка относительной ошибки решения

$$\frac{\|x - \tilde{x}\|}{\|x\|} \leq \frac{\|A^{-1}\| \|A\|}{1 - \|A^{-1}\| \|\Delta A\|} \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right).$$

Если возмущение матрицы мало, то приближённо

$$\frac{\|x - \tilde{x}\|}{\|x\|} \lesssim \text{cond}(A) \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right).$$

Замечания:

- $\text{cond}(A) \geq 1$ для всех подчинённых норм;
- чем больше $\text{cond}(A)$, тем сильнее усиливается относительная ошибка в данных;
- масштабирование $A \mapsto \alpha A$ не меняет $\text{cond}(A)$.

Для спектральной нормы (2-нормы) справедливо:

$$\text{cond}_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)},$$

где $\sigma_{\max}, \sigma_{\min}$ — наибольшее и наименьшее сингулярные числа.

Если A симметричная положительно определённая, то

$$\text{cond}_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

Число обусловленности: примеры и зачем оно нужно

Примеры.

- **Хорошо обусловленные:** ортогональные/унитарные матрицы
 $\Rightarrow \text{cond}_2 = 1$.
- **Плохо обусловленные:** матрицы Гильберта H_n — $\text{cond}_2(H_n)$ растёт экспоненциально по n ; матрицы Вандермонда — обычно сильно плохо обусловлены, при «неудачных» узлах условие растёт экспоненциально по n .

Зачем это нужно

- Даёт *априорную/апостериорную* оценку точности решения СЛАУ.
- Помогает выбирать устойчивые алгоритмы (частичный/полный выбор ведущего элемента в методе Гаусса, QR вместо нормальных уравнений в МНК).
- Дает подсказку для выбора *предобуславливания* перед итерационными методами.
- Диагностика «почти вырожденности» и выбор разрядности (float64 vs float32 и т. п.).

Основные понятия

Прямые методы:

- метод Крамера
- метод Гаусса
- метод прогонки (частный случай метода Гаусса)
- разложение Холецкого
- LU -разложение
- метод Хаусхолдера
- метод вращений

Итерационные методы (предпочтительны для больших СЛАУ):

- метод Якоби
- метод Гаусса-Зейделя
- метод релаксации
- наискорейший градиентный спуск

Основные понятия

Исходная СЛАУ:

$$\begin{cases} a_{11}x_1 + \dots + a_{1n}x_n = b_1 \\ \dots \\ a_{n1}x_1 + \dots + a_{nn}x_n = b_n \end{cases}$$

Матричный вид:

$$Ax = b,$$

где

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \dots & & \\ a_{n1} & \dots & a_{nn} \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix}$$

Матрица A называется основной матрицей системы, b – столбцом свободных членов.

Прямой ход

Согласно свойству элементарных преобразований над строками, основную матрицу этой системы можно привести к ступенчатому виду ($a_{ij} = 0 \ \forall i > j$) (эти же преобразования нужно применять к столбцу свободных членов):

```
DO FOR  $j = 1$  TO  $n - 1$ 
  DO FOR  $i = j + 1$  TO  $n$ 
     $r_{ij} \leftarrow (-a_{ij}/a_{jj})$ 
    DO FOR  $k = j$  TO  $n$ 
       $a_{ik} \leftarrow a_{ik} + r_{ij}a_{jk}$ 
    END DO
     $b_i \leftarrow b_i + r_{ij}b_j$ 
  END DO
END DO
```

Здесь и далее символ $\leftarrow$ обозначает оператор присваивания.

Если считать все операции одинаковыми по сложности, то алгоритм требует $\approx \frac{2}{3}n^3 = \mathcal{O}(n^3)$ арифметических операций.

Метод Гаусса

Результат прямого хода:

$$\begin{pmatrix} \tilde{a}_{11} & \tilde{a}_{12} & \dots & \tilde{a}_{1,n-1} & \tilde{a}_{1n} \\ 0 & \tilde{a}_{22} & \dots & \tilde{a}_{2,n-1} & \tilde{a}_{2n} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & \tilde{a}_{n-1,n-1} & \tilde{a}_{n-1,n} \\ 0 & 0 & \dots & 0 & \tilde{a}_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{pmatrix} = \begin{pmatrix} \tilde{b}_1 \\ \tilde{b}_2 \\ \vdots \\ \tilde{b}_{n-1} \\ \tilde{b}_n \end{pmatrix}$$

Обратный ход

```
DO FOR  $i = n$  DOWNT0 1
```

$$x_i \leftarrow \left(b_i - \sum_{j>i} a_{ij}x_j \right) / a_{ii}$$

```
END DO
```

Алгоритм требует $\mathcal{O}(n^2)$ арифметических операций.

Матричная интерпретация метода Гаусса

Умножение первого уравнения системы на $r_{i1} = -a_{i1}/a_{11}$ и сложение его с i -ым уравнением могут быть представлены в матричном виде как умножение обеих частей системы $Ax = b$ слева на матрицу

$$\begin{pmatrix} 1 & & & & \\ 0 & 1 & & & \\ \vdots & & \ddots & & \\ r_{i1} & & & 1 & \\ \vdots & & & & \ddots \\ 0 & & 0 & & 1 \end{pmatrix}$$

Исключение поддиагональных элементов первого столбца матрицы СЛАУ в прямом ходе метода Гаусса – это последовательное домножение обеих частей этой системы слева на аналогичные матрицы. Произведение двух таких матриц:

$$\begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ r_{i1} & & & 1 & \\ & & & & \ddots \\ 0 & & & & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ r_{k1} & & & 1 & \\ & & & & \ddots \\ 0 & & & & 1 \end{pmatrix} = \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ r_{i1} & & & 1 & \\ & & & & \ddots \\ r_{k1} & & & & 1 \end{pmatrix}$$

Матричная интерпретация метода Гаусса

Результат умножения всех матриц для зануления j -го столбца:

$$E_j = \begin{pmatrix} 1 & & & & & 0 \\ & \ddots & & & & \\ & & 1 & & & \\ 0 & r_{j+1,j} & 1 & & & \\ & \vdots & & \ddots & & \\ & r_{nj} & & & 1 & \end{pmatrix}$$

– нижняя треугольная матрица. В итоге матрица системы

$$E_{n-1} \cdots E_2 E_1 A = U,$$

становится верхней треугольной.

E_j – нижние треугольные матрицы, поэтому их произведение тоже является нижним треугольным. E_j неособенны (нижние треугол. с единицами по главной диагонали). Поэтому неособенно и их произведение $E_{n-1} \cdots E_2 E_1$.

$$L = (E_{n-1} \cdots E_2 E_1)^{-1}$$

– тоже нижняя треугол. матрица с единицами по главной диагонали.

Матричная интерпретация метода Гаусса

Таким образом, исходная матрица СЛАУ представляется в виде произведения нижней и верхней треугольных матриц:

$$A = LU$$

– ***LU-разложение*** (*LU-декомпозиция*, *LU-факторизация*).

Метод Гаусса – один из способов получения *LU-разложения*.

Если СЛАУ записана в виде

$$(LU)x = b,$$

то ее решение можно найти как реш-е двух СЛАУ с треуг. матрицами

$$Ly = b, \quad Ux = y$$

с помощью прямой и обратной подстановок соответственно.

Т. к. с помощью метода Гаусса matr. *L* получается с единицами на диаг., то на компьютере хранить *LU-разл.* можно в виде одной matr.

LU-разложение существует только в том случае, когда матрица *A* обратима, а все угловые главные миноры матрицы *A* невырождены.

Метод Гаусса с выбором ведущего элемента

При выполнении метода Гаусса может возникнуть деление на ноль, когда $a_{jj} = 0$ или один из главных угловых миноров нулевой (пример из методички).

Как сделать метод Гаусса применимым для любых неособенных матриц? Нужно совершать перестановки строк!

Ведущий (главный) элемент в методе Гаусса – элемент, на который делим при выполнении исключения (элементов под диагональю).

Если диаг. элемент $a_{jj} = 0$, то переставляем строки. Это имеет смысл делать для повышения устойчивости алгоритма, даже если $a_{jj} \neq 0$.

$\tilde{a}_{jj} = \max_{j \leq i \leq n} |a_{ij}|$ – **частичный** выбор ведущего элемента.

$\tilde{a}_{jj} = \max_{j \leq k, l \leq n} |a_{kl}|$ – **полный** выбор ведущего элемента (трудоемкий).

Матричная интерпретация метода Гаусса с выбором ведущего элемента

Элементарная матрица перестановок (транспозиции):

$$P = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 0 & \cdots & 1 \\ & & 1 & & \\ & \vdots & \ddots & \ddots & \vdots \\ & & & 1 & \\ 1 & \cdots & & 0 & \\ & & & & \ddots & \\ & & & & & 1 \end{pmatrix}, \quad \begin{array}{l} \leftarrow i\text{-ая строка} \\ \leftarrow j\text{-ая строка} \end{array}$$

Умножение ее слева на любую матрицу приводит к смене в этой матрице i -й и j -й строк, а при умножении справа – к смене столбцов.

Прямой ход метода Гаусса с частичным выбором ведущего элемента:

$$(E_{n-1}P_{n-1}) \cdots (E_1P_1)A = U.$$

Прямой ход метода Гаусса с полным выбором ведущего элемента:

$$(E_{n-1}\check{P}_{n-1}) \cdots (E_1\check{P}_1)A\hat{P}_1 \cdots \hat{P}_{n-1} = U,$$

где $\check{P}_i$ переставляют строки, $\hat{P}_j$ переставляют столбцы.

Вычисление определителя матрицы

Метод Гаусса в данном виде не меняет определителя матрицы, поэтому $\det A = \det U$ – произведение диагональных элементов матрицы U . Здесь $A = LU$.

Другая интерпретация: $\det A = \det L \cdot \det U = 1 \cdot \det U = \det U$.

Требуется $\frac{2}{3}n^3 + n \approx \frac{2}{3}n^3 = \mathcal{O}(n^3)$ операций.

Вычисление обратной матрицы

Считать обратную матрицу по определению

$$A^{-1} = \frac{\text{adj}(A)}{\det A}$$

неэффективно.

Обратная матрица к A – это решение матричного уравнения

$$AX = I.$$

Распадается на n штук векторных уравнений

$$Ax = e^{(j)}, \quad j = 1, \dots, n,$$

где $e^{(j)} = (0, 0, \dots, 0, 1, 0, \dots, 0)^T$ – j -й столбец единичной матрицы.

Из результатов, как столбцов, составляем матрицу $X = A^{-1}$.

Так как у всех векторных систем одинаковая матрица, удобно один раз сделать ее LU -разложение.

Требуется $\mathcal{O}(n^3)$ операций.

Tridiagonal matrix algorithm, Thomas algorithm. Был предложен И. М. Гельфандом и О. В. Локуциевским в 1952 году.

Для решения СЛАУ с трехдиаг. матр. (вид разреженных матриц).

Трехточечное разностное уравнение (разностное ур-е 2-го порядка):

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i,$$

$$a_1 = 0, c_n = 0.$$

СЛАУ:

$$\begin{bmatrix} b_1 & c_1 & & & 0 \\ a_2 & b_2 & c_2 & & \\ & a_3 & b_3 & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ 0 & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_n \end{bmatrix}.$$

Представляет собой вариант метода Гаусса. Алгоритм (прямой и обратный ход): [http:](http://eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf)

[//eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf](http://eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf)

Имеет сложность $8n = O(n)$ операций. Требуется $4n$ ячеек памяти.

Условие применимости прогонки – строгое **преобладание диагональных элементов**:

$$|b_i| > |a_i| + |c_i|.$$

Другая формулировка:

Пусть в трехдиаг. матрице СЛАУ все элементы поддиагонали, за исключением, может быть, последнего, и все элементы наддиагонали, за исключением, возможно, первого, не равны нулю, т.е. $a_i \neq 0$, $c_i \neq 0$, $i = 2, 3, \dots, n-1$, и, кроме того, $b_1 \neq 0$, $b_n \neq 0$. Если матрица системы имеет нестрогое диагональное преобладание,

$$|b_i| \geq |a_i| + |c_i|, \quad i = 1, 2, \dots, n$$

но хотя бы для одного индекса i это неравенство является строгим, то метод прогонки реализуем.

Пятидиагональная прогонка:

`https:`

`//www.intuit.ru/studies/courses/1012/168/lecture/4608?page=5`

Читаем методичку С. В. Смирнова и смотрим формулу

Шермана-Моррисона

https://en.wikipedia.org/wiki/Sherman%E2%80%93Morrison_formula

Метод Холецкого (Холесского)

A. Cholesky – француз, математик-геодезист.

Положительно определенная матрица A – матрица со свойством $x^T A x > 0$ для любых ненулевых $x \in \mathbb{R}$ или $\mathbb{C}$.

Теорема. Матрица A является симметричной положительно определенной $\iff$ существует единственная неособенная нижняя треугольная матрица C такая, что $A = CC^T$.

Представление $A = CC^T$ для симметричной положительно определенной матрицы A называется **разложением Холецкого**, а сама C – **множителем Холецкого** для A . Если оно найдено, то исходная СЛАУ $Ax = b \iff (CC^T)x = b$ может быть решена следующим образом:

$$\begin{cases} Cy = b, \\ C^T x = y. \end{cases}$$

Метод Холецкого (Холесского)

Псевдокод разложения Холецкого:

```
DO FOR  $j = 1$  TO  $n$   
   $c_{jj} \leftarrow \sqrt{a_{jj} - \sum_{k=1}^{j-1} c_{jk}^2}$   
  DO FOR  $i = j + 1$  TO  $n$   
     $c_{ij} \leftarrow \left( a_{ij} - \sum_{k=1}^{j-1} c_{ik} c_{jk} \right) / c_{jj}$   
  END DO  
END DO
```

Если A – симм. и полож. опр., то алгоритм проработает до конца без “авостов” и наоборот, если алгоритм встретит “авост” – извлечение корня из отрицательного числа или деление на ноль, то матрица не полож. опр.

Метод Холецкого – разновидность метода Гаусса. Иногда также называется “метод квадратного корня” и “метод квадратных корней”.

Экономия памяти в 2 раза и более высокая точность за счет “более жесткой” вычислительной схемы.

Опр. Для матрицы A представление $A = QR$ в виде произведения ортогональной матрицы Q и правой треугольной матрицы R называется *QR-разложением*.

Теорема. QR-разложение существует для любой квадратной матрицы.

Если известно QR-разложение матрицы A , то решение исходной СЛАУ

$$Ax = (QR)x = b$$

сводится к решению треугольной СЛАУ $Rx = Q^T b$.

Методы получения QR-разложения:

- процесс Грамма-Шмидта
- метод вращений (Гивенса)
- метод отражений (Хаусхолдера)



Метод Хаусхолдера (метод отражений)

Опр. Тензорным (диадным) произведением векторов называется операция

$$a \otimes b \leftrightarrow \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix} = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \\ a_4 b_1 & a_4 b_2 & a_4 b_3 \end{bmatrix}.$$

Опр. Для вектора $u \in \mathbb{R}^n$ с единичной евклидовой нормой, $\|u\|_2 = 1$, матрица $H = H(u) = I - 2uu^T$ называется *матрицей отражения* или *матрицей Хаусхолдера*. Вектор u называется *порождающим* или *вектором Хаусхолдера* для матрицы отражения $H(u)$.

Предл. Матрицы отражения являются симметричными ортогональными матрицами. Кроме того, для матрицы $H(u)$

- порождающий вектор u является собственным вектором, отвечающим собственному значению (-1) , т. е. $H(u) \cdot u = -u$;
- любой вектор v , ортогональный порождающему вектору u , является собственным вектором, отвечающим собственному значению 1 , т. е. $H(u) \cdot v = v$.

Определитель матрицы отражения равен -1 , т. е. $\det H(u) = -1$.

Метод Хаусхолдера (метод отражений)

Матрицы отражений действительно осуществляют преобразование отражения относительно гиперплоскости, ортогональной порождающему вектору u .

В самом деле, представим произвольный вектор x в виде $\alpha u + v$, где $\alpha \in \mathbb{R}$, u – порождающий матрицу отражения вектор, а v – ему ортогональный. Тогда

$$H(u) \cdot x = H(u) \cdot (\alpha u + v) = -\alpha u + v,$$

т. е. в векторе, преобразованном матрицей $H(u)$, та компонента, которая ортогональна рассматриваемой гиперплоскости, сменила направление на противоположное. Это и соответствует отражению относительно неё.

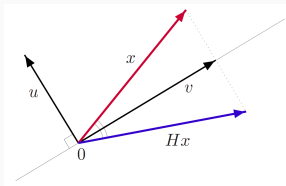


Рис. 1: Геометрическая интерпретация действия матрицы отражения.

Метод Хаусхолдера (метод отражений)

Предл. Пусть задан вектор $e \in \mathbb{R}^n$ единичной длины, $\|e\|_2 = 1$. Для любого ненулевого вектора $x \in \mathbb{R}^n$ существует матрица отражения, переводящая его в вектор, коллинеарный вектору e .

Итог:

- x, e – коллинеарны: $u = \frac{x}{\|x\|_2}$
- x, e – неколлинеарны: $u = \frac{\tilde{u}}{\|\tilde{u}\|_2}, \tilde{u} = x \pm \|x\|_2 e$

Предл. Для любой матрицы A существует конечная последовательность $H_1, H_2, \dots, H_s, s \in \{n-1, n\}$, состоящая из матриц, отражения и, возможно, единичных матриц, таких что матрица

$$H_s H_{s-1} \cdots H_2 H_1 A = R$$

является правой треугольной или трапециевидной матрицей.

Метод Хаусхолдера (метод отражений)

Док-во. Пусть $A = (a_{ij})$. Если хотя бы один из элементов $a_{21}, a_{31}, \dots, a_{n1}$ не равен нулю, то, используя результат предыдущего Предложения, возьмём в качестве H_1 матрицу отражения, которая переводит 1-й столбец A в вектор, коллинеарный $(1, 0, \dots, 0)^\top$. Иначе полагаем $H_1 = I$. Затем переходим ко второму шагу.

Результат:

$$\tilde{A} = \left(\begin{array}{c|cccc} \times & \times & \times & \cdots & \times \\ \hline 0 & \times & \times & \cdots & \times \\ 0 & \times & \times & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \times & \times & \cdots & \times \end{array} \right)$$

Прделаем теперь то же самое с матрицей $\tilde{A}(2 : n, 2 : n)$, обнулив у неё подходящим отражением поддиагональные элементы первого столбца, который является вторым во всей большой матрице $\tilde{A}$.

И так далее до $(n - 1)$ -го столбца.

Метод Хаусхолдера (метод отражений)

Псевдокод QR-разложения матрицы методом Хаусхолдера:

```
DO FOR  $j = 1$  TO  $n - 1$ 
     $\check{I} \leftarrow$  единичная матрица размера  $(n - j + 1)$ ;
    IF ( вектор  $A((j+1) : n, j)$  ненулевой ) THEN
        вычислить вектор Хаусхолдера  $\check{u} \in \mathbb{R}^{n-j+1}$ ,
        порождающий отражение, которое переводит
        вектор  $A(j : n, j)$  в вектор, коллинеарный
        вектору  $(1, 0, \dots, 0)^T$ ;
         $\check{H} \leftarrow \check{I} - 2\check{u}\check{u}^T$ ;
    ELSE
         $\check{H} \leftarrow \check{I}$ ;
    END IF
     $A(j : n, j : n) \leftarrow \check{H} A(j : n, j : n)$ ;
END DO
```

Метод Хаусхолдера (метод отражений)

Замечание. При вычислениях в стандартной арифметике с плавающей точкой имеет смысл брать

$$\tilde{u} = \begin{cases} A(j : n, j) + \|A(j : n, j)\|_2 e, & \text{если } a_{jj} \geq 0 \\ A(j : n, j) - \|A(j : n, j)\|_2 e, & \text{если } a_{jj} \leq 0 \end{cases}$$

где $e = (1, 0, \dots, 0)^T$. Тогда вычисление первого элемента в столбце $A(j : n, j)$, т. е. того единственного элемента из всего столбца, который останется ненулевым, не будет сопровождаться вычитанием чисел одного знака и, как следствие, возможной потерей точности.

Замечание. Недостатком метода Хаусхолдера (и метода вращений) в сравнении с методом Гаусса является привлечение неарифметической операции извлечения квадратного корня, которая приводит к иррациональностям. Это не позволяет точно (без округлений) реализовать соответствующие алгоритмы в поле рациональных чисел, к примеру, в программных системах так называемых «безошибочных вычислений» или языках программирования типа Ruby, которые могут оперировать рациональными дробями.

Замечание. Не нужно формировать в явном виде матрицу отражения $\check{H}$: умножение на неё можно выполнить по экономичной формуле

$$(I - 2uu^\top) A(j:n, j:n) = A(j:n, j:n) - 2u(u^\top A(j:n, j:n))$$

в которой сама $\check{H}$ не фигурирует. При решении СЛАУ можно одновременно с разложением матрицы преобразовывать также правую часть, умножая её на матрицы $\check{H}$. Тогда к моменту окончания QR-разложения у нас уже будет известен вектор правой части $Q^\top b$ для системы с треугольной матрицей R .

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

<http://www.ict.nsc.ru/matmod/files/textbooks/SharyNuMeth.pdf>

Смирнов С. В.: ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ, Часть II

<http://www.phys.nsu.ru/smirnov/ovf2-2017-09-18.pdf>

Князева А. Г.: Различные варианты метода прогонки [http:](http://eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf)

[//eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf](http://eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf)

Пятидиагональная прогонка: [https:](https://www.intuit.ru/studies/courses/1012/168/lecture/4608?page=5)

[//www.intuit.ru/studies/courses/1012/168/lecture/4608?page=5](https://www.intuit.ru/studies/courses/1012/168/lecture/4608?page=5)

Численное решение обыкновенных дифференциальных уравнений

Чеховской Игорь Сергеевич

Задача Коши:

$$\begin{cases} u' = f(x, u(x)), & a \leq x \leq b, \\ u(a) = u_a. \end{cases}$$

Вводим сетку (не обязательно равномерную):

$$x_n, 0 \leq n \leq N : a = x_0 < x_1 < x_2 < \dots < x_N = b.$$

Шаг сетки $h_n = x_{n+1} - x_n$.

Значения искомой функции в узлах сетки: $u_n = u(x_n)$.

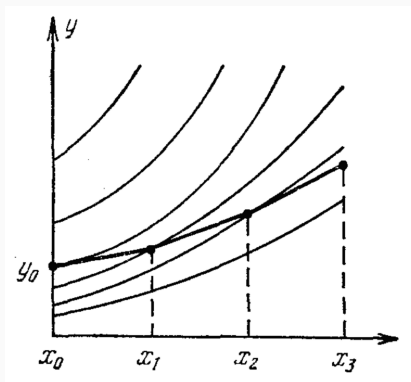
Разложение $u(x)$ в ряд Тейлора на интервале $x_n \leq x \leq x_{n+1}$:

$$u_{n+1} = u_n + h_n u'_n + \frac{1}{2} h_n^2 u''_n + \dots$$

Старшие производные вычисляются последовательным дифференцированием исходного уравнения:

$$u'_n = f(x_n, u_n), \quad u''_n = f'_x(x_n, u_n) + f(x_n, u_n) f'_u(x_n, u_n), \dots$$

Метод Эйлера (схема ломаных). Геометрический смысл.



Ограничиваясь первым членом разложения, получаем схему ломаных приближенного решения y_n :

$$y_{n+1} = y_n + h_n f(x_n, y_n)$$

С начальным условием:

$$y_0 = u_0 = u_a$$

Локальный порядок аппроксимации – 2.

А что с глобальным порядком?

Сходимость метода Эйлера

Рассмотрим погрешность приближенного решения $z_n = y_n - u_n$:

$$z_{n+1} = z_n + h_n [f(x_n, y_n) - f(x_n, u_n)] - \frac{1}{2} h_n^2 u_n''$$

По теореме о среднем

$$f(x_n, y_n) - f(x_n, u_n) = f_u(x_n, \xi_n)(y_n - u_n) = f_u(x_n, \xi_n)z_n,$$

где ξ_n лежит между u_n и y_n , поэтому

$$z_{n+1} = z_n (1 + h_n f_u(x_n, \xi_n)) - \frac{1}{2} h_n^2 u_n''.$$

Погрешность на произвольном шаге m :

$$z_m = z_0 \prod_{n=0}^{m-1} (1 + h_n f_u(x_n, \xi_n)) - \frac{1}{2} \sum_{n=0}^{m-1} h_n^2 u_n'' \prod_{k=n+1}^{m-1} (1 + h_k f_u(x_k, \xi_k))$$

Если шаг интегрирования мал,

$$\prod_{n=0}^{m-1} (1 + h_n f_u(x_n, \xi_n)) \approx \prod_{n=0}^{m-1} \exp(h_n f_u(x_n, \xi_n)) = \exp\left(\sum_{n=0}^{m-1} h_n f_u(x_n, \xi_n)\right).$$

$$z_m = z_0 \exp \left(\int_{x_0}^{x_m} f_u d\tau \right) - \frac{1}{2} \int_{x_0}^{x_m} d\tau h(\tau) u''(\tau) \exp \left(\int_{\tau}^{x_m} f_u d\mu \right)$$

Начальное условие можно задать точно: $z_0 = 0$.

Считая функцию $f(x, u)$ непрерывной и ограниченной вместе с производными: $|f| \leq M_1$, $|f_x| \leq M_2$, $|f_u| \leq M_3$ ($|u''| \leq M_4 = M_2 + M_1 M_3$) получаем мажорантную оценку погрешности:

$$\begin{aligned} |z_m| &\leq M(x_m) \max h_n = \mathcal{O}(\max h_n), \\ M(x_m) &= \frac{1}{2} \int_{x_0}^{x_m} d\tau |u''(\tau)| \exp \left(\int_{\tau}^{x_m} |f_u| d\mu \right) \leq \frac{M_4}{2M_3} e^{M_3(x_m - x_0)} \end{aligned}$$

Т.е. при $h \rightarrow 0$ приближенное решение по схеме Эйлера сходится к точному равномерно с первым порядком точности (**глобальный порядок аппроксимации**).

Невязка (погрешность) аппроксимации разностного уравнения на решении исходного дифференциального уравнения ψ_n – результат подстановки точного решения $u = u(x)$ в разностное уравнение.

Разностный метод (схема) **аппроксимирует** дифференциальное уравнение, если $\psi_n \rightarrow 0$ при $h \rightarrow 0$, где ψ_n – невязка, h – шаг сетки.

Разностный метод (схема) имеет **p -й порядок аппроксимации**, если

$$\psi_n = \mathcal{O}(h^p).$$

Решение y_n , полученное с помощью схемы, **сходится к точному решению $u(x_n)$ с порядком p** , если

$$\|y_n - u(x_n)\| \leq Ch^p, \quad C = \text{const} > 0.$$

Свойства:

- Методы являются одноступенчатыми (одношаговыми): чтобы найти y_{n+1} , нужна информация только о предыдущей точке x_n, y_n
- Методы согласуются с рядом Тейлора вплоть до членов порядка h^p , где степень p различна для различных методов и называется порядком метода
- Методы не требуют вычислений производных от $f(x, u)$, а требуют только вычисления самой функции.

Замечание: метод Эйлера является методом Рунге-Кутты первого порядка.

Являются следствием квадратурных формул:

пусть $f(x, u) = g(x)$, тогда

$$\begin{cases} \frac{du}{dx} = g(x), & a \leq x \leq b, \\ u(a) = u_a. \end{cases}$$

Решение этой задачи Коши может быть выписано в явном виде:

$$u(x) = u_a + \int_a^x g(t) dt, \quad a \leq x \leq b.$$

Тогда метод Эйлера можно интерпретировать как применение формулы левых прямоугольников к вычислению этого интеграла.

Исправленный метод Эйлера

В исправленном методе Эйлера средний тангенс угла наклона касательной вычисляется по двум точкам: x_m, y_m и $x_m + h, y_m + hy'_m$:

$$\Phi(x_m, y_m, h) = \frac{1}{2} [f(x_m, y_m) + f(x_m + h, y_m + hy'_m)], \quad \text{где} \quad y'_m = f(x_m, y_m)$$

Приближенное решение в точке x_{m+1} находится следующим образом:

$$y_{m+1} = y_m + h\Phi(x_m, y_m, h)$$

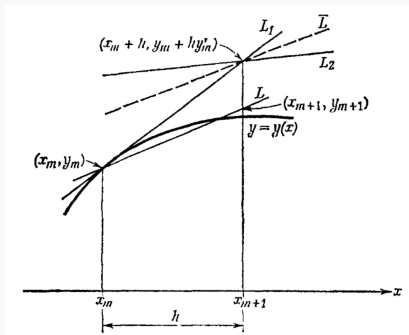


Схема предиктор-корректор (формула трапеций).

Порядок исправленного метода Эйлера

Разложение в ряд Тейлора функции $f(x, y)$ вблизи x_m, y_m

$$f(x, y) = f(x_m, y_m) + f_x(x - x_m) + f_y(y - y_m) + \dots$$

Подставляя $x = x_m + h$, $y = y_m + hf'_m$ с учетом $y'_m = f(x_m, y_m)$, вычислим $\Phi(x_m, y_m, h)$:

$$\Phi(x_m, y_m, h) = f + \frac{h}{2}(f_x + ff_y) + O(h^2)$$

Итоговая формула совпадает с разложением функции y в ряд Тейлора вплоть до членов второго порядка по h :

$$y_{m+1} = y_m + hf + \frac{h^2}{2}(f_x + ff_y) + O(h^3)$$

т.е. является методом Рунге-Кутты второго порядка.

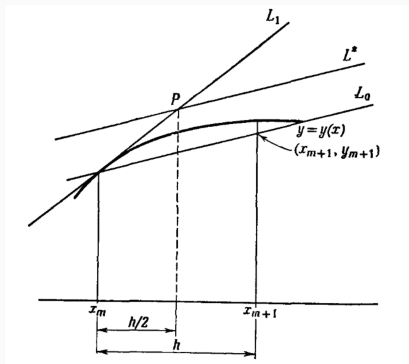
Модифицированный метод Эйлера

Вычислим тангенс угла наклона касательной в усредненной точке $x_m + h/2, y_m + y'_m h/2$:

$$\Phi(x_m, y_m, h) = f\left(x_m + \frac{h}{2}, y_m + \frac{h}{2}y'_m\right), \quad \text{где} \quad y'_m = f(x_m, y_m)$$

Приближенное решение в точке x_{m+1} находится следующим образом:

$$y_{m+1} = y_m + h\Phi(x_m, y_m, h)$$



Порядок модифицированного метода Эйлера

Разложение в ряд Тейлора функции $f(x, y)$ вблизи x_m, y_m

$$f(x, y) = f(x_m, y_m) + f_x(x - x_m) + f_y(y - y_m) + \dots$$

Подставляя $x = x_m + h/2$, $y = y_m + hy'_m/2$ с учетом $y'_m = f(x_m, y_m)$, вычислим $\Phi(x_m, y_m, h)$:

$$\Phi(x_m, y_m, h) = f + \frac{h}{2}(f_x + ff_y) + O(h^2)$$

Итоговая формула совпадает с разложением функции y в ряд Тейлора вплоть до членов второго порядка по h :

$$y_{m+1} = y_m + hf + \frac{h^2}{2}(f_x + ff_y) + O(h^3)$$

т.е. является методом Рунге-Кутты второго порядка.

Методы Рунге-Кутты второго порядка

Общий вид схем:

$$y_{m+1} = y_m + h\Phi(x_m, y_m, h). \quad (1)$$

Тангенс угла наклона в общем виде вычисляется по некоторым 2 точкам (двухстадийный метод):

$$\Phi(x_m, y_m, h) = a_1 f(x_m, y_m) + a_2 f(x_m + b_1 h, y_m + b_2 h f(x_m, y_m))$$

Для того, чтобы схема была второго порядка, необходимо 3 параметра. Один является произвольным. Разложим функцию $f(x, y)$ в ряд Тейлора вблизи точки $x = x_m + b_1 h, y = y_m + b_2 h f$ и подставим в (1)

$$y_{m+1} = y_m + h [a_1 f + a_2 f + h(a_2 b_1 f_x + a_2 b_2 f f_y)] + O(h^3)$$

Условия: $a_1 + a_2 = 1, a_2 b_1 = 1/2, a_2 b_2 = 1/2$. Пусть произвольный параметр $a_2 = \alpha \neq 0$:

$$y_{m+1} = y_m + h \left[(1 - \alpha) f(x_m, y_m) + \alpha f \left(x_m + \frac{h}{2\alpha}, y_m + \frac{h}{2\alpha} f(x_m, y_m) \right) \right] + O(h^3)$$

При $\alpha = 0.5$ – исправленный метод Эйлера, при $\alpha = 1$ – модифицированный метод Эйлера.

Четырехстадийный метод:

$$\begin{aligned}y_{m+1} &= y_m + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \\k_1 &= f(x_m, y_m), \\k_2 &= f\left(x_m + \frac{h}{2}, y_m + \frac{hk_1}{2}\right), \\k_3 &= f\left(x_m + \frac{h}{2}, y_m + \frac{hk_2}{2}\right), \\k_4 &= f(x_m + h, y_m + hk_3).\end{aligned}$$

Пусть $f(x, y) = F(x)$, т.е. зависит только от x . $y(x) = \int_{x_0}^x F(x)dx$. Введем обозначения $p = h/2$, $F_j = F(x_0 + jp)$. Т.к. $y_m = y(x_0 + mh) = y(x_0 + 2mp)$, то обозначив $Y_j = y(x_0 + jp)$ ($y_m = Y_{2m}$) имеем:

$$Y_{2m+2} - Y_{2m} = \frac{p}{3}(F_{2m} + 4F_{2m+1} + F_{2m+2})$$

Суммируя на всем интервале, убеждаемся, что совпадает с формулой Симпсона.

Метод Рунге-Кутты четвертого порядка. Пример.

Рассмотрим решение уравнения $y' = y, y(0) = 1$. Вычислим коэффициенты в схеме Рунге-Кутты 4-го порядка:

$$\begin{aligned}k_1 &= y_m, \\k_2 &= y_m + \frac{h}{2}y_m, \\k_3 &= y_m + \frac{h}{2} \left(y_m + \frac{h}{2}y_m \right), \\k_4 &= y_m + h \left(y_m + \frac{h}{2} \left(y_m + \frac{h}{2}y_m \right) \right).\end{aligned}$$

Подставляя вычисленные коэффициенты в схему РК:

$$y_{m+1} = \left(1 + h + \frac{h^2}{2} + \frac{h^3}{6} + \frac{h^4}{24} \right) y_m$$

Сумма в скобках - пять первых членов разложения решения e^h в ряд Тейлора.

Переменный (адаптивный) шаг. Пример

Рассмотрим схему РК4. Будем одновременно совершать вычисления с шагом h и с шагом $2h$, и попробуем отсюда извлечь дополнительную информацию для оценки шага интегрирования.

Количество вычислений правой части $f(x, y)$ на одном отрезке длиной $2h$: 8 с шагом h и 4 с шагом $2h$.

Если производятся вычисления одновременно с двумя шагами h и $2h$, то на один такой отрезок нужно 11 вычислений функции $f(x, y)$ (в первой точке функция вычисляется одинаково).

Пусть y_h и y_{2h} – численные решения, полученные методами с соответствующим размером шага.

Тогда точное решение удовлетворяет:

$$\begin{aligned}y(x + 2h) &= y_h + 2Kh^5 + \mathcal{O}(h^6), \\y(x + 2h) &= y_{2h} + K(2h)^5 + \mathcal{O}(h^6).\end{aligned}$$

Переменный (адаптивный) шаг. Пример

Вычитая эти формулы, получаем оценку локальной ошибки:

$$R_1 = y(x + 2h) - y_h = \frac{y_h - y_{2h}}{15} + \mathcal{O}(h^6).$$

Поэтому в качестве практической оценки локальной погрешности можно использовать

$$|R_1| \approx \frac{|y_h - y_{2h}|}{15}.$$

Требование на точность:

$$|R_1| < \delta + |y|\varepsilon.$$

Также эту оценку можно использовать для уточнения решения (**метод Рунге–Ромберга, экстраполяция Ричардсона**):

$$y(x + 2h) = y_h + \frac{y_h - y_{2h}}{15} + \mathcal{O}(h^6).$$

Это дает выигрыш на один порядок по точности.

Переменный (адаптивный) шаг. Пример

На практике адаптивный шаг часто реализуют не через удвоение шага, а через встроенные пары методов разных порядков.

Например, в MATLAB решатель `ode45` использует явную пару методов РК 4-го и 5-го порядков (Dormand and Prince):

на каждом шаге одновременно строятся приближения 4-го и 5-го порядка, по их разности оценивается локальная ошибка, после чего шаг автоматически уменьшается или увеличивается.

Система ОДУ называется **линейной**, если она имеет вид

$$\begin{cases} y' = A(t)y + g(t), & t_0 \leq t \leq t_N, \\ y(t_0) = y_0. \end{cases}$$

Если $g(t) \equiv 0$ – **однородная**.

Опр. Решение $\tilde{y}(t)$ системы ОДУ называется **устойчивым по Ляпунову**, если

$$\forall \varepsilon > 0 \exists \delta > 0 : \forall y(t) : |y(t_0) - \tilde{y}(t_0)| < \delta \Rightarrow |y(t) - \tilde{y}(t)| < \varepsilon \quad \forall t \geq t_0.$$

Если также $\lim_{t \rightarrow \infty} |y(t) - \tilde{y}(t)| = 0$, то решение называется **асимптотически устойчивым**.

Теорема (критерий асимптотической устойчивости). Нулевое решение линейной автономной системы ОДУ

$$y' = Ay$$

асимптотически устойчиво тогда и только тогда, когда

$$\Re \lambda_i < 0, \quad i = 1, \dots, N,$$

где λ_i – собственные значения матрицы A .

Опр. Система линейных уравнений $y' = Ay$ является **жесткой**, если велико отношение максимального и минимального модуля собственных значений

- $\forall i \operatorname{Re} \lambda_i < 0, i = 1, \dots, n,$
- $s = \frac{\max_i \operatorname{Re}(-\lambda_i)}{\min_i \operatorname{Re}(-\lambda_i)} \gg 1, s$ – число жесткости системы,
- $\max_i \operatorname{Re}(-\lambda_i)(t_n - t_0) \gg 1 .$

Пример: $y(t) = c_1 e^{\lambda_1 t} + c_2 e^{\lambda_2 t}$ – решение с медленно и быстро затухающими экспонентами.

Пусть U_h – линейное нормированное пространство сеточных функций u_h , определенных на сетке $\bar{\omega}_h$.

Для краевой задачи для дифф. уравнения

$$Lu = f$$

записана разностная схема

$$L_h u_h = f_h, \quad f_h \in F_h.$$

Решение:

$$u_h = \{u_j\} \in U_h, \quad j = 1, \dots, N.$$

Опр. Невязка (погрешность) аппроксимации разностного уравнения на решении исходного дифференциального уравнения $\psi_h = L_h(u)_h - f_h$ – результат подстановки точного решения $u = u(x, t)$ в разностное уравнение.

Опр. Разностная схема $L_h u_h = f_h$ аппроксимирует задачу $Lu = f$ на ее решении $u(x)$, если

$$\|\psi_h\|_{F_h} \rightarrow 0 \text{ при } h \rightarrow 0,$$

Если сверх того имеет место неравенство

$$\|\psi_h\|_{F_h} \leq Ch^p$$

где $C > 0, p > 0$ – некоторые постоянные, не зависящие от h , то говорят, что схема аппроксимирует с порядком p по h .

Теорема (о сходимости). Пусть разностная схема $L_h u_h = f_h$ аппроксимирует дифференциальную задачу $Lu = f$ на ее решении u с порядком p по h и устойчива. Тогда решение разностной схемы сходится к решению дифференциальной задачи, причем имеет место оценка

$$\|(u)_h - u_h\|_{U_h} \leq Ch^p \quad C = \text{const} > 0, \quad p > 0.$$

Качественное определение. Разностная схема называется устойчивой, если она имеет единственное решение и малые возмущения в начальных данных и правой части разностной схемы приводят к малому изменению решения.

Опр. Разностная схема $L_h u_h = f_h$ с произвольным, в т.ч. нелинейным, оператором L_h **устойчива**, если существуют числа $h_0 > 0$ и $\delta > 0$ т., ч. $\forall h < h_0$ и $\forall \varepsilon_h \in F_h$, для которого $\|\varepsilon_h\|_{F_h} < \delta$, разностная задача $L_h z_h = f_h + \varepsilon_h$, полученная из исходной разностной схемы добавлением к правой части возмущения ε_h , имеет единственное решение z_h и

$$\|z_h - u_h\|_{U_h} \leq C \|\varepsilon_h\|_{F_h}$$

где $C \neq C(h)$ – некоторая постоянная.

Опр. Разностная схема $L_h u_h = f_h$ с линейным оператором L_h **устойчива**, если существует число $h_0 > 0$ т., ч. $\forall h < h_0$ и $\forall f_h \in F_h$ она имеет единственное решение $u_h \in U_h$, причем

$$\|u_h\|_{U_h} \leq C \|f_h\|_{F_h}$$

где $C \neq C(h)$.

Рассмотрим понятие устойчивости методов численного интегрирования ОДУ на примере модельного уравнения:

$$y' = \lambda y, \Re \lambda < 0$$

Решение уравнения:

$$y(x) = y(0)e^{\lambda x}$$

Для любого $a > 0$ справедливо:

$$|y(x + a)| \leq |y(x)|$$

т.е. решение уравнения асимптотически устойчиво.

Часто возникает необходимость получать решение задачи Коши на больших отрезках по времени. Исследование численных методов на модельном уравнении позволяет отбросить заведомо непригодные для этого методы.

Для y_n , построенного по схеме Эйлера, имеем:

$$y_{n+1} = y_n + hf(x_n, y_n) = (1 + h\lambda)y_n = R(z)y_n, \quad z = h\lambda.$$

Для устойчивости численного решения необходимо условие

$$|R(z)| = |1 + z| < 1.$$

Следовательно, область абсолютной устойчивости явного метода Эйлера:

$$|1 + z| < 1.$$

В частном случае вещественного $\lambda < 0$ это условие принимает вид

$$-2 < h\lambda < 0,$$

то есть

$$0 < h < \frac{2}{-\lambda}.$$

Поскольку на шаг интегрирования накладывается ограничение, такую схему называют **условно устойчивой**.

Рассмотрим численное решение того же уравнения, полученного с помощью неявной схемы Эйлера:

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) = y_n + h\lambda y_{n+1}$$

Исследуем асимптотическую устойчивость численного решения

$$y_{n+1} = \frac{y_n}{1 - h\lambda}, \quad R(z) = \frac{1}{1 - z}$$

Из условия $|1 - z| > 1$ видно, что для любого шага сетки h численное решение будет устойчивым.

Такая численная схема называется *абсолютно устойчивой*.

Для численного решения жестких систем предпочтительны абсолютно устойчивые методы. Многие явные методы непригодны из-за малого размера их области абсолютной устойчивости.

Опр. Метод **абсолютно устойчив с шагом h** , если при его применении к модельному уравнению $y_n \rightarrow 0$ при $n \rightarrow \infty$.

Опр. Область $D \subset \mathbb{C}_-$ называется **областью абсолютной устойчивости метода**, если метод абсолютно устойчив для всех $z = \lambda h \in D$.

Опр. Метод называется **A-устойчивым**, если $D = \mathbb{C}_-$.

Опр. Метод называется **L-устойчивым**, если он A-устойчивый и $|R(z)| \rightarrow 0$ при $\Re z \rightarrow -\infty$, где $R(z)$ – функция роста (множитель перехода, функция устойчивости):

$$y_{n+1} = R(z)y_n.$$

Явные методы РК не являются А-устойчивыми, поскольку для них функция устойчивости $R(z)$ является полиномом.

Отсутствие L-устойчивости может вызывать паразитные осцилляции численного решения при расчете жестких систем ОДУ.

Модельное уравнение:

$$u' = \lambda u \quad u(0) = u_0$$

Общая формула явных методов Рунге-Кутты:

$$\begin{aligned} u^{n+1} &= u^n + h \sum_{i=1}^s b_i k_i \\ k_i &= \lambda \left(u^n + h \sum_j a_{ij} k_j \right) \end{aligned} \Rightarrow \begin{aligned} (E - h\lambda A)k &= \lambda u^n e \\ e &= (1, \dots, 1)^T \\ k &= (k_1, \dots, k_s)^T \end{aligned} \quad k = (E - h\lambda A)^{-1} \lambda u^n e$$

[Видео](#)

[Википедия](#)

Области устойчивости

Общая формула явных методов:

$$\begin{aligned} u^{n+1} &= u^n + \tau \sum_{i=1}^s b_i k_i = u^n + \tau(b, k) = \\ &= u^n + h(b, (E - h\lambda A)^{-1} \lambda u^n e) \end{aligned}$$

Функция устойчивости определяется из соотношения

$$u^{n+1} = R(z)u^n, \quad z = h\lambda.$$

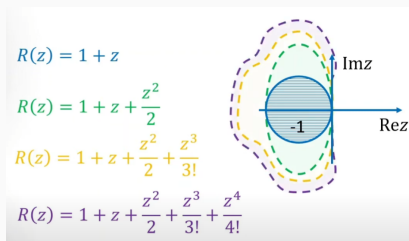
Для метода Рунге–Кутты:

$$R(z) = 1 + z b^T (E - zA)^{-1} e.$$

Эквивалентно:

$$R(z) = \frac{\det(E - zA + z e b^T)}{\det(E - zA)}.$$

Для явных методов Рунге–Кутты матрица A строго нижнетреугольная, поэтому $\det(E - zA) = 1$, и функция устойчивости является полиномом:



Неявные методы Рунге-Кутты

$$y_{n+1} = y_n + h \sum_{i=1}^s b_i k_i$$

где

$$k_i = f \left(t_n + c_i h, y_n + h \sum_{j=1}^s a_{ij} k_j \right), \quad i = 1, \dots, s$$

c_1	a_{11}	a_{12}	$\dots$	a_{1s}
c_2	a_{21}	a_{22}	$\dots$	a_{2s}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_s	a_{s1}	a_{s2}	$\dots$	a_{ss}
	b_1	b_2	$\dots$	b_s
	b_1^*	b_2^*	$\dots$	b_s^*

$$= \frac{c}{b^T} \bigg| \frac{A}{b^T}$$

Нижняя дополнительная строка b_i^* может использоваться для встроенной формулы меньшего порядка и оценки локальной погрешности.

Общий вид p -шагового метода:

$$F_n(y_{n-p}, y_{n-p+1}, \dots, y_n, y_{n+1}) = 0.$$

Заменяем $f(x, y)$ некоторой функцией $F(x)$:

$$F(x) = f(x, y(x)) + \mathcal{O}(h^p).$$

Пусть известны значения $y_n, y_{n-1}, \dots, y_{n-p}$.

Интерполяционный полином в форме Ньютона:

$$\begin{aligned} F(x) = & F(x_n) + \\ & + (x - x_n)F(x_n, x_{n-1}) + \\ & + (x - x_n)(x - x_{n-1})F(x_n, x_{n-1}, x_{n-2}) + \\ & + (x - x_n)(x - x_{n-1})(x - x_{n-2})F(x_n, x_{n-1}, x_{n-2}, x_{n-3}) + \dots \end{aligned}$$

Дифф. уравнение в интегральной форме:

$$y_{n+1} = y_n + \int_{x_n}^{x_{n+1}} f(x, y(x)) dx \approx y_n + \int_{x_n}^{x_{n+1}} F(x) dx.$$

Интегрируя на каждом интервале полином 3-й степени, получим:

$$\int_{x_n}^{x_{n+1}} (x - x_n) dx = \frac{h_n^2}{2}, \quad \text{где } h_n = x_{n+1} - x_n$$

$$\int_{x_n}^{x_{n+1}} (x - x_n) \cdot (x - x_{n-1}) dx = \frac{h_n^3}{3} + \frac{h_n^2}{2} h_{n-1}$$

$$\int_{x_n}^{x_{n+1}} \left(\prod_{k=n-2}^n (x - x_k) \right) dx = \frac{h_n^4}{4} + \frac{h_n^3}{3} (2h_{n-1} + h_{n-2}) + \frac{h_n^2}{2} h_{n-1} (h_{n-1} + h_{n-2}).$$

Получаем **формулу Адамса для переменного шага** 4-го порядка:

$$\begin{aligned} y_{n+1} = & y_n + h_n F(x_n) + \frac{1}{2} h_n^2 F(x_n, x_{n-1}) + \\ & \left[\frac{1}{3} h_n^3 + \frac{1}{2} h_n^2 h_{n-1} \right] \cdot F(x_n, x_{n-1}, x_{n-2}) + \\ & \left[\frac{h_n^4}{4} + \frac{1}{3} h_n^3 (2h_{n-1} + h_{n-2}) + \frac{1}{2} h_n^2 h_{n-1} (h_{n-1} + h_{n-2}) \right] \cdot F(x_n, x_{n-1}, x_{n-2}, x_{n-3}) \end{aligned}$$

Явные методы Адамса – **методы Адамса–Башфорта** (Bashforth).

Неявные методы Адамса – **методы Адамса–Мултона** (Moulton).

Локальная погрешность метода Адамса–Башфорта 4-го порядка:

$$R = \frac{251}{720} h^5 F^{(4)}(x) + \mathcal{O}(h^6).$$

Она большая из-за экстраполяции $F(x)$ за пределы $[x_{n-p}, x_n]$.

Погрешность метода РК4 (как и у метода Симпсона)

$$R = \frac{1}{2880} h^5 f^{(4)}(x) + \mathcal{O}(h^6)$$

в 1004 раза больше у метода Адамса–Башфорта 4-го порядка.

Это означает, что при одинаковой гладкости решения и одинаковых прочих условиях для РК4 можно взять шаг в $\sqrt[4]{1004} \approx 5,62$ раз меньше!

Кроме того, методы Адамса требуют предварительного вычисления функции $y(x)$ в p начальных точках!

Многошаговые методы: нуль-устойчивость и корневое условие

Рассмотрим линейный многошаговый метод

$$\sum_{j=0}^k \alpha_j y_{n+j} = h \sum_{j=0}^k \beta_j f_{n+j}, \quad \alpha_k \neq 0.$$

Ему соответствует характеристический многочлен

$$\rho(\xi) = \sum_{j=0}^k \alpha_j \xi^j.$$

Опр. Говорят, что многошаговый метод удовлетворяет **корневому условию**, если все корни многочлена $\rho(\xi)$ лежат внутри или на границе единичного круга:

$$|\xi_i| \leq 1,$$

причем корни, лежащие на единичной окружности, являются простыми.

Для линейных многошаговых методов выполнение корневого условия эквивалентно **нуль-устойчивости**.

Многошаговые методы: теорема Далквиста о сходимости многошаговых методов

Теорема. Если многошаговый метод согласуется с порядком p (т.е. аппроксимирует задачу с порядком p) и является **нуль-устойчивым** (т.е. удовлетворяет корневому условию), то метод сходится с порядком p .

Методы Розенброка: общая идея

Одношаговые, многостадийные, неявные, линейно-аппроксимированные методы Рунге–Кутты. Для эффективного решения жёстких ОДУ

Происхождение: линейнизация неявных методов РК

Неявный РК:

$$k_i = f \left(u_n + \tau \sum_{j=1}^s a_{ij} k_j \right)$$

Розенброк: аппроксимация через разложение Тейлора:

$$f(u_n + \delta) \approx f(u_n) + J_n \delta, \quad J_n = \frac{\partial f}{\partial u}(u_n)$$

Получаем:

$$(I - \gamma \tau J_n) k_i = f \left(u_n + \tau \sum_{j=1}^{i-1} a_{ij} k_j \right) + \tau J_n \sum_{j=1}^{i-1} c_{ij} k_j$$

$$u_{n+1} = u_n + \tau \sum m_i k_i$$

Вместо решения нелин. системы (как в неявных РК) — решаем линейную.

Пример: одностадийный метод Розенброка второго порядка

$$\left(I - \frac{1}{2}\tau J_n\right) k_1 = f(u_n), \quad u_{n+1} = u_n + \tau k_1$$

- 1 стадия, 1 шаг
- Порядок: 2
- Устойчивость: **A-устойчивый**
- Требуется только одна линейная система на шаг

Преимущества:

- A- и L-устойчивость
- Подходят для жёстких систем
- Только линейная система на шаг
- Возможна реализация адаптивного шага

Недостатки:

- Требуется якобиан
- Трудности при сильно нелинейных системах
- Меньше порядок по сравнению с классическими РК

Методами Гира обычно называют неявные многошаговые методы обратного дифференцирования (BDF, Backward Differentiation Formula).

Наиболее употребимый вариант – BDF2:

$$\frac{3y_{n+1} - 4y_n + y_{n-1}}{2h} = f(t_{n+1}, y_{n+1}).$$

Их основные свойства:

- особенно эффективны для жёстких систем ОДУ,
- являются неявными,
- требуют решения нелинейной или линеаризованной системы на каждом шаге.

https://intuit.ru/studies/professional_skill_improvements/1374/courses/168/lecture/4606?page=8

Задача 5

1. Реализовать явный и неявный методы Эйлера 1-го порядка и метод Рунге-Кутты 4-го порядка точности для системы ОДУ произвольного размера, исследовать порядок аппроксимации (пример выбрать самостоятельно).
2. Решить явным и неявным методом Эйлера жесткую систему уравнений для произвольных начальных условий и произвольного интервала интегрирования:

$$\begin{cases} u' = 998 u + 1998 v \\ v' = -999 u - 1999 v \end{cases}.$$

3. Решить явным методом РК4 систему уравнений хищник-жертва

$$\begin{cases} \dot{x} = a x - b x y \\ \dot{y} = c x y - d y \end{cases}$$

при $a = 10$, $b = 2$, $c = 2$, $d = 10$. Нарисовать фазовые траектории для различных начальных данных. Определить типы особых точек.

Калиткин Н. Н.: Численные методы

Смирнов С. В.: ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ, Часть II

<http://www.phys.nsu.ru/smirnov/ovf2-2017-09-18.pdf>

Хакимзянов Г.С., Черный С.Г. Методы вычислений: В 4 ч.: Учеб. пособие.
– Новосибирск: Новосиб. гос. ун-т., 2003. – Часть 1. Численные методы
решения задачи Коши для обыкновенных дифференциальных уравнений.

<http://www.ict.nsc.ru/matmod/files/textbooks/KhakiCherny1.pdf>

Хакимзянов Г.С., Черный С.Г. Методы вычислений: В 4 ч.: Учеб. пособие.
– Новосибирск: Новосиб. гос. ун-т., 2005. – Часть 2. Численные методы
решения краевых задач для обыкновенных дифференциальных уравнений.

[http:](http://www.ict.nsc.ru/matmod/files/textbooks/KhakimzyanovCherny-2.pdf)

[//www.ict.nsc.ru/matmod/files/textbooks/KhakimzyanovCherny-2.pdf](http://www.ict.nsc.ru/matmod/files/textbooks/KhakimzyanovCherny-2.pdf)

Дискретное преобразование Фурье

Чеховской Игорь Сергеевич

Преобразование Фурье и ряд Фурье

Используется в большом числе приложений.

Интегральное преобразование Фурье:

$$\tilde{f}(\omega) = \int_{-\infty}^{+\infty} f(t)e^{i\omega t} dt, \quad f(t) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} \tilde{f}(\omega)e^{-i\omega t} d\omega,$$

$\omega = 2\pi\nu$ – угловая (циклическая) частота (рад/с), ν – частота (1/с, Гц).

Ряд Фурье T -периодической функции:

$$f(x) = \sum_{m=-\infty}^{+\infty} a_m e^{ik_m x}, \quad a_m = \frac{1}{T} \int_0^T f(x) e^{-ik_m x} dx,$$

где $k_m = 2\pi m/T$.

Выбор знака в показателе экспоненты прямого преобразования Фурье, как и выбор коэффициента перед интегралом является произвольным.

Важно лишь, чтобы знаки в показателе в определении прямого и обратного преобразования были противоположными, а произведение коэффициентов перед интегралами было равно $(2\pi)^{-1}$.

δ -функция: физический смысл

Пусть у нас есть импульс, сосредоточенный около точки $t = a$:

- он почти всюду равен нулю;
- он очень высокий;
- его площадь равна 1.

В пределе, когда ширина импульса стремится к нулю, а площадь сохраняется, получают δ -**функцию Дира́ка** $\delta(t - a)$.

Ее удобно воспринимать как объект со свойствами

$$\delta(t - a) = 0 \quad \text{при } t \neq a,$$

$$\int_{-\infty}^{+\infty} \delta(t - a) dt = 1.$$

Самое важное свойство:

$$\int_{-\infty}^{+\infty} g(t) \delta(t - a) dt = g(a).$$

То есть $\delta(t - a)$ как бы “выбирает” значение функции в точке $t = a$.

δ -функция: строгая математическая трактовка

С математической точки зрения $\delta(t - a)$ — это не обычная функция, а **обобщенная функция** (или **распределение**).

Она определяется не своими значениями в точках, а действием на достаточно гладкие пробные функции $\varphi(t)$:

$$\langle \delta(t - a), \varphi(t) \rangle = \varphi(a).$$

Если говорить чуть более строго, обычно берут

$$\varphi \in C_c^\infty(\mathbb{R}),$$

то есть бесконечно гладкие функции с компактным носителем.

Запись

$$\int_{-\infty}^{+\infty} \varphi(t) \delta(t - a) dt = \varphi(a)$$

— это просто привычная форма записи того же самого определения.

Поэтому δ -функция — это удобный язык для описания **сосредоточенной в точке массы, заряда, импульса или спектральной линии**.

1. Сдвиг:

$$\int_{-\infty}^{+\infty} g(t) \delta(t - a) dt = g(a).$$

2. Масштабирование:

$$\delta(\alpha t) = \frac{1}{|\alpha|} \delta(t), \quad \alpha \neq 0.$$

3. Симметрия:

$$\delta(-t) = \delta(t), \quad \delta(a - t) = \delta(t - a).$$

4. Нормировка на отрезке, содержащем точку a :

$$\int_{a-\varepsilon}^{a+\varepsilon} \delta(t - a) dt = 1.$$

Простейшие примеры преобразования Фурье

Напомним, что в этой лекции используется соглашение

$$\tilde{f}(\omega) = \int_{-\infty}^{+\infty} f(t) e^{i\omega t} dt.$$

1. Для δ -функции:

$$\mathcal{F}\{\delta(t)\} = 1,$$

поскольку

$$\int_{-\infty}^{+\infty} \delta(t) e^{i\omega t} dt = e^{i\omega \cdot 0} = 1.$$

2. Для сдвинутой δ -функции:

$$\mathcal{F}\{\delta(t - a)\} = e^{i\omega a}.$$

3. Для прямоугольного импульса

$$f(t) = \begin{cases} 1, & |t| \leq b, \\ 0, & |t| > b, \end{cases}$$

имеем

$$\tilde{f}(\omega) = \int_{-b}^b e^{i\omega t} dt = \frac{e^{i\omega b} - e^{-i\omega b}}{i\omega} = 2 \frac{\sin(\omega b)}{\omega}.$$

Преобразование Фурье гармоник

Откуда в спектре синуса и косинуса берутся δ -функции?

1. Для константы:

$$\mathcal{F}\{1\} = 2\pi \delta(\omega).$$

2. Для комплексной экспоненты:

$$\mathcal{F}\{e^{-i\omega_0 t}\} = 2\pi \delta(\omega - \omega_0).$$

3. Для косинуса:

$$\cos(\omega_0 t) = \frac{e^{i\omega_0 t} + e^{-i\omega_0 t}}{2},$$

поэтому

$$\mathcal{F}\{\cos(\omega_0 t)\} = \pi\delta(\omega - \omega_0) + \pi\delta(\omega + \omega_0).$$

4. Для синуса:

$$\sin(\omega_0 t) = \frac{e^{i\omega_0 t} - e^{-i\omega_0 t}}{2i},$$

поэтому

$$\mathcal{F}\{\sin(\omega_0 t)\} = i\pi\delta(\omega - \omega_0) - i\pi\delta(\omega + \omega_0).$$

Итак, гармоническому сигналу соответствуют **отдельные спектральные линии**, а не сплошной спектр.

Связь между преобразованием Фурье и рядом Фурье

Рассмотрим интегральное преобр-е Фурье от T -периодической функции $f(t)$.

Распишем интеграл по всей числовой оси на сумму интегралов по отрезкам длины T и воспользуемся условием $f(t + T) = f(t) \quad \forall t$:

$$\tilde{f}(\omega) = \sum_{n=-\infty}^{+\infty} \int_{nT}^{(n+1)T} f(t) e^{i\omega t} dt = \lim_{N \rightarrow \infty} \sum_{n=-N}^N e^{i\omega nT} \int_0^T f(t') e^{i\omega t'} dt'$$

Интеграл в полученном выражении есть преобразование Фурье от одного периода функции $f(t)$, т. е. от

$$f_1(t) = \begin{cases} f(t) & \text{при } 0 \leq t < T \\ 0 & \text{при } t \notin [0, T). \end{cases}$$

Вычисляя сумму экспонент и переходя к пределу при $N \rightarrow \infty$, имеем:

$$\tilde{f}(\omega) = \tilde{f}_1(\omega) \cdot \frac{2\pi}{T} \sum_{m=-\infty}^{+\infty} \delta\left(\omega - \frac{2\pi m}{T}\right)$$

– спектр регулярной последовательности большого числа N одинаковых сигналов имеет вид эквидистантного набора узких линий. Интервал между спектральными линиями равен частоте повторения сигналов, а их огибающая совпадает со спектром единичного сигнала в последовательности.

https://en.wikipedia.org/wiki/Poisson_summation_formula

Дискретное преобразование Фурье (ДПФ)

$f(t)$ задана в n узлах равномерной сетки $t_k = Tk/n$, $k = 0, 1, n-1$.

$\tilde{f}(\omega)$ задана в n узлах равномерной сетки $\omega_j = 2\pi j/T$, $j = 0, 1, n-1$.

Используем формулу прямоугольников.

Прямое преобразование:

$$\tilde{f}_j = \frac{T}{n} \sum_{k=0}^{n-1} f_k \exp(i\omega_j t_k),$$

где $T = t_n - t_0 = n\tau$ – ширина сетки.

Обратное преобразование:

$$f_k = \frac{1}{T} \sum_j \tilde{f}_j \exp(-i\omega_j t_k),$$

$2\pi/T$ – шаг частотной сетки.

Частоты ДПФ для T -периодической функции совпадают с частотами ее ряда Фурье!

Функции $f(t)$ и $\tilde{f}(\omega)$ – периодические ($\omega_j \rightarrow \omega_j + 2\pi n/T = \omega_j + 2\pi/\tau$) (аналог ряда Фурье) (пример с НУШ).

Дискретное преобразование Фурье (ДПФ)

Формальное определение:

$$\tilde{f}_j = C_+ \sum_{k=0}^{n-1} f_k \exp\left(\frac{2\pi ijk}{n}\right), \quad f_k = C_- \sum_{j=0}^{n-1} \tilde{f}_j \exp\left(-\frac{2\pi ijk}{n}\right)$$

где C_+ и C_- – коэффициенты, $C_+ C_- = 1/n$.

ДПФ – вз.-однозначное отображение набора коэффициентов $f_k \leftrightarrow \tilde{f}_j$.

Основные свойства:

- ДПФ – линейное преобразование векторного пространства $\mathbb{C}^n$ с матрицами $\mathcal{F}_{jk}^+ = C_+ \exp(2\pi ijk/n)$ и $\mathcal{F}_{jk}^- = C_- \exp(-2\pi ijk/n)$
- Матрицы прямого и обратного ДПФ вз.-обратны: $\mathcal{F}_{jk}^+ \mathcal{F}_{kl}^- = \delta_{kl}$.
- При симметричном выборе коэффициентов $C_+ = C_- = 1/\sqrt{n}$ матрицы ДПФ унитарны: $(\mathcal{F}^\pm)^\dagger = \mathcal{F}^\mp = (\mathcal{F}^\pm)^{-1}$.
- ДПФ сохраняет квадратичную норму (энергию).
- ДПФ является хорошо обусловленным, т. е. последовательное применение прямого и обратного преобразований оставляет вектор f_k неизменным с точностью порядка машинного ε .
- Требуется $\mathcal{O}(n^2)$ операций (можно уменьшить до $\mathcal{O}(n \log n)$).

Быстрое Преобразование Фурье (БПФ)

Эффективный алгоритм для вычисления ДПФ, который значительно сокращает количество необходимых вычислений. Особенности:

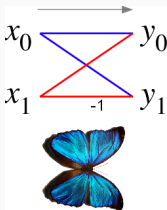
- **Высокая скорость:** снижает вычислительную сложность с $O(N^2)$ до $O(N \log N)$, где N – количество точек.
- **Эффективность:** позволяет анализировать сигналы большой длины в реальном времени.
- **Практичность:** является основой для многих технологий в обработке сигналов и изображений.

Основная идея БПФ:

- **Разделяй и властвуй:** разбиваем сложную задачу на несколько более простых подзадач.
- **Разделение последовательности:**
 - делим исходную последовательность на чётные и нечётные элементы.
 - это позволяет преобразовывать две последовательности длиной $N/2$ вместо одной длиной N .
- **Рекурсия:** повторяем процесс разбиения до тех пор, пока не дойдём до последовательностей длиной 1.
- **Комбинирование результатов:** используем специальные операции, называемые “бабочки”, для объединения результатов подзадач.

Быстрое Преобразование Фурье (БПФ)

Как работает “бабочка”:



- **Бабочка** — базовый строительный блок БПФ.
- Комбинирует результаты двух подзадач с использованием комплексных множителей (так называемых "вращающих коэффициентов").
- Позволяет эффективно пересчитать преобразования без повторных вычислений.

Применение:

- Спектральный анализ
- Вычисление свертки (convolution)

$$(x * y_N)_n \triangleq \sum_{\ell=-\infty}^{\infty} x_{\ell} \cdot (y_N)_{n-\ell} = \mathcal{F}^{-1}\{X \cdot Y\}_n.$$

- Вычисление взаимной корреляции (cross-correlation)

$$(x \star y_N)_n \triangleq \sum_{\ell=-\infty}^{\infty} x_{\ell}^* \cdot (y_N)_{n+\ell} = \mathcal{F}^{-1}\{X^* \cdot Y\}_n.$$

- Интерполяция $\left(\widehat{f(x - x_0)} = e^{-i\omega x_0} \hat{f}(\omega)\right)$.
- Вычисление произведения полиномов через свертку.
- Умножение больших целых чисел.
- Уравнения в частных производных $\left(\widehat{(f')} = i\omega \hat{f}\right)$.
- Сжатие данных (JPEG – discrete cosine transform (DCT))

Узлы обратной (частотной) сетки

Каждый узел обратной (частотной) сетки в общем случае может соответствовать различным физическим частотам:

$$\omega_j \sim \frac{2\pi j}{T} + m\Omega, \quad \Omega \equiv \frac{2\pi}{\tau},$$

где τ и $T = n\tau$ – шаг и ширина временной сетки,

n – количество узлов,

Ω – частота дискретизации (циклическая),

m – произвольное целое число.

Как устранить неоднозначность по m ?

Можем ли всегда брать $m = 0$?

Нет, так как спектр вещественного сигнала ($f(t) = f^*(t)$) по определению симметричен относительно нуля ($\tilde{f}(\omega) = \tilde{f}^*(-\omega)$).

Кроме того, получатся только неотрицательные частоты.

Узлы обратной (частотной) сетки

Чтобы найти правильное решение, рассмотрим гармонический сигнал $f(t) = \cos \omega_* t$ в “идеальном” случае:

- пусть мы можем регистрировать этот сигнал в течение длительного времени T , кратного периоду косинуса: $T = 2\pi q / \omega_*$, $q \in \mathbb{N}$, что позволит нам избежать эффекта частотола;
- пусть также мы можем использовать сетку со сколь угодно малым шагом τ и сколь угодно большим числом узлов $n = T/\tau$, чтобы избежать эффекта подмены частот.

Что даст ДПФ от косинуса в этом “хорошем” предельном случае?

Подставляя $\omega_* = 2\pi q / T$, $f_k = \cos(\omega_* t_k)$, $t_k = kT/n$ в определение ДПФ и пользуясь ортогональностью комплексных экспонент, получим в спектре лишь два ненулевых коэффициента:

$$\tilde{f}_j = \frac{C_+}{2} \sum_{k=0}^{n-1} \left(e^{2\pi i q k / n} + e^{-2\pi i q k / n} \right) e^{2\pi i j k / n} = \frac{C_+ n}{2} (\delta_{j, n-q} + \delta_{j, q})$$

Отличные от нуля коэффициенты соответствуют частотам

$$\omega_{n-q} = (m+1)\Omega - \omega_* \text{ и } \omega_q = m\Omega + \omega_*.$$

Узлы обратной (частотной) сетки

Поскольку речь идет о преобразовании Фурье функции $\cos \omega_* t$, мы ожидаем увидеть в спектре две линии: $\omega = \pm \omega_*$.

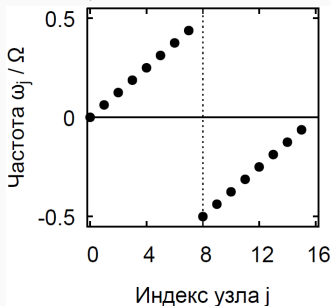
Поэтому для половины узлов ($0 \leq j < n/2$) нужно положить $m = 0$, для оставшейся половины считать $m = -1$:

$$\omega_j = \begin{cases} j\Omega/n, & \text{при } j < n/2 \\ (-1 + j/n)\Omega, & \text{при } j \geq n/2 \end{cases},$$

где $\Omega = 2\pi/\tau = 2\pi n/T$ – частота дискретизации (циклическая).

Максимальное по модулю значение частоты вдвое меньше частоты дискретизации ($\max \omega_j = \Omega/2 = \Omega_N$) и называется **частотой Найквиста (Найквиста-Шеннона)**.

Физический смысл: в каждом узле сетки j нужно разрешить неоднозначность выбора частоты ω_j таким образом, чтобы абсолютное значение $|\omega_j|$ было минимальным.



Связь ДПФ с рядом Фурье

Для T -периодической функции $f(t)$ ДПФ вычисляет коэффициенты ряда Фурье:

$$\tilde{f}_j = \frac{T}{N} \sum_{k=0}^{N-1} f(t_k) e^{i\omega_j t_k} \approx c_j = \frac{1}{T} \int_0^T f(t) e^{-i\omega_j t} dt$$

где $\omega_j = \frac{2\pi j}{T}$ — частоты ряда Фурье.

Частоты ДПФ для T -периодической функции совпадают с частотами ее ряда Фурье.

Чем больше берется отсчетов n (точек дискретизации) функции (т.е. чем больше частота дискретизации ν), тем больше частота Найквиста ($\Omega_N = \pi/\tau$), т.е. больше частот учтено.

Скорость сходимости

Зависит от скорости убывания коэффициентов Фурье c_j при $j \rightarrow \infty$:

- **Аналитические функции:** $|c_j| \sim e^{-\alpha j}$ — экспоненциальная сходимость
- **Гладкие функции (C^m):** $|c_j| \sim j^{-m-1}$ — степенная сходимость $O(N^{-m})$
- **Разрывные функции:** $|c_j| \sim j^{-1}$ — медленная сходимость $O(N^{-1})$ + явление Гиббса

Почему такая зависимость?

- Гладкость $\Leftrightarrow$ быстрое убывание спектра
- ДПФ для N точек учитывает частоты до $\Omega_N = \pi/\tau$
- Ошибка определяется отброшенными высокочастотными компонентами
- Для гладких функций эти компоненты экспоненциально малы

Теорема Котельникова (теорема отсчетов)

Независимо сформулирована У. В. Котельниковым (1933),
Х. Найквистом (1928) и К. Э. Шенноном (1949).

Формулировка. Пусть непрерывный сигнал $f(t)$ имеет ограниченный спектр: $\tilde{f}(\omega) = 0$ при $|\omega| > \omega_{\max}$. Если отсчёты снимаются с шагом τ , то для *совершенного* восстановления необходимо и достаточно

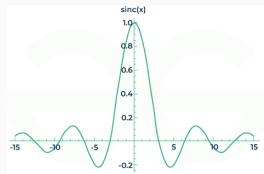
$$\Omega_s = \frac{2\pi}{\tau} > 2\omega_{\max} \iff \tau < \frac{\pi}{\omega_{\max}},$$

где Ω_s — циклическая частота дискретизации, а $\Omega_N = \Omega_s/2$ — частота Найквиста.

Формула реконструкции (sinc-интерполяция, ряд Котельникова):

$$f(t) = \sum_{k=-\infty}^{\infty} f(k\tau) \operatorname{sinc}\left(\frac{\pi}{\tau}(t - k\tau)\right), \quad \operatorname{sinc} x = \frac{\sin x}{x}.$$

Если условие $\Omega_s > 2\omega_{\max}$ нарушается, происходит катастрофическое и необратимое искажение — алиасинг (подмена частот, наложение спектров). Высокочастотные компоненты сигнала "маскируются" под низкочастотные, искажая истинную картину.



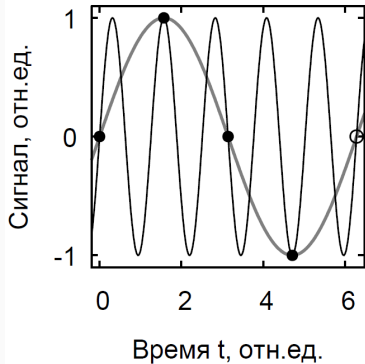
Подмена, переналожение, или маскировка частот. Также элайзинг, или алиасинг (от англ. *alias* – псевдоним, прозвище).

В силу периодичности узел временной сетки t_k соответствует бесконечному набору моментов времени $t_k + mT$, $m = 0, \pm 1, \pm 2 \dots$

Также и каждый узел ω_j обратной (частотной) сетки соответствует различным физическим частотам $\omega_j + 2\pi m/\tau$, $m \in \mathbb{Z}$, $\tau = T/n$.

Если гармонический сигнал с частотой ω измеряется в дискретные моменты времени с равными интервалами τ , то частота ω по результатам таких измерений может быть определена лишь по модулю: $\omega \bmod \frac{2\pi}{\tau}$.

Действительно, регистрируя изменение фазы $\delta\varphi$ сигнала между моментами времени измерений $t = 0, \tau, 2\tau, \dots$, мы никак не можем отличить его от $\delta\varphi + 2\pi k$ при произвольном целом k .



Графики $\sin t$ и $\sin 5t$:

их значения совпадают в узлах сетки с шагом $\tau = \pi/2$ (показаны круглыми маркерами), поскольку аргумент первой гармоники возрастает между узлами сетки на величину $\delta\varphi = \pi/2$, а пятой гармоники — на $\frac{\pi}{2} + 2\pi = \frac{5\pi}{2}$. Как следствие, частоты $\omega = 1$ и $\omega = 5$ оказываются неотличимыми на данной сетке.

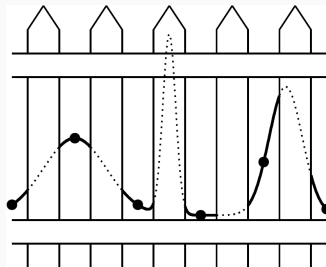
На этом принципе основано действие стробоскопа.

Из-за подмены частот колеса автомобилей в кинофильмах вращаются в противоположную движению сторону, а экраны мониторов на видеозаписях непривычно сильно мерцают.

Эффект частокола

Picket fence effect. Связан с потерей спектральной информации, попадающей между узлами обратной сетки.

Сплошные линии – видимые в промежутках между досками частокола участки спектра
Пунктирные линии – скрытые
Круглые маркеры – узловые точки обратной сетки



После наложения частокола на некоторую функцию (спектр) только один из трех пиков (первый) был передан без искажений. Второй (средний) пик полностью исчез, а высота третьего пика после наложения сетки оказалась в два раза меньше истинного значения.

Для исправления как минимум необходимо увеличить область регистрации сигнала T , чтобы уменьшить шаг обратной сетки!

Однако одно лишь увеличение ширины сетки T не всегда позволяет полностью решить проблему. Рассмотрим комплексную экспоненту:

$$f(t) = \exp(-i\omega_* t), \quad \omega_* = \left(q + \frac{1}{2}\right) \frac{2\pi}{T}, \quad q \in \mathbb{N}.$$

Частоту ω_* выберем так, чтобы на сетке ширины T укладывалось $q + \frac{1}{2}$ периодов осцилляций комплексной экспоненты.

Положив $C_+ = 1/n$ в определении ДПФ, получим:

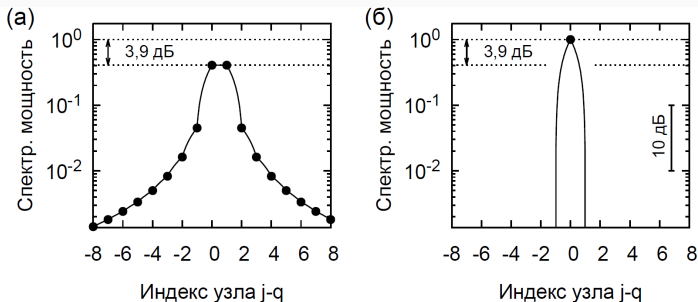
$$\tilde{f}_j = \frac{1}{n} \sum_{k=0}^{n-1} \exp\left(\frac{2\pi i(j - q - 1/2)k}{n}\right)$$

Так как частота ω_* является полуцелой (т. е. существует дробная часть у безразмерной частоты $\omega_* T / (2\pi) = \nu_*$) комплексные экспоненты не являются ортогональными.

Используя формулы для геом. прогрессии, получаем:

$$|\tilde{f}_j|^2 = \frac{1}{n^2} \left| \frac{\sin\left(\pi\left(j - q - \frac{1}{2}\right)\right)}{\sin\left(\frac{\pi}{n}\left(j - q - \frac{1}{2}\right)\right)} \right|^2 = \frac{1}{n^2 \sin^2\left(\frac{\pi}{n}\left(j - q - \frac{1}{2}\right)\right)}$$

Эффект частоты



Результат дискретного преобразования Фурье для гармонического сигнала с полуцелой (а) и целой (б) частотой на сетке с $n = 256$ узлами; сплошная кривая соответствует линейной интерполяции между узлами

Круглые маркеры соответствуют значениям функции в узлах обратной сетки, соединяющие их кривые – линейной интерполяции между узлами.

Как и следовало ожидать, максимум спектральной мощности достигается в узлах ω_q и ω_{q+1} , ближайших к частоте ω_* .

Проблемы:

вычислим максимальное значение спектральной мощности:

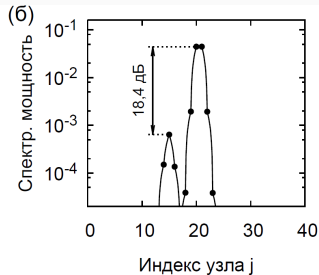
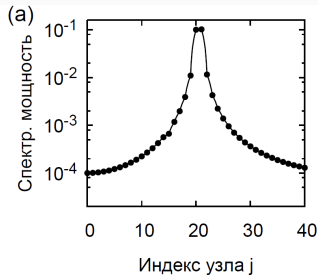
$$\max_j \left| \tilde{f}_j \right|^2 = \left(n \sin \left(\frac{\pi}{2n} \right) \right)^{-2} = \frac{4}{\pi^2} + \mathcal{O}(n^{-2}) \text{ при } n \gg 1$$

— разновидность преобразование Фурье, определяемая следующим образом:

$$F(t, \omega) = \int_{-\infty}^{\infty} f(\tau) W(\tau - t) e^{-i\omega\tau} d\tau,$$

где $W(\tau - t)$ — некоторая оконная функция. В случае дискретного преобразования оконная функция используется аналогично:

$$F(m, \omega) = \sum_{n=-\infty}^{\infty} f[n] w[n - m] e^{-j\omega n}.$$

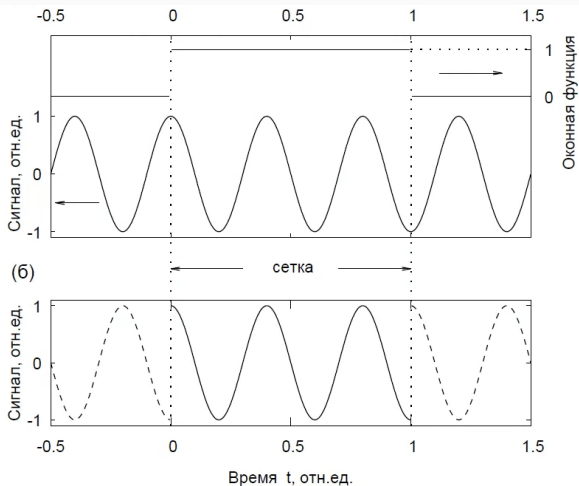


Результат дискретного преобразования Фурье, вычисленный с прямоугольным окном (а) и оконной функцией Ханна (б)

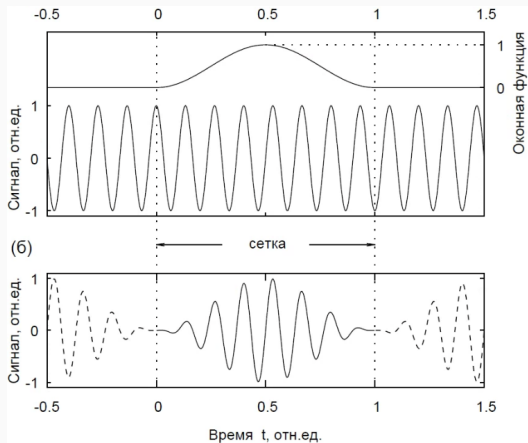
$$f(t) = 0.1 \sin(15t) + \cos(20.5t)$$

Прямоугольное окно ($f(t) \cdot h(t; T)$):

$$h_k = h(t_k) = \begin{cases} 1, & \text{при } 0 \leq k < n, \\ 0, & \text{при } k < 0 \text{ и } k \geq n. \end{cases}$$



Сигнал на сетке можно рассматривать как произведение $f(t)$ и ступенчатой оконной функции $h(t)$ (а), при этом периодическое продолжение сеточной функции fh может быть разрывным (б)



Использование окна Ханна делает периодическое продолжение сеточной функции непрерывным и гладким

Окно Ханна:

$$h_k = \frac{1}{2} \left(1 - \cos \left(\frac{2\pi k}{n} \right) \right), \quad 0 \leq k < n$$

`https://finufft.readthedocs.io/en/latest/`

Смирнов С. В.: ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ, Часть II
<http://www.phys.nsu.ru/smirnov/ovf2-2017-09-18.pdf>

Численное решение краевых задач для обыкновенных дифференциальных уравнений

Чеховской Игорь Сергеевич

1. Краевая задача для линейной системы ОДУ первого порядка
2. Краевая задача Штурма–Лиувилля для ОДУ второго порядка
3. Численное решение нелинейных краевых задач
4. Краевые задачи на собственные значения для обыкновенных дифференциальных уравнений
5. Решение краевой задачи методом Фурье

Краевая задача для системы ОДУ первого порядка

Рассмотрим линейную систему

$$\frac{du}{dt} = A(t)u + f(t), \quad u(t) \in \mathbb{R}^n, \quad t \in [0, L].$$

Краевые условия задаются на концах отрезка:

$$Ru(0) + Su(L) = q,$$

где

$$R, S \in \mathbb{R}^{n \times n}, \quad q \in \mathbb{R}^n.$$

Требуется найти функцию

$$u(t),$$

которая удовлетворяет дифференциальной системе на всем отрезке $[0, L]$ и краевым условиям на его концах.

В отличие от задачи Коши, здесь условия связывают значения решения на разных концах отрезка.

Структура решения линейной неоднородной системы

Сначала рассматривается однородная система

$$\frac{du}{dt} = A(t)u.$$

Пусть

$$u^{(1)}(t), u^{(2)}(t), \dots, u^{(n)}(t)$$

— линейно независимые решения этой системы. Тогда ее общее решение имеет вид

$$u_h(t) = c_1 u^{(1)}(t) + c_2 u^{(2)}(t) + \dots + c_n u^{(n)}(t).$$

Пусть далее

$$\bar{u}(t)$$

— некоторое частное решение неоднородной системы

$$\frac{d\bar{u}}{dt} = A(t)\bar{u} + f(t).$$

Тогда общее решение исходной системы записывается в виде

$$u(t) = \bar{u}(t) + u_h(t),$$

то есть

$$u(t) = \bar{u}(t) + c_1 u^{(1)}(t) + \dots + c_n u^{(n)}(t).$$

Как используются краевые условия

Подставим общее решение

$$u(t) = \bar{u}(t) + \sum_{k=1}^n c_k u^{(k)}(t)$$

в краевое условие

$$Ru(0) + Su(L) = q.$$

Получим

$$R\bar{u}(0) + S\bar{u}(L) + \sum_{k=1}^n c_k (Ru^{(k)}(0) + Su^{(k)}(L)) = q.$$

Это система линейных алгебраических уравнений относительно коэффициентов

$$c_1, \dots, c_n.$$

Следовательно, линейная краевая задача для системы первого порядка сводится к двум шагам:

1. построение частного решения и набора линейно независимых решений однородной системы;
2. определение коэффициентов из краевых условий.

Для краевых задач используются два основных подхода.

1. Методы стрельбы.

Краевая задача сводится к одной или нескольким задачам Коши с неизвестными начальными параметрами. Эти параметры подбираются так, чтобы выполнялись условия на другом конце отрезка.

Метод плохо подходит для жестких систем ОДУ. Для них чаще используют

2. Прямые дискретные методы.

Дифференциальная задача аппроксимируется на сетке, после чего возникает система алгебраических уравнений относительно значений решения в узлах.

Линейная краевая задача для ОДУ второго порядка

Рассмотрим линейную краевую задачу второго порядка

$$\frac{d}{dt} \left[g(t) \frac{du}{dt} \right] + h(t) \frac{du}{dt} + s(t)u = f(t), \quad t \in [a, b],$$

с граничными условиями

$$A \frac{du}{dt} + Bu = D, \quad t = a,$$

$$A' \frac{du}{dt} + B'u = D', \quad t = b.$$

Коэффициенты

$$g(t), h(t), s(t)$$

могут зависеть от переменной t .

Для таких задач естественно применять конечно-разностную аппроксимацию, так как после дискретизации возникают уравнения, связывающие соседние значения сеточной функции.

Разностная аппроксимация

Коэффициент $g(t)$ может быть не гладким, а лишь кусочно-гладким. Например, это возникает в задачах теплопроводности при переходе от одного материала к другому.

Представим разностную задачу в виде

$$\frac{1}{\tau} \left(g_{n+1/2} \frac{u_{n+1} - u_n}{\tau} - g_{n-1/2} \frac{u_n - u_{n-1}}{\tau} \right) + h_n \frac{u_{n+1} - u_{n-1}}{2\tau} + s_n u_n = f_n, \quad n = 1, \dots, l$$

$$g_{n+1/2} = g \left(t_n + \frac{\tau}{2} \right), \quad h_n = h(t_n),$$

$$A \frac{u_1 - u_0}{\tau} + B u_0 = D, \quad t = a,$$

$$A' \frac{u_N - u_{N-1}}{\tau} + B' u_N = D', \quad t = b.$$

Такая аппроксимация приводит к консервативной разностной схеме и естественным образом учитывает поток через границы ячеек сетки.

Простейшая разностная схема

Выписанные выше соотношения определяют простейшую **разностную схему**. Под разностной схемой понимается совокупность разностных уравнений для определения значений сеточной функции внутри расчетной области, дополненная граничными условиями.

Для определения значений сеточной функции получаем СЛАУ с трехдиагональной матрицей

$$-b_0 u_0 + c_0 u_1 = d_0,$$

$$a_n u_{n-1} - b_n u_n + c_n u_{n+1} = d_n, \quad n = 1, \dots, N-1,$$

$$a_N u_{N-1} - b_N u_N = d_N,$$

где

$$a_n = \frac{g_{n-1/2}}{\tau^2} - \frac{h_n}{2\tau}, \quad c_n = \frac{g_{n+1/2}}{\tau^2} + \frac{h_n}{2\tau},$$

$$b_n = a_n + c_n - s_n, \quad d_n = f_n,$$

$$b_0 = \frac{A}{\tau} - B, \quad c_0 = \frac{A}{\tau}, \quad d_0 = D,$$

$$a_N = \frac{A'}{\tau}, \quad b_N = \frac{A'}{\tau} + B', \quad d_N = -D'.$$

Эта СЛАУ представима в каноническом виде

$$A' u = D,$$

где A' – матрица

$$A' = \begin{pmatrix} -b_0 & c_0 & 0 & \cdots & 0 \\ a_1 & -b_1 & c_1 & \cdots & 0 \\ 0 & a_2 & -b_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -b_N \end{pmatrix},$$

а u и D – векторы-столбцы,

$$D = (d_0, d_1, \dots, d_N)^T.$$

Такая матрица имеет ненулевые элементы только на главной диагонали и на двух соседних диагоналях.

Для решения таких систем используется специальный экономичный алгоритм – **метод прогонки**.

Метод стрельбы

Рассмотрим нелинейную краевую задачу для системы ОДУ

$$\begin{cases} \frac{du}{dt} = f(t, u), & t \in [0, L], \\ G[u(0), u(L)] = 0, \end{cases} \quad u \in \mathbb{R}^n.$$

Введем неизвестный начальный параметр

$$u(0) = \alpha, \quad \alpha \in \mathbb{R}^n,$$

и рассмотрим соответствующую задачу Коши

$$\frac{du}{dt} = f(t, u), \quad u(0) = \alpha.$$

Пусть ее решение обозначено через

$$u(t, \alpha).$$

Тогда краевая задача сводится к нелинейной системе алгебраических уравнений относительно параметра α :

$$\Phi(\alpha) = G(\alpha, u(L, \alpha)) = 0.$$

Эта система решается итер. методами, например методом Ньютона.

Пример: метод стрельбы

Рассмотрим краевую задачу

$$u'' = \sin x,$$

$$u(0) = 0,$$

$$u(L) = 0.$$

Введем новую неизвестную $v = u'$. Тогда получаем систему 1-го порядка

$$u' = v,$$

$$v' = \sin x.$$

На левом конце задано условие $u(0) = 0$, но значение $v(0)$ неизвестно.

Обозначим его через параметр s :

$$u(0) = 0, \quad v(0) = s.$$

Для каждого s решается задача Коши, после чего вычисляется значение $u(L; s)$. Требуемое значение параметра определяется из уравнения

$$u(L; s) = 0.$$

Тем самым краевая задача сводится к задаче поиска неизвестного начального наклона.

Метод квазилинеаризации (метод Ньютона)

Рассмотрим нелинейную краевую задачу второго порядка

$$\frac{d^2 u}{dt^2} = f(u), \quad u(0) = U_1, \quad u(L) = U_2.$$

Построим ее простейшую разностную аппроксимацию:

$$\frac{u_{n-1} - 2u_n + u_{n+1}}{\tau^2} = f(u_n), \quad n = 1, \dots, N-1,$$
$$u_0 = U_1, \quad u_N = U_2, \quad \tau = \frac{L}{N}.$$

Метод квазилинеаризации состоит в том, что на каждой итерации нелинейная задача заменяется линейной, полученной линеаризацией около предыдущего приближения.

Для этого задаем начальное приближение

$$u_n^0 = \varphi(t_n)$$

и используем линейное приближение правой части:

$$f(u_n^{i+1}) \approx f(u_n^i) + f'_u(u_n^i)(u_n^{i+1} - u_n^i).$$

Итерационный процесс метода квазилинеаризации

Подставляя линеаризацию в разностное уравнение, получаем на каждой итерации линейную краевую задачу:

$$\frac{u_{n-1}^{i+1} - 2u_n^{i+1} + u_{n+1}^{i+1}}{\tau^2} = f(u_n^i) + f'_u(u_n^i)(u_n^{i+1} - u_n^i),$$
$$n = 1, \dots, N-1, \quad u_0^{i+1} = U_1, \quad u_N^{i+1} = U_2.$$

Начальное приближение должно удовлетворять граничным условиям:

$$\varphi(0) = U_1, \quad \varphi(L) = U_2.$$

На первой итерации при $i = 0$ получаем первое линейное приближение к решению. Затем процесс повторяется.

На каждом шаге возникает линейная разностная краевая задача, которую можно решать методом прогонки.

Итерации продолжаются до выполнения условия

$$\|u^{i+1} - u^i\| \leq \varepsilon.$$

Аппроксимация граничных условий

Для граничных условий 1-го рода аппроксимация обычно не вызывает трудностей.

Сложнее обстоит дело для условий на производную и смешанных граничных условий. Рассм. линейную задачу с постоянными коэфф-ми

$$\frac{d^2 u}{dt^2} + h \frac{du}{dt} + su = f,$$
$$u'(0) = a, \quad u'(1) = b.$$

Во внутренних узлах используем разностную схему

$$\frac{u_{n+1} - 2u_n + u_{n-1}}{\tau^2} + h \frac{u_{n+1} - u_{n-1}}{2\tau} + su_n = f_n.$$

Если на границе заменить производную односторонней разностью первого порядка, например

$$u_1 - u_0 = \tau a, \quad u_N - u_{N-1} = \tau b,$$

то порядок аппроксимации на границе окажется ниже, чем во внутренних узлах.

Поэтому возникает задача повышения порядка аппроксимации граничных условий.

В результате получили разностную схему для аппроксимации дифференциальной задачи. Несмотря на то, что во внутренних узлах разностные формулы приближают дифференциальные уравнения со 2-м порядком аппроксимации, решение по этой схеме получается только с 1-м порядком из-за понижения порядка аппроксимации в граничных точках. Естественно было бы повысить порядок схемы до 2-го во всех точках, включая граничные.

Наиболее распространены три способа.

1. Использование формул одностороннего дифференцирования более высокого порядка точности. Подход очевиден, однако имеет недостаток – матрица системы уравнений для определения решения будет уже не трехдиагональной, следовательно, алгоритм прогонки неприменим. Можно избавиться от этого недостатка, проведя перед численным решением системы соответствующие алгебраические преобразования.

Повышение порядка аппроксимации на границе

2. Использование фиктивной ячейки (фиктивного узла). Рассмотрим расширение сеточной области. Введем в рассмотрение узлы с индексами 0 и $N + 1$, находящиеся формально за пределами рассматриваемой области. Пусть в них тоже определено значение сеточной функции. Тогда узел с индексом 0 выступает как внутренний узел, и в нем может быть записано соотношение

$$\frac{u_1 - 2u_0 + u_{-1}}{\tau^2} + h \frac{u_1 - u_{-1}}{2\tau} + su_0 = f_0,$$

при этом можно для граничного условия использовать формулу для вычисления производной со вторым порядком аппроксимации по формуле центральных разностей

$$\frac{u_1 - u_{-1}}{\tau} = a.$$

Из этих двух уравнений осталось только исключить значение в фиктивном узле (фиктивной ячейке).

Такой метод аппроксимации граничных условий очень хорошо зарекомендовал себя при решении нелинейных задач для уравнений в частных производных. В таком случае значения сеточной функции в фиктивных точках не исключаются из системы, а тоже находятся численно.

Повышение порядка аппроксимации на границе

На правом конце отрезка формулы аналогичны.

Можно обобщить этот подход и для граничных условий третьего рода.

3. Использование ряда Тейлора. Для формулы на левой границе области интегрирования

$$u_1 - u_0 = \tau a$$

выпишем разложение:

$$\frac{u_1 - u_0}{\tau} = u'(0) + \frac{\tau}{2} u''(0) + o(\tau).$$

Используя само дифференциальное уравнение, можно выразить значение второй производной функции в окрестности границы:

$$\frac{d^2 u(0)}{dt^2} = f(0) - h \frac{du(0)}{dt} - su(0),$$

откуда следует

$$\frac{u_1 - u_0}{\tau} = a + \frac{\tau}{2} \left(f(0) - h \frac{du(0)}{dt} - su(0) \right) + o(\tau).$$

Заменяя в последнем соотношении производную на конечную разность, получим

$$\frac{u_1 - u_0}{\tau} = a + \frac{\tau}{2} \left(f_0 - h \frac{u_1 - u_0}{\tau} - su_0 \right).$$

Осталось только привести это соотношение к виду, удобному для использования в методе прогонки.

В случае линейных уравнений с постоянными коэффициентами два последних способа приводят к одному и тому же выражению для значения сеточной функции в граничном узле. Для нелинейных уравнений эти способы будут различаться, но каждый из них приводит к повышению порядка аппроксимации граничных условий и, следовательно, разностной схемы.

Периодические граничные условия и метод Фурье

Рассмотрим краевую задачу

$$y'' = f(x), \quad 0 < x < 2\pi,$$

с периодическими граничными условиями

$$y(0) = y(2\pi), \quad y'(0) = y'(2\pi).$$

Для таких задач естественно использовать тригонометрические полиномы и дискретное преобразование Фурье.

Идея метода состоит в том, что решение и правая часть представляются в виде сумм гармоник:

$$y(x) = \sum_{k \in \mathbb{Z}} \hat{y}_k e^{ikx}, \quad f(x) = \sum_{k \in \mathbb{Z}} \hat{f}_k e^{ikx}.$$

Тогда дифференцирование выполняется покомпонентно:

$$y''(x) = \sum_{k \in \mathbb{Z}} (-k^2) \hat{y}_k e^{ikx}.$$

Поэтому исходное дифференциальное уравнение переходит в алгебраические соотношения для коэффициентов Фурье.

Дискретное преобразование Фурье на равномерной сетке

Введем равномерную периодическую сетку

$$x_j = \frac{2\pi j}{N}, \quad j = 0, 1, \dots, N-1.$$

На этой сетке значения функции

$$y_j = y(x_j), \quad f_j = f(x_j)$$

связываются с дискретными коэффициентами Фурье

$$\hat{y}_k, \quad \hat{f}_k.$$

Для четного N частоты удобно нумеровать так:

$$k = 0, 1, \dots, \frac{N}{2} - 1, -\frac{N}{2}, \dots, -1.$$

Численное дискретное преобразование Фурье переводит набор значений

$$\{f_j\}_{j=0}^{N-1}$$

в набор коэффициентов

$$\{\hat{f}_k\}.$$

После этого уравнение решается отдельно для каждой гармоники.

Как решается уравнение в Фурье-пространстве

Подставляя Фурье-разложения в уравнение

$$y'' = f,$$

получаем для каждой гармоники

$$-k^2 \hat{y}_k = \hat{f}_k.$$

Следовательно, для всех ненулевых частот

$$\hat{y}_k = -\frac{\hat{f}_k}{k^2}, \quad k \neq 0.$$

Для нулевой гармоники возникает соотношение $0 = \hat{f}_0$. Это условие разрешимости периодической задачи:

$$\hat{f}_0 = 0 \quad \Longleftrightarrow \quad \int_0^{2\pi} f(x) dx = 0.$$

После этого коэффициент $\hat{y}_0$ остается произвольным, то есть решение определяется с точностью до добавления константы.

Чтобы сделать решение единственным, обычно накладывают условие нулевого среднего:

$$\hat{y}_0 = 0.$$

Алгоритм решения периодической задачи методом ДПФ имеет вид:

1. задать значения правой части $f_j = f(x_j)$ на равномерной сетке;
2. вычислить дискретное преобразование Фурье и получить коэффициенты $\hat{f}_k$;
3. для всех $k \neq 0$ вычислить

$$\hat{y}_k = -\frac{\hat{f}_k}{k^2};$$

4. положить

$$\hat{y}_0 = 0;$$

5. выполнить обратное дискретное преобразование Фурье и получить значения решения y_j .

1. Методом прогонки решить разностный аналог граничной задачи для уравнения

$$y'' = \sin x$$

на промежутке $0 < x < \pi$. Реализовать различные варианты граничных условий на функцию и ее производную.

2. Для уравнения

$$y'' = \sin x$$

на промежутке $0 < x < 2\pi$ с периодическими граничными условиями

$$y(0) = y(2\pi), \quad y'(0) = y'(2\pi)$$

реализовать численное решение с помощью обычного дискретного преобразования Фурье (FFT).

3. Исследовать порядок аппроксимации.

Краевые задачи на собственные значения для обыкновенных дифференциальных уравнений

Краевые задачи на собственные значения достаточно часто встречаются в физических приложениях. Например, это задача определения собственных колебаний струны, сводящаяся к ОДУ вида

$$\frac{d}{dt} \left[k(t) \frac{du}{dt} \right] + \lambda r(t) u = 0.$$

В приведенном уравнении краевые условия зависят от способа закрепления струны.

Это и задачи собственных колебаний упругого стержня (ОДУ четвертого порядка), нахождения энергетических уровней атома водорода, вычисления критических нагрузок в теории стержней и оболочек и др.

В задачах на собственные значения добавляется еще один пристрелочный параметр – λ , поэтому эти задачи часто решаются методом стрельбы.

Приведем простейший пример – одно дифференциальное уравнение первого порядка с двумя краевыми условиями (второе краевое условие появляется из-за присутствия неизвестного параметра λ):

$$\frac{du}{dt} + f(u, t, \lambda) = 0, \quad u(0) = u_1.$$

Если не учитывать правое краевое условие, то получим задачу Коши. Ее численное интегрирование приводит к некоему значению на правом конце, зависящему от λ и, вообще говоря, не равному u_2 .

Варьируя параметр λ , можно добиться выполнения правого краевого условия с некоторой заданной точностью. При этом, разумеется, используются методы численного нахождения корней алгебраического уравнения, обычно метод касательных.

Пример задачи на собственные значения второго порядка

Краевая задача на собственные значения для ОДУ второго порядка с нулевыми краевыми условиями:

$$\frac{d^2 u}{dt^2} + b'(t) \frac{du}{dt} + [c'(t) + \lambda]u = 0,$$

$$u(0) = u(L) = 0.$$

Поскольку это уравнение второго порядка с неизвестным параметром λ (собственное значение дифференциального оператора), то для его решения требуется третье условие. Однако в силу линейности и однородности задачи решение определяется с точностью до произвольного постоянного множителя, что и является неявным заданием третьего условия. Его можно задать, например, следующим образом:

$$\frac{du(0)}{dt} = 1.$$

Трудности при использовании метода стрельбы возникают, если соответствующая задача Коши плохо обусловлена, а также в случае жестких краевых задач. В этих случаях появляется сильная зависимость численного решения от пристрелочного параметра λ .

В качестве тестового примера для сравнения с результатами численного расчета удобно использовать модельную краевую задачу на собственные значения:

$$\frac{d^2 u}{dt^2} + \lambda u = 0, \quad u(0) = u(L) = 0,$$

имеющую точное решение

$$\lambda_k = \left(\frac{\pi k}{L}\right)^2, \quad u_k(t) = \sin\left(\frac{\pi k t}{L}\right), \quad k = 1, 2, \dots$$

Непосредственной подстановкой показывается, что решениями соответствующей разностной задачи на собственные значения

$$\frac{u_{n-1} - 2u_n + u_{n+1}}{\tau^2} + \lambda u_n = 0, \quad n = 1, 2, \dots, N-1, \quad \tau N = L, \quad u_0 = u_N = 0,$$

являются собственные значения и собственные функции

$$\lambda_k^\tau = \frac{4}{\tau^2} \sin^2 \left(\frac{\pi k \tau}{2L} \right), \quad u_n^\tau = \sin \left(\frac{\pi k t_n}{L} \right), \quad k, n = 1, 2, \dots, N-1.$$

Откуда видно, что

$$\lim_{\tau \rightarrow 0} \frac{4}{\tau^2} \sin^2 \left(\frac{\pi k \tau}{2L} \right) = \left(\frac{\pi k}{L} \right)^2 = \lambda_k,$$

т.е. имеет место сходимость решения разностной задачи к решению дифференциальной задачи

$$\lambda_k^\tau \rightarrow \lambda_k, \quad \tau \rightarrow 0.$$

Лекция на «ИНТУИТ» https://intuit.ru/studies/professional_skill_improvements/1374/courses/168/lecture/4608

Хакимзянов Г.С., Черный С.Г. Методы вычислений: В 4 ч.: Учеб. пособие. – Новосибирск: Новосиб. гос. ун-т., 2005. – Часть 2. Численные методы решения краевых задач для обыкновенных дифференциальных уравнений. <http://www.ict.nsc.ru/matmod/files/textbooks/KhakimzyanovCherny-2.pdf>

Князева А. Г.: Различные варианты метода прогонки http://eqworld.ipmnet.ru/ru/library/books/Knyazeva_progonka_2006ru.pdf

Вычислительные методы линейной алгебры. Численное нахождение собственных значений

Чеховской Игорь Сергеевич

Численное нахождение собственных значений

Проблема собственных значений (спектральная задача, eigenvalue problem) – задача вычисления собственных значений и собственных векторов (с. з. и с. в.).

Различают полную проблему с. з. и частичную проблему с. з., когда требуется найти часть с. з. и/или с. в.

Особенности проблемы с. з.:

- необходимость выхода в комплексную плоскость $\mathbb{C}$, даже для вещественных матриц
- нелинейность ($Ax = \lambda x$, где λ и x – неизвестные), т.е. возможна неустойчивость решения.

Непрерывная спектральная задача (напр., $\hat{H}\psi(x) = E\psi(x)$) может быть также представлена в виде матричной спектральной задачи после перехода к ее конечномерному (матричному) аналогу $Ax = \lambda x$.

Численное нахождение собственных значений

Характеристический полином (степени n):

$$0 = \det |A - \lambda I| = P_n(\lambda)$$

Алгебраические методы не подходят: вычисление за $\mathcal{O}(n!)$ операций.

Теорема Абеля-Руффини. Корни алгебраического уравнения степени ≥ 5 не выражаются в конечном виде через операции сложения, вычитания, умножения, деления и взятия корня.

Численные методы решения спектральной задачи делятся на прямые и итерационные.

Методы для несимметричных матриц:

- метод интерполяции
- степенной метод
- QR -алгоритм

Методы для симметричных матриц:

- метод Якоби

Метод интерполяции

Ш. Е. Микеладзе (1948).

Выберем произвольным образом $n + 1$ точку $x_0, \dots, x_n$ (желательно выбирать ближе к спектру для устойчивости вычислений) и вычислим в них характеристический полином $P(x_j) = y_j, j = 0, \dots, n$.

Построим интерполяционный полином $\tilde{P}_n$ степени n , проходящий через точки $(x_0, y_0), \dots, (x_n, y_n)$.

В силу однозначности построения полинома $P_n(x) \equiv \tilde{P}_n(x) \forall x$.

Корни полинома и есть искомый спектр. Для нахождения спектра необходимо:

- вычислить $n + 1$ определитель $\det |A - x_j I|$, т. е. $\approx \frac{2}{3} n^4$ операций
- построить интерполяционный полином, $\mathcal{O}(n^2)$ операций
- найти корни полинома: напр., итерационно методом Ньютона для k различных нач. условий, $\mathcal{O}(kn)$ операций

Недостатки метода:

- вычисление конечных разностей высокого порядка – плохо обусловленная задача
- несколько этапов алгоритма, сложность программирования

Степенной метод (счет на установление, power method) – численный метод для решения частичной проблемы с. з. – нахождения макс. и мин. по модулю с. з. и соответствующего с. в.

Такие задачи часто встречаются на практике (определение основной и нескольких первых мод в оптическом волокне, основное состояние в квантовой механике).

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|.$$

λ_1 – доминирующее с. з.

Идея метода

Возьмем $x \in \mathbb{C}$, $x \neq 0$, и вычислим Ax .

В Ax возрастет доля направления, отвечающего наиб. по модулю с. з. λ_1 матрицы A . Повторим умножение необходимое число раз и получим “почти с. в.” для λ_1 .

Организуем последовательное умножение $x^{(k+1)} \leftarrow Ax^{(k)}$, $k = 0, 1, 2, \dots$, задавшись $x^{(0)}$.

Тогда $x^{(k)}$ сходится к с. в. A , а в качестве с. з. можно взять:

- отношение $x_i^{(k+1)} / x_i^{(k)}$ для некоторого фиксированного i
- отношение проекций $x^{(k+1)}$ и $x^{(k)}$ на какое-либо направление $l^{(k)}$, неортогональное $x^{(k)}$:

$$\frac{\langle x^{(k+1)}, l^{(k)} \rangle}{\langle x^{(k)}, l^{(k)} \rangle}.$$

Второй способ более устойчив. Удобно взять в нем $l^{(k)} = x^{(k)}$.

Для симметричных (эрмитовых) положительно определенных матриц можно брать отношение

$$\frac{\|x^{(k+1)}\|_2}{\|x^{(k)}\|_2}, \quad x^{(k+1)} = Ax^{(k)}.$$

Что будет, если в $x^{(0)}$ не представлено направление, отвечающее λ_1 ?
Реализация метода на компьютере неизбежно вызовет ошибки округления, которые приведут к появлению ненулевого коэффициента при с. в. для λ_1 , т.е. метод “исправится” сам. Если этого не происходит – запускаем с различными начальными векторами (мультистарт).

При последовательных умножениях можно выйти за границы представимых чисел:

при $|\lambda_1| > 1$ получим **переполнение**,

при $|\lambda_1| < 1$ получим **исчезновение порядка**.

Нужна нормировка!

Псевдокод:

```
 $k \leftarrow 0;$   
выбираем вектор  $x^{(0)} \neq 0;$   
нормируем  $x^{(0)} \leftarrow x^{(0)} / \|x^{(0)}\|_2;$   
DO WHILE ( метод не сошёлся )  
     $y^{(k+1)} \leftarrow Ax^{(k)};$   
     $\tilde{\lambda} \leftarrow \langle y^{(k+1)}, x^{(k)} \rangle;$   
     $x^{(k+1)} \leftarrow y^{(k+1)} / \|y^{(k+1)}\|_2;$   
     $k \leftarrow k + 1;$   
END DO
```

Степенной метод исключительно прост в реализации, однако достаточно эффективен лишь для ленточных матриц при $|\lambda_1/\lambda_2| \gg 1$. В случае близких по модулю собственных значений скорость сходимости степенного метода падает обратно пропорционально разности $1 - |\lambda_2/\lambda_1|$.

Однако даже если собственные значения заметно различаются, другие методы, как правило, демонстрируют более высокую эффективность, что ограничивает практическую применимость степенного метода.

Теорема. Пусть $n \times n$ -матрица A является матрицей простой структуры (т.е. диагонализуема) и у неё имеется простое доминирующее собственное значение. Если начальный вектор $x^{(0)}$ не лежит в линейной оболочке $\text{lin}\{x_2, \dots, x_n\}$ собственных векторов A , которые не являются доминирующими, то степенной метод сходится.

Доказательство. В качестве начального приближения выберем вектор

$$x^{(0)} = a_1 x_1 + a_2 x_2 + \dots,$$

где x_j — j -й собственный вектор матрицы: $Ax_j = \lambda_j x_j$. На каждой итерации будем умножать вектор на матрицу: $x^{(k)} = Ax^{(k-1)}$. На k -й итерации данного процесса будем иметь:

$$x^{(k)} = \sum_{j=1}^n a_j \lambda_j^k x_j = \lambda_1^k \left(a_1 x_1 + \sum_{j=2}^n a_j \left(\frac{\lambda_j}{\lambda_1} \right)^k x_j \right).$$

Коэффициенты при x_j ($j = 2, \dots, n$) содержат малые величины вида $(\lambda_j/\lambda_1)^k$, которые стремятся к нулю при $k \rightarrow \infty$. Следовательно, в пределе $k \rightarrow \infty$ направление вектора $x^{(k)}$ будет совпадать с направлением искомого вектора x_1 . Обрывая процесс на k -м шаге, получаем в результате искомый вектор x_1 с погрешностью $\mathcal{O}\left((\lambda_2/\lambda_1)^k\right)$.

Обратные степенные итерации

– степенной метод, примененный к матрице A^{-1} .

Если λ – с. з. для A , то $\frac{1}{\lambda}$ – с. з. для A^{-1} .

Псевдокод:

```
 $k \leftarrow 0;$   
выбираем вектор  $x^{(0)} \neq 0;$   
DO WHILE ( метод не сошёлся )  
    найти  $y^{(k+1)}$  из системы  $Ay^{(k+1)} = x^{(k)};$   
     $\tilde{\lambda} \leftarrow \langle x^{(k)}, y^{(k+1)} \rangle / \langle y^{(k+1)}, y^{(k+1)} \rangle;$   
     $x^{(k+1)} \leftarrow y^{(k+1)} / \|y^{(k+1)}\|_2;$   
     $k \leftarrow k + 1;$   
END DO
```

При реализации удобно выполнить LU - или QR -разложение матрицы A , чтобы при решении СЛАУ $Ay^{(k+1)} = x^{(k)}$ использовать решение треугольных СЛАУ.

Сдвиг матрицы – прибавление к ней скалярной матрицы, т.е. вместо A получаем $A + \eta I$, где $\eta \in \mathbb{C}$.

Если λ – с. з. A , то $(\lambda + \eta)$ – с. з. для $(A + \eta I)$.

Цель сдвигов – “улучшение” матрицы с тем, чтобы применять, ускорять различные методы решения проблемы с. з.

Пример.

Пусть у матрицы A два максимальных по модулю с. з.: (-2) и 2 .
у матрицы $(A + \eta I)$ будут с. з. (-1) и 3 . Теперь степенной метод применим.

Путем сдвигов можно сделать максимальной по модулю любую точку выпуклой оболочки спектра, т.е. добиться сход-ти к ней степенного метода.

Для вычисления внутренних точек спектра используют комбинацию сдвига и обратных степ. итераций. Необходимо хорошее начальное приближение.

Величину сдвига можно выбирать по априорной информации о сдвиге, либо также можно динамически его менять в процессе итераций. Скорость сходимости можно повысить (сделать квадратичной), если вместо фиксированного сдвига использовать приближение $\lambda^{(k)}$, найденное на предыдущей итерации.

Метод линеаризации (метод Ньютона)

Посмотрим на спектральную задачу $Au - \lambda u = 0$ как на систему из n уравнений на $n + 1$ неизвестную величину $u_1, \dots, u_n, \lambda$.

Система является нелинейной ввиду наличия члена λu .

Линеаризуем её подстановками $u = u^{(k)} + \delta u^{(k+1)}$, $\lambda = \lambda^{(k)} + \varepsilon^{(k+1)}$.

Здесь $u^{(k)}$ и $\lambda^{(k)}$ известные на k -й итерации приближения к искомым собственному вектору и собственному значению, $\delta u^{(k+1)}$ и $\varepsilon^{(k+1)}$ малые поправки к ним, которые будут найдены на $(k + 1)$ -й итерации.

Пренебрегая квадратичными членами $\varepsilon^{(k+1)}\delta u^{(k+1)}$, получаем линейную систему на $(k + 1)$ -й итерации:

$$(A - \lambda^{(k)}E) \delta u^{(k+1)} - \varepsilon^{(k+1)}u^{(k)} = - (A - \lambda^{(k)}E) u^{(k)}.$$

Поскольку собственный вектор определён с точностью до множителя, можно положить $\delta u_n^{(k+1)} = 0$, в результате чего получим систему из n уравнений на n неизвестных величин $\delta u_1^{(k+1)}, \dots, \delta u_{n-1}^{(k+1)}, \varepsilon^{(k+1)}$:

Метод линеаризации (метод Ньютона)

$$B^{(k)} \cdot \begin{pmatrix} \delta u_1^{(k+1)} \\ \vdots \\ \delta u_{n-1}^{(k+1)} \\ \varepsilon^{(k+1)} \end{pmatrix} = - \left(A - \lambda^{(k)} E \right) u^{(k)}, \quad \text{где } B^{(k)} \equiv$$
$$\equiv \begin{pmatrix} A_{11} - \lambda^{(k)} & A_{12} & \dots & A_{1,n-1} & -u_1^{(k)} \\ \vdots & \vdots & & \vdots & \vdots \\ A_{n-1,1} & A_{n-1,2} & \dots & A_{n-1,n-1} - \lambda^{(k)} & -u_{n-1}^{(k)} \\ A_{n,1} & A_{n,2} & \dots & A_{n,n-1} & -u_n^{(k)} \end{pmatrix}$$

Решение исходной спектральной задачи получается в результате итерационного процесса $u^{(k+1)} = u^{(k)} + \delta u^{(k+1)}$, $\lambda^{(k+1)} = \lambda^{(k)} + \varepsilon^{(k+1)}$.

Метод линеаризации (метод Ньютона)

Достоинства метода:

- Методы линеаризации и обратных итераций со сдвигом обеспечивают достаточно быструю сходимость (квадратичную вблизи корня) и особенно эффективны в случае ленточных матриц.
- Дополнительным преимуществом по сравнению со степенным методом является возможность нахождения любого собственного значения, для которого известно начальное приближение.

Недостатки метода

- Как и в случае трансцендентных уравнений, сходимость метода Ньютона не гарантируется: при неудачно выбранном начальном приближении итерационный процесс может быть расходящимся или сходиться не к искомому собственному значению, а к другому.

Метод Якоби для симметричной проблемы с. з.

Если A – симм. матрица, то существует ортог. матрица V ($V^T = V^{-1}$):

$$V^{-1}AV = \Lambda,$$

– диагональная матрица, составленная из с. з. матрицы A .

Идея метода – последовательное подавление недиагональных элементов матрицы путем преобразования подобия с ортогональной матрицей вращения $V(k, l, \theta)$. Данные преобразования не меняют спектр исходной матрицы.

$$\begin{array}{l} k\text{-ая строка} \\ l\text{-ая строка} \end{array} \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & \cos \theta & \cdots & -\sin \theta \\ & & 1 & & \\ & \vdots & & \ddots & \vdots \\ \sin \theta & \cdots & & 1 & \cos \theta \\ & & & & \ddots \\ & & & & & 1 \end{pmatrix}$$

Строится сходящаяся к Λ последовательность матриц:

$$A^{(0)} = A, A^{(1)}, A^{(2)}, \dots, A^{(k)}.$$

Метод Якоби для симметричной проблемы с. з.

Пусть найдена $A^{(k)}$. Как найти $A^{(k+1)}$?

Ищем максимальный по модулю элемент, лежащий выше главной диагонали, и запоминаем его индексы i_k, j_k :

$$i_k, j_k : \quad \left| a_{i_k j_k}^{(k)} \right| = \max_{i < j} \left| a_{ij}^{(k)} \right|$$

С помощью поворота в плоскости (i_k, j_k)

$$A^{(k+1)} = V_{(i_k j_k)}^T A^{(k)} V_{(i_k j_k)} \quad (1)$$

зануляем элемент с индексами (i_k, j_k) .

Как определить матрицу поворота?

Обозначим для краткости $c \equiv \cos \theta_k$, $s \equiv \sin \theta_k$.

Раскрывая поэлементно выражение (1), получим для элемента (i_k, j_k) матрицы $A^{(k+1)}$:

$$a_{i_k j_k}^{(k+1)} = (c^2 - s^2) a_{i_k j_k}^{(k)} + c \cdot s \cdot \left(a_{j_k j_k}^{(k)} - a_{i_k i_k}^{(k)} \right).$$

Метод Якоби для симметричной проблемы с. з.

Находя θ_k по правилу

$$\operatorname{tg}(2\theta_k) = \frac{2a_{i_k j_k}^{(k)}}{a_{i_k i_k}^{(k)} - a_{j_k j_k}^{(k)}}, \quad |\theta_k| \leq \frac{\pi}{4},$$

получим $a_{i_k j_k}^{(k+1)} = 0$.

Значения c и s находим по следующим формулам:

$$c = \sqrt{\frac{1}{2} \left(1 + \frac{|q|}{d} \right)}, \quad s = \operatorname{sign}(q a_{i_k j_k}) \sqrt{\frac{1}{2} \left(1 - \frac{|q|}{d} \right)},$$

где

$$q = a_{i_k i_k} - a_{j_k j_k}, \quad d = \sqrt{q^2 + 4a_{i_k j_k}^2}.$$

Метод Якоби для симметричной проблемы с. з.

В выражении (1) не вычисляем произведение матриц, так как большинство элементов матрицы вращения нулевые!

Большая часть коэфф-в матрицы $A^{(k)}$ остается без изменений:

$$a_{\alpha\beta}^{(k+1)} = a_{\alpha\beta}^{(k)} \quad \text{при} \quad \alpha \neq i_k, \alpha \neq j_k, \beta \neq i_k, \beta \neq j_k$$

Следующие две формулы определяют преобразование большинства коэффициентов в двух строках с индексами i_k и j_k :

$$a_{\alpha i_k}^{(k+1)} = a_{i_k \alpha}^{(k+1)} = c a_{\alpha i_k}^{(k)} + s a_{\alpha j_k}^{(k)}, \quad \text{при} \quad \alpha \neq i_k, \alpha \neq j_k$$

$$a_{\alpha j_k}^{(k+1)} = a_{j_k \alpha}^{(k+1)} = c a_{\alpha j_k}^{(k)} - s a_{\alpha i_k}^{(k)}, \quad \text{при} \quad \alpha \neq i_k, \alpha \neq j_k$$

Преобразование коэффициентов на пересечении строк и столбцов с номерами i_k, j_k

$$a_{i_k i_k}^{(k+1)} = c^2 a_{i_k i_k}^{(k)} + 2cs a_{i_k j_k}^{(k)} + s^2 a_{j_k j_k}^{(k)}$$

$$a_{j_k j_k}^{(k+1)} = s^2 a_{i_k i_k}^{(k)} - 2cs a_{i_k j_k}^{(k)} + c^2 a_{j_k j_k}^{(k)}$$

$$a_{i_k j_k}^{(k+1)} = a_{j_k i_k}^{(k+1)} = (c^2 - s^2) a_{i_k j_k}^{(k)} + cs \left(a_{j_k j_k}^{(k)} - a_{i_k i_k}^{(k)} \right).$$

Метод Якоби для симметричной проблемы с. з.

Вход Симметричная матрица A.
Выход Матрица, на диагонали которой стоят приближения к собственным значениям A.
Алгоритм DO WHILE (метод не сошёлся) выбрать ненулевой внедиагональный элемент a_{pq} в A ; обнулить a_{pq} и a_{qp} преобразованием подобия с матрицей вращения $G(p, q, \theta)$; END DO

Один из наиболее эффективных методов решения полной спектральной задачи. Был разработан в конце 1950-х годов независимо В. Н. Кублановской и Дж. Фрэнсисом

Псевдокод:

```
k ← 0;  
A(0) ← A;  
DO WHILE ( метод не сошёлся )  
    вычислить QR-разложение A(k) = Q(k) R(k);  
    A(k+1) ← R(k) Q(k);  
    k ← k + 1;  
END DO
```



Необходимо совершать матричные преобразования, не меняющие спектра исходной матрицы:

$$A^{(k+1)} = R^{(k)} Q^{(k)} = \left(Q^{(k)} \right)^{\top} \left(Q^{(k)} R^{(k)} \right) Q^{(k)} = \left(Q^{(k)} \right)^{\top} A^{(k)} Q^{(k)}$$

Результат (предельная матрица) – верхняя треугольная либо верхняя блочно-треугольная матрица (разложение Шура).

Исай Шур (29 декабря 1875 (10 января 1876), Могилёв — 10 января 1941, Тель-Авив) – русский, немецкий и израильский математик.



Размеры блоков зависят от типа с. з.:

- если алгоритм выполняется в вещественной (комплексной) арифметике и все с. з. матрицы вещественны (комплексны) и различны по модулю, то предельная матрица – верхняя треугольная
- если алгоритм выполняется в вещественной (комплексной) арифметике и некоторое с. з. матрицы вещественно (комплексно) и имеет кратность p , то в предельной матрице ему соответствует диагональный блок размера $p \times p$
- если алгоритм выполняется для вещественной матрицы в вещественной арифметике, то простым комплексно-сопряженным с. з. (они имеют равные модули) отвечают диагональные 2×2 -блоки в предельной матрице (пример в книге Шарого, с. 537)
- если некоторое комплексное с. з. вещественной матрицы имеет кратность p , так что ему соответствует еще такое же комплексно-сопряженное с. з. кратности p , то при выполнении QR-алгоритма в вещественной арифметике предельная матрица получит диагональный блок размера $2p \times 2p$

Для улучшения сходимости метода могут применяться сдвиги (с. 539):

```
 $k \leftarrow 0;$   
 $A^{(0)} \leftarrow A;$   
DO WHILE ( метод не сошёлся )  
    выбрать сдвиг  $\vartheta_k$ , приближённо равный  
        собственному значению  $A$  ;  
    вычислить QR-разложение сдвинутой  
        матрицы  $A^{(k)} - \vartheta_k I = Q^{(k)} R^{(k)}$  ;  
     $A^{(k+1)} \leftarrow R^{(k)} Q^{(k)} + \vartheta_k I$  ;  
     $k \leftarrow k + 1$  ;  
END DO
```

Получаемые матрицы, как и в обычном QR-алгоритме, ортогонально подобны:

$$\begin{aligned} A^{(k+1)} &= R^{(k)} Q^{(k)} + \vartheta_k I = \left(Q^{(k)} \right)^\top Q^{(k)} R^{(k)} Q^{(k)} + \vartheta_k \left(Q^{(k)} \right)^\top Q^{(k)} \\ &= \left(Q^{(k)} \right)^\top \left(Q^{(k)} R^{(k)} + \vartheta_k I \right) Q^{(k)} = \left(Q^{(k)} \right)^\top A^{(k)} Q^{(k)} \end{aligned}$$

Также применяется предварительное приведение к хессенберговой форме (матрица $H = (h_{ij})$ называется **верхней почти треугольной** или **верхней хессенберговой**, если $h_{ij} = 0$ при $i > j + 1$):

$$H = \begin{pmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2n} \\ 0 & h_{32} & h_{33} & \cdots & h_{3n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{nn-1} & h_{nn} \end{pmatrix}$$

Матрица, являющаяся одновременно и верхней, и нижней матрицами Хессенберга, трёхдиагональна.

Сложность обычного QR -алгоритма – $\mathcal{O}(n^4)$, т. к. на каждом шаге выполняется QR -разложение за $\mathcal{O}(n^3)$.

Приведение к хессенберговой форме (методами вращения/отражения) – $\mathcal{O}(n^3)$, а одна итерация QR -алгоритма с такой матрицей – $\mathcal{O}(n^2)$, т. е. всего $\mathcal{O}(n^3)$ операций!

QR-алгоритм. Нахождение корней полиномов

Библиотека `eiscor` позволяет находить спектр матриц в хессенберговой форме: <https://github.com/eiscor/eiscor>

С помощью библиотеки `eiscor` можно находить корни полиномов $P_n(x)$ за $\mathcal{O}(n^2)$ операций.

Для нахождения корней полинома необходимо построить *сопровождающую матрицу* (*companion matrix*)

https://en.wikipedia.org/wiki/Companion_matrix

и найти её спектр. Сопровождающая матрица для полинома

$$p(t) = c_0 + c_1 t + \dots + c_{n-1} t^{n-1} + t^n$$

строится следующим образом:

$$C(p) = \begin{bmatrix} 0 & 0 & \dots & 0 & -c_0 \\ 1 & 0 & \dots & 0 & -c_1 \\ 0 & 1 & \dots & 0 & -c_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & -c_{n-1} \end{bmatrix}$$

Найти уровень энергии и волновую функцию $\psi(x)$ основного состояния в потенциальной яме $U(x)$, решая конечномерный аналог спектральной задачи для одномерного стационарного уравнения Шрёдингера

$$(\hat{H} - E) \psi(x, t) = \left(-\frac{1}{2} \frac{\partial^2}{\partial x^2} + U(x) - E \right) \psi(x, t) = 0, \quad |\psi(x)| \rightarrow 0 \text{ при } |x| \rightarrow \infty.$$

Для поиска наименьшего собственного значения E_0 дискретной версии гамильтониана $\hat{H}$ (соответствующей ему трёхдиагональной матрицы) использовать метод обратных итераций. Проверить работу программы, сравнив с точным решением для $U(x) = \frac{1}{2}x^2$.

Сингулярные числа зависят от элементов матрицы существенно более плавным образом, чем собственные числа (теорема Вейля и теорема Виландта-Хоффмана).

Собственные числа могут изменяться в зависимости от элементов матрицы сколь угодно быстро, тогда как скорость изменения сингулярных чисел соразмерна норме матрицы.

Простейшие методы нахождения сингулярных чисел матриц основаны на том, что они являются собственными числами матриц $A^T A$ и AA^T ($A^* A$ и AA^* в комплексном случае).

Пример на Python

Калиткин Н. Н.: Численные методы

Смирнов С. В.: ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ, Часть II

<http://www.phys.nsu.ru/smirnov/ovf2-2017-09-18.pdf>

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

<http://www.ict.nsc.ru/matmod/files/textbooks/SharyNuMeth.pdf>

Вычислительные методы линейной алгебры. Стационарные итерационные методы решения систем линейных алгебраических уравнений

Чеховской Игорь Сергеевич

Задачи вычислительной линейной алгебры

1. решение систем линейных алгебраических уравнений (СЛАУ)
2. вычисление определителей матриц
3. нахождение обратной матрицы
4. нахождение собственных значений и собственных векторов матриц
5. нахождение сингулярных чисел и сингулярных векторов матриц
6. линейная задача наименьших квадратов: найти вектор x , минимизирующий $\langle Ax - b, Ax - b \rangle$ для заданных $m \times n$ -матрицы A и m -вектора b (решение переопределенных СЛАУ).

Стационарные:

- метод Рундсона
- Метод Якоби
- Метод Гаусса-Зейделя
- Метод релаксации

Нестационарные:

- Наискорейший градиентный спуск
- Метод сопряжённых градиентов
- Метод бисопряжённых градиентов
- Стабилизированный метод бисопряжённых градиентов

История:

- Итерационные методы известны с древности (вычисление квадратного корня (Вавилон, метод Герона), метод Ньютона).
- XIX век: работы Ж. Лиувилля и К. Г. Неймана о решении интегральных уравнений.

Особенности:

- **Самоисправление:** ошибки вычислений устраняются в процессе сходимости.
- Прямые методы такой корректировки не обеспечивают.
- Более простая реализация и возможность использовать матричную структуру.

Стационарные итерационные методы (общая схема)

Итерационные методы — это процедуры построения последовательности приближений $\{x^{(k)}\}_{k=0}^{\infty}$ к решению x^* :

$$x^* = \lim_{k \rightarrow \infty} x^{(k)}.$$

Общая схема:

$$x^{(k+1)} \leftarrow T_k(x^{(0)}, x^{(1)}, \dots, x^{(k)}), \quad k = \nu, \nu + 1, \nu + 2, \dots$$

p-шаговый метод: $x^{(k+1)}$ зависит от p предыдущих приближений:

$$x^{(k+1)} \leftarrow T_k(x^{(k)}, x^{(k-1)}, \dots, x^{(k-p+1)})$$

Стационарный одношаговый метод (метод простой итерации):

$$x^{(k+1)} = T(x^{(k)}), \quad k = 0, 1, 2, \dots$$

Оператор перехода $T_k = T$, т. е. не зависит от номера шага k .

Линейный p-шаговый метод:

$$x^{(k+1)} = C^{(k)}x^{(k)} + C^{(k,k-1)}x^{(k-1)} + \dots + C^{(k,k-p+1)}x^{(k-p+1)} + d^{(k)}$$

$C^{(k)}$ — матрицы перехода, $d^{(k)}$ — свободный член.

Сходимость стационарных одношаговых итерационных методов

СЛАУ $Ax = b$ приводится к рекуррентному виду

$$x = Cx + d$$

для возможности организации одношагового итерационного процесса.

Случай со СЛАУ аналогичен случаю одного уравнения $f(x) = 0$, но при этом $f(x) = Ax - b$.

Предложение (достаточное условие сходимости). Если $\|C\| < 1$ в какой-нибудь матричной норме, то стационарный одношаговый итерационный процесс

$$x^{(k+1)} \leftarrow Cx^{(k)} + d, \quad k = 0, 1, 2, \dots$$

сходится при любом начальном приближении $x^{(0)}$.

Доказательство основывается на оценке погрешности через рекурсивное применение формулы метода:

$$\|x^{(k)} - x^*\| \leq \|C\|^k \cdot \|x^{(0)} - x^*\|$$

Сходимость обеспечивается, если $\|C\| < 1$.

Определение. **Спектральным радиусом** квадратной матрицы A называется наибольшее по модулю собственное значение этой матрицы:

$$\rho(A) = \max\{|\lambda_k| : \lambda_k \in \sigma(A)\},$$

где $\sigma(A)$ — множество собственных значений матрицы A (её спектр).

Теорема (критерий сходимости). Пусть система $x = Cx + d$ имеет единственное решение. Стационарный одношаговый итерационный процесс

$$x^{(k+1)} \leftarrow Cx^{(k)} + d, \quad k = 0, 1, 2, \dots$$

сходится при любом начальном приближении $x^{(0)} \Leftrightarrow \rho(C) < 1$.

В итоге:

- Условие $\|C\| < 1$ в фиксированной норме — **только достаточное**.
- Если $\rho(C) < 1$, то найдётся такая норма, в которой $\|C\| < 1$.
- Лучше исследовать сходимость в стандартных нормах ($\|\cdot\|_2$, $\|\cdot\|_\infty$ и др.).
- При несимметричных C нормы, близкие к спектральному радиусу, могут быть сложными или экзотичными.
- Спектральный радиус $\rho(C)$ — **главный критерий сходимости** даже без привязки к конкретной норме.

Замечание. При решении СЛАУ итерационным методом выбор начального приближения $x^{(0)}$ может опираться на:

- содержательные соображения (физика задачи),
- известные приближения,
- либо быть формальным (например, $x^{(0)} = 0$).

Если $\|C\| \ll 1$, то

$$x^{(0)} = d$$

— хороший выбор:

- $x^* = Cx^* + d \approx d$,
- первая итерация даёт $x^{(1)} = Cd + d$.

Экономия вычислений за счёт разумного начального приближения.

Рассматривается переход от системы $Ax = b$ к эквивалентной в рекуррентном виде:

$$x = Cx + d$$

Наивный способ: прибавить x к обеим частям:

$$x + Ax = x + b \Rightarrow x = (I - A)x + b$$

Но спектральный радиус $C = I - A$ может быть ≥ 1 , и сходимость не гарантирована.

Определение. Предобуславливанием системы $Ax = b$ называется умножение слева обеих её частей на матрицу Λ :

$$(\Lambda A)x = \Lambda b$$

Матрица Λ называется **предобуславливающей**.

Цель: улучшить спектральные свойства A , чтобы сделать $\rho(C) < 1$.

Пример. Если $\Lambda = A^{-1}$ (или приближение к обратной), то:

$$(\Lambda A)x = A^{-1}Ax = x = A^{-1}b$$

то есть $x = A^{-1}b$ — точное решение, если $\Lambda = A^{-1}$.

Другой способ: **расщепление** матрицы A :

$$A = G - H$$

где G — неособенная. Тогда система $Ax = b$ переписывается как:

$$Gx = Hx + b \quad \Rightarrow \quad x^{(k+1)} = G^{-1}Hx^{(k)} + G^{-1}b$$

Неявный метод:

$$y = Hx^{(k)} + b,$$

$$x^{(k+1)} = \text{решение системы } Gx = y$$

Важно: не каждое расщепление $A = G - H$ эффективно!

- Матрица G должна **обращаться просто**, иначе итерационный процесс усложнит задачу.
- Норма $\|G^{-1}\|$ должна быть **достаточно малой**, чтобы:

$$\|G^{-1}H\| \leq \|G^{-1}\| \cdot \|H\| < 1$$

иначе возможна расходимость: $\rho(G^{-1}H) > 1$.

Популярный приём: выбрать G и H так, чтобы их элементы были взаимодополнительными:

$$g_{ij}h_{ij} = 0 \quad \text{для всех } i, j$$

(т.е. чтобы в одних и тех же позициях не было одновременно ненулевых значений).

Тогда ненулевые элементы G и $-H$ совпадают с ненулевыми элементами исходной матрицы A .

Примеры матриц G , которые легко обратить:

- диагональные;
- треугольные;
- трёхдиагональные.

Скалярный предобуславливатель: метод Ричардсона

Рассматривается простейшее предобуславливание системы $Ax = b$ с помощью скалярной матрицы:

$$\Lambda = \tau I, \quad \tau \in \mathbb{R}, \quad \tau \neq 0$$

Тогда итерационный процесс принимает вид:

$$x^{(k+1)} = (I - \tau A)x^{(k)} + \tau b$$

– метод Ричардсона (предложен в 1910 году).

При каком τ этот метод сходится?

Пусть λ_i — собственные значения матрицы A , тогда собственные значения оператора перехода:

$$\lambda'_i = 1 - \tau \lambda_i$$

Метод сходится тогда и только тогда, когда:

$$\rho(I - \tau A) = \max_i |1 - \tau \lambda_i| < 1$$

Если A — произвольная матрица, то достижение этого условия может быть невозможно.

Скалярный предобуславливатель: оптимальный параметр τ

Рассмотрим частный случай: A — симм-я положительно определённая.

Пусть $\lambda_i \in [\mu, M]$, где $0 < \mu \leq \lambda_i \leq M$ — собственные значения матрицы A .

Тогда:

$$\|I - \tau A\|_2 = \max_{\lambda \in [\mu, M]} |1 - \tau \lambda| =: g(\tau)$$

Оптимум достигается, когда значения на концах интервала равны:

$$1 - \tau_{\text{опт}} \mu = -(1 - \tau_{\text{опт}} M)$$

$$\Rightarrow \tau_{\text{опт}} = \frac{2}{M + \mu}$$

График функции $1 - \tau \lambda$:

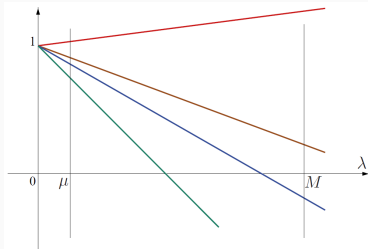
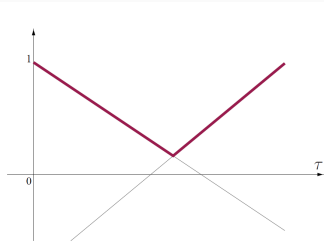


График функции $g(\tau)$:



Скалярный предобуславливатель: связь с обусловленностью и скоростью сходимости

При $\tau = \tau_{\text{опт}}$ норма оператора перехода:

$$\|I - \tau_{\text{опт}}A\|_2 \leq 1 - \tau_{\text{опт}}\mu = 1 - \frac{2}{M + \mu} \cdot \mu = \frac{M - \mu}{M + \mu}$$

Эта величина определяет скорость сходимости итераций:

$$\|x^{(k)} - x^*\|_2 \leq \left(\frac{M - \mu}{M + \mu} \right)^k \cdot \|x^{(0)} - x^*\|_2$$

Сравнение с обусловленностью:

$$\frac{M - \mu}{M + \mu} \leq \frac{\text{cond}_2(A) - 1}{\text{cond}_2(A) + 1}$$

Чем хуже обусловленность A , тем медленнее сходимость метода Рундсона.

На практике значение нижней границы спектра μ может быть неизвестно. Тем не менее, если известно $M \approx \lambda_{\max}(A)$, а $\mu \geq 0$, можно применить приближенную формулу:

$$\tau_{\text{прибл}} = \frac{2}{M}$$

Даже в этом случае:

- Метод Рундсона всё ещё может сходиться.
- Значение $\|I - \tau_{\text{прибл}} A\|_2$ может быть больше 1, если M выбрано слишком точно.
- Поэтому имеет смысл слегка завысить M вручную.

Оптимальности при этом не будет, но **сходимость может быть достигнута**.

Предложен К. Г. Яко́би в XIX веке.

Пусть в СЛАУ $Ax = b$ диагональные элементы $n \times n$ -матрицы $A = (a_{ij})$ отличны от нуля: $a_{ii} \neq 0$, $i = 1, \dots, n$.

Перепишем уравнения по компонентам:

$$\sum_{j=1}^n a_{ij}x_j = b_i, \quad i = 1, \dots, n.$$

Выразим x_i :

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j \right), \quad i = 1, \dots, n.$$

Метод Якоби: векторная запись и псевдокод

Введем отображение $T(x) = (T_1(x), \dots, T_n(x))^T$:

$$T_i(x) = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right).$$

Тогда итерационный процесс:

$$x^{(k+1)} \leftarrow T(x^{(k)}), \quad k = 0, 1, 2, \dots$$

Псевдокод (метод Якоби или метод одновременных смещений):

```
k ← 0;  
выбираем начальное приближение  $x^{(0)}$ ;  
DO WHILE ( метод не сошёлся )  
    DO FOR i = 1 TO n  
        
$$x_i^{(k+1)} \leftarrow \frac{1}{a_{ii}} \left( b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$
  
    END DO  
    k ← k + 1;  
END DO
```

Метод Якоби: расщепление матрицы и матричная форма

Пусть $A = \tilde{L} + D + \tilde{U}$, где

$$\tilde{L} = \begin{pmatrix} 0 & 0 & \cdots & 0 & 0 \\ a_{21} & 0 & \ddots & \vdots & \vdots \\ a_{31} & a_{32} & \ddots & 0 & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ a_{n1} & a_{n2} & \cdots & a_{n,n-1} & 0 \end{pmatrix}$$

— строго нижняя
треугольная матрица,

$$D = \text{diag}\{a_{11}, a_{22}, \dots, a_{nn}\}$$

— диагональ матрицы A ,

$$\tilde{U} = \begin{pmatrix} 0 & a_{12} & \cdots & a_{1,n-1} & a_{1n} \\ 0 & 0 & \ddots & a_{2,n-1} & a_{2n} \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \cdots & 0 & 0 & a_{n-1,n} \\ 0 & \cdots & \cdots & 0 & 0 \end{pmatrix}$$

— строго верхняя
треугольная матрица.

Тогда метод Якоби может быть представлен как метод, основанный на таком расщеплении матрицы системы $A = G - H$, что

$$G = D, \quad H = -(\tilde{L} + \tilde{U}).$$

Соответственно, в матричном виде метод Якоби записывается как

$$x^{(k+1)} \leftarrow -D^{-1}(\tilde{L} + \tilde{U})x^{(k)} + D^{-1}b, \quad k = 0, 1, 2, \dots$$

Метод Якоби: достаточное условие сходимости

По теореме сходимости:

$$\rho(D^{-1}(\tilde{L} + \tilde{U})) < 1 \Leftrightarrow \text{метод сходится при любом } x^{(0)}.$$

Матрица перехода:

$$D^{-1}(\tilde{L} + \tilde{U}) = \begin{pmatrix} 0 & a_{12}/a_{11} & \dots & a_{1n}/a_{11} \\ a_{21}/a_{22} & 0 & \dots & a_{2n}/a_{22} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1}/a_{nn} & \dots & a_{n,n-1}/a_{nn} & 0 \end{pmatrix}$$

Нахождение её спектрального радиуса является задачей, сравнимой по сложности с выполнением самого итерационного процесса, и потому применять его для исследования сходимости метода Якоби непрактично.

Теорема. Если в $A = (a_{ij})$ выполнено условие диагонального преобладания

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, \dots, n,$$

то метод Яко́би сходится при любом $x^{(0)}$.

Равносильная запись через норму:

$$\left\| D^{-1}(\tilde{L} + \tilde{U}) \right\|_{\infty} < 1.$$

Особенности:

- Используется редко из-за невысокой эффективности.
- Удобен для параллельных вычислений.

Следующая задача

$Ax = b(x)$, $b(x)$ – нелинейная вектор-функция от x ,

не может быть решена обычными методами решения СЛАУ.

Но на основе метода Якоби может быть построен соответствующий процесс:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i(x^{(k)}) - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Если $b(x)$ меняется достаточно медленно, то сходимость сохраняется:

$$\rho(D^{-1}(\tilde{L} + \tilde{U} + b'(x))) < 1,$$

где $b'(x)$ – матрица Якоби вектор-функции $b(x)$.

Для нелинейной системы

$$\begin{cases} F_1(x_1, \dots, x_n) = 0 \\ F_2(x_1, \dots, x_n) = 0 \\ \vdots \\ F_n(x_1, \dots, x_n) = 0 \end{cases}$$

метод Яко́би принимает форму:

- Задаём начальное приближение $x^{(0)}$.
- На k -м шаге для каждого i решаем:

$$F_i(x_1^{(k)}, \dots, x_{i-1}^{(k)}, \tilde{x}_i, x_{i+1}^{(k)}, \dots, x_n^{(k)}) = 0.$$

- Затем кладем $x_i^{(k+1)} \leftarrow \tilde{x}_i$.

Метод Гаусса–Зейделя: основная идея

Итерационный метод Гаусса–Зейделя отличается от метода Якоби тем, что компоненты $x_i^{(k+1)}$ вычисляются последовательно, и новые значения сразу используются:

$$x_i^{(k+1)} \leftarrow \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n.$$

- Вычисления идут по i от 1 до n ; значения $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ уже найдены.
- Более свежая информация вовлекается в расчёт: улучшение скорости сходимости.

Псевдокод:

```
 $k \leftarrow 0;$   
выбираем начальное приближение  $x^{(0)}$ ;  
DO WHILE ( метод не сошёлся )  
  DO FOR  $i = 1$  TO  $n$   
    
$$x_i^{(k+1)} \leftarrow \frac{1}{a_{ii}} \left( b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$
  
  END DO  
   $k \leftarrow k + 1;$   
END DO
```

- Метод предложен К.Ф. Гауссом и Ф.Л. Зейделем в 1874 г.
- Иногда называют методом *последовательных смещений*.

Вводим разложение:

$$A = D + \tilde{L} + \tilde{U},$$

где:

- D — диагональная часть,
- $\tilde{L}$ — строго нижняя треугольная часть,
- $\tilde{U}$ — строго верхняя треугольная часть.

Тогда метод имеет вид:

$$(D + \tilde{L})x^{(k+1)} = -\tilde{U}x^{(k)} + b,$$

или:

$$x^{(k+1)} = -(D + \tilde{L})^{-1}\tilde{U}x^{(k)} + (D + \tilde{L})^{-1}b.$$

Теорема. Метод Гаусса–Зейделя сходится при любом начальном приближении тогда и только тогда, когда:

$$\rho\left((D + \tilde{L})^{-1}\tilde{U}\right) < 1.$$

- Аналогично условию для метода Якоби.
- На практике: оценивают спектральный радиус через нормы.

Метод Гаусса–Зейделя: достаточное условие сходимости

Теорема. Если в системе $Ax = b$ матрица A обладает **диагональным преобладанием**, то метод Гаусса–Зейделя сходится при любом начальном приближении.

Условие диагонального преобладания:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, 2, \dots, n.$$

- Существует $\chi \in (0, 1)$, что:

$$\sum_{j \neq i} \left| \frac{a_{ij}}{a_{ii}} \right| \leq \chi < 1.$$

- Тогда: $\|z^{(k+1)}\|_{\infty} \leq \chi \|z^{(k)}\|_{\infty} \Rightarrow$ метод сходится.

Метод Гаусса–Зейделя: симметричная положительно определённая матрица

Теорема. Если A — симметричная положительно определённая, то метод Гаусса–Зейделя сходится при любом начальном приближении.

- Следует из теоремы Остроградского–Райха (или Самарского).
- Спектральный радиус $\rho((D + \tilde{L})^{-1}\tilde{U}) < 1$ в этом случае.

- Метод Гаусса–Зейделя — модификация метода Якоби.
- Обычно сходится быстрее (новые значения используются сразу).
- Но есть примеры, когда метод Якоби сходится, а Гаусса–Зейделя — нет (и наоборот).
- Практическое применение аналогично методу Якоби (например, для СЛАУ с диагональным преобладанием).
- Особенно эффективен при решении задач с большим числом переменных.

Метод релаксации: идея и принцип

Принцип релаксации — это подход, при котором на каждом шаге итерационного процесса уменьшается некоторая величина, характеризующая погрешность приближения $x^{(k)}$ к решению.

Поскольку точное решение x^* неизвестно, вместо прямой оценки $x^{(k)} - x^*$ оценивают **невязку**:

$$r^{(k)} = Ax^{(k)} - b.$$

Цель — уменьшить норму или абсолютные значения компонент $r^{(k)}$.

Методы Якоби и Гаусса–Зейделя реализуют **локальную релаксацию**, т.е. коррекцию каждой компоненты с использованием уже полученной информации.

Полная релаксация — учитывает влияние компоненты на всю систему.

Неполная релаксация — обновление отдельной компоненты независимо.

Метод релаксации: идея и принцип

Для улучшения сходимости: вводится **параметр релаксации** ω .

Пусть $x_i^{(k)}$ — i -я компонента на шаге k , $x_i^{(k+1)}$ — новое значение.

Обновление с релаксацией:

$$\omega x_i^{(k+1)} + (1 - \omega)x_i^{(k)}.$$

Типичная реализация — на основе метода Гаусса–Зейделя:

$$x_i^{(k+1)} \leftarrow (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right).$$

Значения ω :

- $\omega < 1$ — *нижняя релаксация*,
- $\omega = 1$ — метод Гаусса–Зейделя,
- $\omega > 1$ — *верхняя релаксация*, *SOR – Successive OverRelaxation* (последовательная перерелаксация) (обычно это лучший выбор).

На практике популярен так называемый симметричный метод релаксации (SSOR – Symmetric Successive Overrelaxation).

Метод релаксации: матричная форма

Перепишем формулу метода:

$$a_{ii}x_i^{(k+1)} + \omega \sum_{j < i} a_{ij}x_j^{(k+1)} = (1 - \omega)a_{ii}x_i^{(k)} - \omega \sum_{j > i} a_{ij}x_j^{(k)} + \omega b_i.$$

Обозначим:

$$A = D + \tilde{L} + \tilde{U}, \quad G_\omega = D + \omega\tilde{L}, \quad H_\omega = (1 - \omega)D - \omega\tilde{U}.$$

Итерационный процесс:

$$(D + \omega\tilde{L})x^{(k+1)} = ((1 - \omega)D - \omega\tilde{U})x^{(k)} + \omega b,$$

или

$$x^{(k+1)} = G_\omega^{-1}H_\omega x^{(k)} + G_\omega^{-1}\omega b.$$

Необходимое и достаточное условие сходимости:

$$\rho(G_\omega^{-1}H_\omega) < 1.$$

Пример

$$\begin{pmatrix} 6 & 2 & 2 \\ 2 & 3 & 4 \\ 2 & 4 & 8 \end{pmatrix} x = \begin{pmatrix} 6 \\ 3 \\ 6 \end{pmatrix}, \quad x^* = (1, -1, 1)^\top.$$

- Метод Ричардсона ($\tau = 0.1638$): 148 шагов до $\|r^{(k)}\|_1 < 10^{-6}$.
- Метод Якоби: расходится.
- Метод Гаусса–Зейделя: 42 шага.
- Метод верхней релаксации ($\omega = 1.288$): 16 шагов.

Метод релаксации: лемма Кэхэна

Лемма Кэхэна. Пусть рассматривается метод релаксации с параметром ω , так что матрицей оператора перехода является

$$C_\omega = (D + \omega \tilde{L})^{-1} ((1 - \omega)D - \omega \tilde{U}).$$

Тогда $\rho(C_\omega) \geq |\omega - 1|$, и, как следствие, для сходимости метода релаксации из любого начального приближения необходимо выполнение неравенства

$$0 < \omega < 2.$$

Лемма даёт лишь **необходимое** условие сходимости метода релаксации:

Пример

$$\begin{pmatrix} 2 & 1 & -1 \\ 1 & -5 & 4 \\ 3 & 2 & 6 \end{pmatrix} x = \begin{pmatrix} 0 \\ 10 \\ 7 \end{pmatrix}, \quad x^* = (1, -1, 1)^\top.$$

- $\omega = 0.8$: 15 итераций до 10^{-6} .
- $\omega \gtrsim 1.0356$: расходимость.

Теорема Островского–Райха (критерий сходимости). Пусть A — вещественная симметричная матрица с положительными диагональными элементами и $\omega \in (0, 2)$. Тогда метод релаксации с ω сходится к решению СЛАУ из любого начального приближения тогда и только тогда, когда A положительно определена.

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

Вычислительные методы линейной алгебры. Нестационарные итерационные методы решения систем линейных алгебраических уравнений

Чеховской Игорь Сергеевич

Стационарные:

- метод Рундсона
- Метод Якоби
- Метод Гаусса-Зейделя
- Метод релаксации

Нестационарные:

- Наискорейший градиентный спуск
- Метод сопряжённых градиентов
- Метод бисопряжённых градиентов
- Стабилизированный метод бисопряжённых градиентов

Нестационарные итерационные методы: обобщение процесса Ричардсона

Рассмотрим обобщение итерационного процесса Ричардсона с переменным параметром τ_k :

$$x^{(k+1)} \leftarrow x^{(k)} - \tau_k(Ax^{(k)} - b), \quad k = 0, 1, 2, \dots$$

Матричное обобщение:

$$x^{(k+1)} \leftarrow x^{(k)} - \Lambda_k(Ax^{(k)} - b), \quad k = 0, 1, 2, \dots,$$

где $\{\Lambda_k\}_{k=0}^{\infty}$ — некоторая последовательность матриц. Условие сходимости этого метода зависит от выбора Λ_k и начального приближения $x^{(0)}$.

Полуитерационные методы Чебышёва (Chebyshev semi-iterative methods) позволяют ускорить метод Рундсона и обеспечивают оптимальный выбор параметров τ_k :

$$x^{(k+1)} \leftarrow x^{(k)} - \tau_k(Ax^{(k)} - b),$$

где τ_k выбираются исходя из полинома Чебышёва k -й степени и зависят от оценки спектрального интервала $[\mu, M]$ симм. положит. опред. матрицы A :

- при малом спектральном интервале $[M/\mu \approx 1]$ метод быстро сходится.
- при высокой обусловленности A (большое M/μ) требуется больше итераций.

Метод верхней релаксации (Successive Overrelaxation, SOR) применяется к решению системы $Ax = b$ и использует параметр релаксации ω_k , который может изменяться на каждом шаге итерации.

Итерационная формула имеет вид:

$$x_i^{(k+1)} \leftarrow (1 - \omega_k)x_i^{(k)} + \frac{\omega_k}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right).$$

- Параметр ω_k управляет темпом продвижения по направлению невязки.
- Его изменение позволяет ускорить сходимость или стабилизировать процесс.
- Последовательность $\{\omega_k\}$ может быть задана априори или адаптивно рассчитываться по ходу итераций.
- Метод применяется для разреженных и симметричных матриц, а также в схемах численного решения дифференциальных уравнений.

Другой путь построения нестационарных итерационных методов — использование вариационных принципов.

Идея: решить систему $Ax = b$ можно как задачу оптимизации — найти минимум нормы невязки:

$$Ax = b \Leftrightarrow \min \|Ax - b\|.$$

Наиболее естественно использовать евклидову норму, что эквивалентно минимизации квадрата нормы:

$$\min \|Ax - b\|^2 = \min (Ax - b, Ax - b).$$

Это линейная задача наименьших квадратов.

Теорема. Вектор $x^* \in \mathbb{R}^n$ является решением СЛАУ $Ax = b$ с симм. полож. опр. матр. $A \Leftrightarrow$ он доставляет минимум функционалу энергии $\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$. Иными словами:

$$\text{решение } Ax = b \quad \Leftrightarrow \quad \text{нахождение } \min \Psi(x).$$

Доказательство. Если A — симметричная положительно-определённая матрица, то решение x^* системы $Ax = b$ существует и единственно.

Введём энергетическую норму:

$$\|x\|_A = \sqrt{(Ax, x)}.$$

Из единственности x^* следует: x — решение $Ax = b \Leftrightarrow x - x^* = 0 \Leftrightarrow \|x - x^*\|_A = 0$.

То есть:

$$x \text{ есть решение } Ax = b \quad \Leftrightarrow \quad \frac{1}{2}\|x - x^*\|_A^2 = 0.$$

Преобразуем выражение:

$$\begin{aligned}\frac{1}{2}\|x - x^*\|_A^2 &= \frac{1}{2}(A(x - x^*), x - x^*) = \\&= \frac{1}{2}(Ax, x) - \frac{1}{2}(Ax^*, x) - \frac{1}{2}(Ax, x^*) + \frac{1}{2}(Ax^*, x^*) = \\&= \frac{1}{2}(Ax, x) - (b, x) + \frac{1}{2}\|x^*\|_A^2 = \Psi(x) + \frac{1}{2}\|x^*\|_A^2.\end{aligned}$$

Следовательно, x — точка минимума $\Psi(x)$ тогда и только тогда, когда x — решение $Ax = b$. ■

Следствие. $\Psi(x)$ достигает минимума в точке x^* и отличается от $\frac{1}{2}\|x - x^*\|_A^2$ лишь на константу. Поэтому:

$$\Psi(x) = \frac{1}{2}\|x - x^*\|_A^2 + \text{const.}$$

Этот факт будет неоднократно использоваться далее.

Поскольку A – симм. матрица, то ортогональным преобразованием подобия она может быть приведена к диагональной матрице:

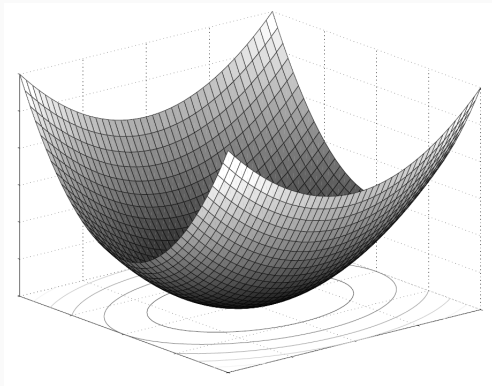
$$A = Q^T D Q,$$

где Q ортогональна, $D = \text{diag}\{\lambda_1, \lambda_2, \dots, \lambda_n\}$, а λ_i — с. з. матрицы A , причём все $\lambda_i > 0$ в силу положительной определённости A . Подставляя это представление в выражение для функционала энергии $\Psi(x)$, получим

$$\begin{aligned}\Psi(x) &= \frac{1}{2}(Q^T D Q x, x) - (b, x) = \frac{1}{2}(D(Qx), Qx) - (Qb, Qx) \\ &= \frac{1}{2}(Dy, y) - (Qb, y) = \frac{1}{2} \sum_{i=1}^n \lambda_i y_i^2 - \sum_{i=1}^n (Qb)_i y_i,\end{aligned}$$

где обозначено $y = Qx$.

График функционала энергии: параболоид с линиями уровня — эллипсами, центр которых — точное решение.

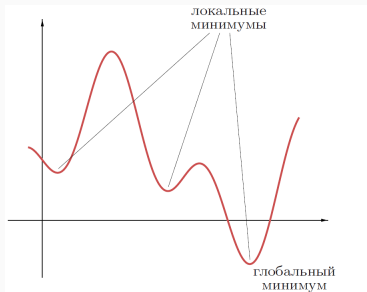


Чем больше обусловленность матрицы A , тем сильнее сплющены эллипсоиды уровня, так что для плохообусловленных СЛАУ решение находится на дне длинного и узкого "оврага".

Метод спуска: постановка задачи

Рассмотрим задачу минимизации функции $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$f(x) \geq f(x^*) = \min_{x \in \mathbb{R}^n} f(x) \quad \text{для всех } x \in \mathbb{R}^n.$$



Глобальные и локальные минимумы функции.

- x^* — точка глобального минимума.
- Функция f называется *целевой функцией*.
- Типичные алгоритмы — построение последовательности $\{x^{(k)}\}$:

$$\lim_{k \rightarrow \infty} f(x^{(k)}) = \min_{x \in \mathbb{R}^n} f(x).$$

Метод спуска: шаг алгоритма

Пусть известно приближение $x^{(k)}$ к точке минимума.

- Выбираем направление спуска s , в котором f убывает.
- Затем выбираем шаг τ и делаем переход:

$$x^{(k+1)} \leftarrow x^{(k)} + \tau s.$$

- Требование:

$$f(x^{(k+1)}) < f(x^{(k)}).$$

Выбор шага τ вдоль уже выбранного направления s называют **line search**.

Т. к. при фиксированном направлении s неизвестной остается только одна величина τ , такой выбор шага также называют **одномерным поиском**.

Выбор s и τ критически влияет на наличие сходимости и её скорость.

Если целевая функция имеет более одного локального экстремума, то метод спуска может сходиться к какому-нибудь одному из них, который не обязательно является глобальным.

Тогда предпринимают многократный запуск метода спуска из различных начальных точек ("мультистарт"), применяют другие модификации, которые помогают преодолеть локальный характер метода спуска.

Метод спуска: идея наискорейшего спуска

Но для матрицы $A = A^T > 0$ минимум функционала энергии $\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$ только один — глобальный.

Наискорейший спуск:

- выбираем направление спуска s ;
- шаг τ выбирается **точным line search**, то есть точной минимизацией $\Psi(x^{(k)} + \tau s)$ по τ :

$$\tau_k = \arg \min_{\tau} \Psi(x^{(k)} + \tau s).$$

То есть при фиксированном направлении s мы минимизируем функцию одной переменной τ .

$$\begin{aligned} \Psi(x^{(k)} + \tau s) &= \frac{1}{2}(A(x^{(k)} + \tau s), x^{(k)} + \tau s) - (b, x^{(k)} + \tau s) \\ &= \frac{1}{2}(Ax^{(k)}, x^{(k)}) + \tau(Ax^{(k)}, s) + \frac{1}{2}\tau^2(As, s) - (b, x^{(k)}) - \tau(b, s). \end{aligned}$$

При дифф-ии по τ не зависящие от него слагаемые исчезнут, и получим

$$\begin{aligned} \frac{d}{d\tau} \Psi(x^{(k)} + \tau s) &= (Ax^{(k)}, s) + \tau(As, s) - (b, s) \\ &= \tau(As, s) + (Ax^{(k)} - b, s) = \tau(As, s) + (r^{(k)}, s), \end{aligned}$$

где $r^{(k)} := Ax^{(k)} - b$ — невязка текущего приближения к решению.

Таким образом, в точке экстремума по τ условие

$$\frac{d}{d\tau} \Psi(x^{(k)} + \tau s) = 0$$

необходимо влечёт

$$\tau = -\frac{(r, s)}{(As, s)}.$$

Минимум действительно достигается, так как

$$\frac{d^2}{d\tau^2} \Psi(x^{(k)} + \tau s) = (As, s) > 0$$

в силу положительной определённости матрицы A .

Градиентный спуск: определение градиента функционала энергии

Если целевая функция $f(x)$ дифф-ма, то направление её наибольшего убывания в т. $x^{(k)}$ противоположно направлению вектора градиента

$$\nabla f(x) := f'(x^{(k)}) = \left(f'_1(x^{(k)}), f'_2(x^{(k)}), \dots, f'_n(x^{(k)}) \right)^\top.$$

Это наблюдение даёт начало одному из распространённых вариантов метода спуска для минимизации функций — **методу градиентного спуска**, в котором направлением спуска берётся “антиградиент”, т.е. вектор $-f'(x^{(k)})$, а очередной шаг выполняется по формуле

$$x^{(k+1)} \leftarrow x^{(k)} - \tau_k f'(x^{(k)}),$$

для некоторого $\tau_k \in \mathbb{R}$. Какой вид имеет градиентный спуск для минимизации функционала энергии

$$\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)?$$

Градиентный спуск: определение градиента функционала энергии

Вычислим его градиент по компоненте x_l :

$$\frac{\partial \Psi(x)}{\partial x_l} = \frac{\partial}{\partial x_l} \left(\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j - \sum_{i=1}^n b_i x_i \right) = \sum_{j=1}^n a_{lj} x_j - b_l,$$

где $l = 1, 2, \dots, n$. Т. о., градиент можно записать в векторной форме:

$$\nabla \Psi(x) = \left(\frac{\partial \Psi(x)}{\partial x_1}, \frac{\partial \Psi(x)}{\partial x_2}, \dots, \frac{\partial \Psi(x)}{\partial x_n} \right)^T = Ax - b,$$

т. е. градиент функционала Ψ равен невязке решаемой системы линейных уравнений в рассматриваемой точке.

Формула обновления градиентного спуска:

$$x^{(k+1)} \leftarrow x^{(k)} - \tau \nabla \Psi(x^{(k)}), \quad k = 0, 1, 2, \dots$$

Это эквивалентно методу Ричардсона:

$$x^{(k+1)} \leftarrow x^{(k)} - \tau (Ax^{(k)} - b).$$

Таким образом, метод градиентного спуска с постоянным шагом τ совпадает с итерационной схемой Ричардсона.

Для метода градиентного спуска с постоянным шагом его трактовка как метода Ричардсона позволяет выбрать шаг $\tau_k = \text{const}$, который наверняка обеспечивает сходимость процесса.

Пусть μ и M — нижняя и верхняя границы спектра симм. положит. определенной матрицы A . Тогда для сходимости итерационного процесса шаг τ должен быть выбран как:

$$\tau = \tau_k = \frac{2}{M + \mu}.$$

Наискорейший градиентный спуск

Можно искать τ_k наибольшим возможным, обеспечивающим убывание функционала Ψ вдоль выбранного направления спуска по антиградиенту. При этом получается **метод наискорейшего градиентного спуска (steepest descent algorithm)**, теория которого была разработана в конце 40-х годов XX века Л.В. Канторовичем.

Выражение для шага

$$\tau_k = \frac{(r^{(k)}, r^{(k)})}{(Ar^{(k)}, r^{(k)})}, \quad r^{(k)} := Ax^{(k)} - b.$$

```
k ← 0 ;  
выбираем начальное приближение  $x^{(0)}$  ;  
DO WHILE ( метод не сошёлся )  
     $r^{(k)} \leftarrow Ax^{(k)} - b$  ;  
     $\tau_k \leftarrow \frac{\|r^{(k)}\|_2^2}{\langle Ar^{(k)}, r^{(k)} \rangle}$  ;  
     $x^{(k+1)} \leftarrow x^{(k)} - \tau_k r^{(k)}$  ;  
    k ← k + 1 ;  
END DO
```

Наискорейший градиентный спуск: теорема о сходимости

Теорема. Если A – симметричная положительно определённая матрица, то метод наискорейшего спуска сходится к решению x^* системы уравнений $Ax = b$ из любого начального приближения $x^{(0)}$, и скорость этой сходимости оценивается неравенством

$$\|x^{(k)} - x^*\|_A \leq \left(\frac{M - \mu}{M + \mu} \right)^k \|x^{(0)} - x^*\|_A, \quad k = 0, 1, 2, \dots,$$

где μ , M – нижняя и верхняя границы спектра матрицы A .

Идея доказательства (сравнение с методом Ричардсона):

- Метод наискорейшего спуска минимизирует $\Psi(x)$ на каждом шаге.
- Сравним с методом Ричардсона с постоянным шагом $\tau = 2/(M + \mu)$.
- Тогда погрешность $\|x^{(k)} - x^*\|_A$ у наискорейшего метода не хуже, чем у Ричардсона:

$$\|x^{(k)} - x^*\|_A \leq \left(\frac{M - \mu}{M + \mu} \right)^k \|x^{(0)} - x^*\|_A.$$

Наискорейший градиентный спуск: геометрическая иллюстрация

- Градиент $\nabla\Psi(x)$ ортогонален уровням $\Psi(x)$:

если $x_0 + \varepsilon$ лежит на уровне $\Psi(x) = \Psi(x_0)$, то

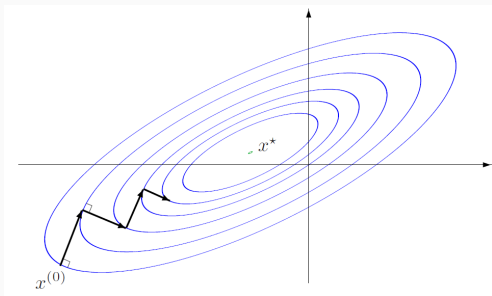
$$0 = \Psi(x_0 + \varepsilon) - \Psi(x_0) = (\nabla\Psi(x_0), \varepsilon) + o(\|\varepsilon\|).$$

Поэтому

$$\left(\nabla\Psi(x_0), \frac{\varepsilon}{\|\varepsilon\|} \right) = o(1), \quad \|\varepsilon\| \rightarrow 0,$$

и в пределе $\nabla\Psi(x_0)$ ортогонален касательной к линии уровня.

- Шаги метода идут по направлению $-\nabla\Psi(x)$, до касания уровня.
- В результате траектория – ломаная с перпендикулярными отрезками.



Градиентный спуск: применение в нейронных сетях

В нейронных сетях минимизируют функционал ошибки

$$Q(w) \rightarrow \min_w,$$

где w — набор всех весов модели.

- Градиентный спуск имеет вид

$$w^{(k+1)} = w^{(k)} - \tau_k \nabla Q(w^{(k)}).$$

- При большом числе данных полный градиент считать дорого.
- Поэтому на практике часто используют **стохастический градиентный спуск** (mini-batch):

$$w^{(k+1)} = w^{(k)} - \tau_k \nabla Q_{B_k}(w^{(k)}),$$

где B_k — текущий мини-батч, то есть часть обучающей выборки.

- Идея остается той же: двигаться в направлении убывания функционала, но направление вычисляется лишь приближенно.
- В отличие от задачи минимизации функционала энергии

$$\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$$

для $A = A^T > 0$, ф-я $Q(w)$ в нейросетях обычно невыпукла, поэтому гарантии нахождения глобального минимума, вообще говоря, нет.

Метод сопряженных направлений: переход от наискорейшего спуска

При точном выборе шага вдоль направления спуска (**exact line search**) в методе наискорейшего спуска новый градиент ортогонален предыдущему направлению спуска.

Так как в методе наискорейшего спуска направление равно антиградиенту, то антиградиенты у соседних шагов оказываются взаимно перпендикулярными.

Для квадратичного функционала это то же самое, что ортогональность соседних невязок:

$$(r^{(k+1)}, r^{(k)}) = 0, \quad r^{(k)} = Ax^{(k)} - b.$$

Для функционала энергии

$$\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$$

естественная геометрия определяется матрицей A .

Поэтому обычная евклидова ортогональность

$$(r^{(k+1)}, r^{(k)}) = 0$$

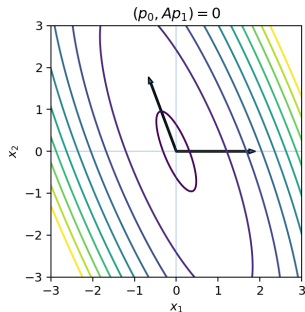
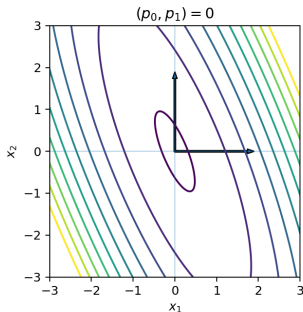
сама по себе не гарантирует, что новый шаг не изменит результат минимизации вдоль старого направления.

Метод сопряженных направлений: переход от наискорейшего спуска

Нужно заменить обычную ортогональность направлений на ортогональность, согласованную с матрицей A :

$$(p_i, Ap_j) = 0, \quad i \neq j.$$

Именно это условие приводит к методу сопряженных направлений.



Метод сопряженных направлений: вывод условия

Пусть на предыдущем шаге мы уже минимизировали Ψ вдоль направления p_j . Это означает

$$\left. \frac{d}{d\tau} \Psi(x + \tau p_j) \right|_{\tau=0} = (\nabla \Psi(x), p_j) = 0.$$

Теперь сделаем новый шаг вдоль направления p_i :

$$x \leftarrow x + \alpha p_i.$$

Хотим, чтобы минимум вдоль старого направления p_j не испортился:

$$(\nabla \Psi(x + \alpha p_i), p_j) = 0.$$

Но так как для $\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$ при $A = A^\top$ имеем $\nabla \Psi(x) = Ax - b$, то

$$\nabla \Psi(x + \alpha p_i) = A(x + \alpha p_i) - b = Ax - b + \alpha A p_i = \nabla \Psi(x) + \alpha A p_i.$$

Поэтому достаточно потребовать

$$(A p_i, p_j) = 0.$$

То есть новый шаг должен быть ортогонален старому не в обычном, а в A -скалярном произведении.

Метод сопряженных направлений: определение

Векторы $p_0, p_1, \dots, p_m$ называются **A-сопряженными**, если

$$(p_i, Ap_j) = 0, \quad i \neq j.$$

Эквивалентно:

$$(p_i, p_j)_A = 0, \quad i \neq j, \quad (u, v)_A := (Au, v).$$

- Это не обычная ортогональность, а ортогональность в геометрии, задаваемой матрицей A .
- Для функционала энергии

$$\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$$

такие направления особенно удобны.

- Минимизация вдоль нового A -сопряженного направления не разрушает минимум, найденный вдоль предыдущих направлений.
- Поэтому метод не возвращается к уже исправленным компонентам ошибки.

Метод сопряженных направлений

Пусть построены A -сопряженные направления

$$p_0, p_1, \dots, p_{n-1}.$$

После k шагов приближение ищется среди векторов вида

$$x^{(0)} + \gamma_0 p_0 + \gamma_1 p_1 + \dots + \gamma_{k-1} p_{k-1},$$

то есть среди всех векторов из множества

$$x^{(0)} + \text{Lin}(p_0, \dots, p_{k-1}),$$

где $\text{Lin}(p_0, \dots, p_{k-1})$ — линейная оболочка векторов $p_0, \dots, p_{k-1}$.

На следующем шаге минимизируем Ψ вдоль очередного направления:

$$x^{(k+1)} = x^{(k)} + \alpha_k p_k.$$

Из условия

$$\frac{d}{d\alpha} \Psi(x^{(k)} + \alpha p_k) = 0$$

получаем

$$\alpha_k = -\frac{(r^{(k)}, p_k)}{(Ap_k, p_k)}, \quad r^{(k)} = Ax^{(k)} - b.$$

В точной арифметике при линейной независимости направлений решение находится не более чем за n шагов.

Метод сопряженных градиентов (CG)

Метод сопряженных градиентов (conjugate gradient method – CG) — это частный случай метода сопряженных направлений, в котором направления строятся автоматически по невязкам.

- После шага по $p^{(k)}$ новый антиградиент

$$-r^{(k+1)}$$

указывает направление убывания, но сам по себе не обязан быть A -сопряженным прежнему направлению $p^{(k)}$.

- Поэтому новое направление берут в виде

$$p^{(k+1)} = -r^{(k+1)} + \beta_k p^{(k)}.$$

- Коэффициент β_k выбирают так, чтобы выполнялось условие сопряженности

$$(p^{(k+1)}, Ap^{(k)}) = 0.$$

- Для линейного CG получается формула

$$\beta_k = \frac{(r^{(k+1)}, r^{(k+1)})}{(r^{(k)}, r^{(k)})}.$$

Метод сопряженных градиентов (CG): алгоритм

Пусть $A = A^T > 0$. Зададим

$$r^{(0)} = Ax^{(0)} - b, \quad p^{(0)} = -r^{(0)}.$$

Для $k = 0, 1, 2, \dots$:

$$\alpha_k = -\frac{(r^{(k)}, p^{(k)})}{(Ap^{(k)}, p^{(k)})},$$

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)},$$

$$r^{(k+1)} = r^{(k)} + \alpha_k Ap^{(k)},$$

$$\beta_k = \frac{(r^{(k+1)}, r^{(k+1)})}{(r^{(k)}, r^{(k)})},$$

$$p^{(k+1)} = -r^{(k+1)} + \beta_k p^{(k)}.$$

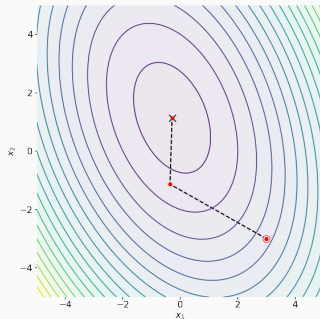
Метод сопряженных градиентов (CG): геометрическая иллюстрация

Для квадратичного функционала

$$\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$$

метод сопряженных градиентов строит направления, которые не уничтожают уже сделанное продвижение.

- Наискорейший спуск может долго идти зигзагами.
- Метод сопряженных градиентов использует A -сопряженные направления.
- Поэтому для двумерной квадратичной задачи он может прийти в минимум за два шага.
- В n -мерном случае в точной арифметике нужно не более n шагов.



Метод сопряженных градиентов (CG): свойства

Метод сопряженных градиентов применим к системам

$$Ax = b$$

с симметричной положительно определенной матрицей

$$A = A^T > 0.$$

- В точной арифметике метод находит решение не более чем за n шагов.

- На практике обычно останавливаются раньше:

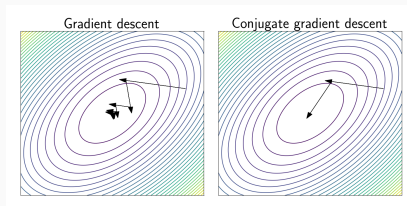
$$\frac{\|Ax^{(k)} - b\|}{\|b\|} < \varepsilon.$$

- Скорость сходимости зависит от распределения собственных значений.
- Если собственные значения хорошо сгруппированы, метод обычно сходится быстро.
- Плохая обусловленность замедляет сходимость.

Метод сопряженных градиентов (CG): сравнение с градиентным спуском

На плохо обусловленной квадратичной задаче градиентный спуск может долго двигаться по узкому оврагу.

- Обычный градиентный спуск использует только локальное направление антиградиента.
- Метод сопряженных градиентов учитывает геометрию, задаваемую матрицей A .
- Поэтому он обычно требует существенно меньше итераций.



Метод сопряженных градиентов: крыловские подпространства

До сих пор в лекции невязка обозначалась как

$$Ax^{(k)} - b,$$

потому что для функционала энергии она совпадает с градиентом.

В литературе по **крыловским методам** чаще используют обозначение

$$r^{(k)} = b - Ax^{(k)}.$$

Тогда определяют **крыловское подпространство**

$$K_k(A, r^{(0)}) = \text{Lin}\{r^{(0)}, Ar^{(0)}, \dots, A^{k-1}r^{(0)}\}.$$

При таком обозначении метод сопряженных градиентов тоже можно понимать как поиск приближения в множестве

$$x^{(k)} \in x^{(0)} + K_k(A, r^{(0)}).$$

Это описание будет удобно для MINRES, GMRES, BiCG и BiCGSTAB.

Предобуславливание: основная идея

Если матрица A плохо обусловлена, то даже метод сопряженных градиентов может сходиться медленно.

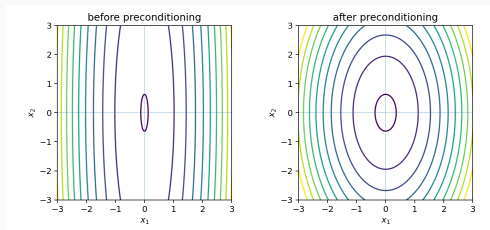
Идея предобуславливания: заменить

систему $Ax = b$ эквивалентной системой $M^{-1}Ax = M^{-1}b$, где M устроена проще, чем A , но приближает ее.

- Нужно, чтобы новая система была лучше обусловлена:

$$\kappa(M^{-1}A) \ll \kappa(A).$$

- При этом системы с матрицей M должны решаться быстро.
- Предобуславливатель меняет геометрию задачи так, чтобы итерационный метод двигался эффективнее.



Предобуславливание: примеры

Предобуславливатель M выбирают так, чтобы

- он был проще, чем A ;
- систему

$$Mz = r$$

можно было решать быстро;

- при этом M в каком-то смысле приближал матрицу A .

Поэтому возникают уже знакомые идеи:

- **Диагональный предобуславливатель**

$$M = \text{diag}(A).$$

Это та же идея, что и в методе Якоби: берем только диагональ.

- **PCG** (Preconditioned Conjugate Gradient) — предобусловленный метод сопряженных градиентов.
- В PCG вместо невязки используют

$$z^{(k)} = M^{-1}r^{(k)}, \quad r^{(k)} = b - Ax^{(k)}.$$

- Для симметричных положительно определенных матриц часто используют неполное разложение Холецкого:

$$M \approx LL^T.$$

- Для несимметричных матриц используют неполное LU-разложение:

$$M \approx LU.$$

- В случае Холецкого отдельный верхний фактор не выписывают, потому что он равен L^T .
- В SciPy есть функция `scipy.sparse.linalg.spilu`, где

$$\text{ILU} = \text{Incomplete LU}.$$

- Отдельной публичной sparse-функции для неполного разложения Холецкого в SciPy сейчас нет.

Предобуславливание: пример ILU в SciPy

В SciPy `spilu` строит приближенное разложение $M \approx LU$.

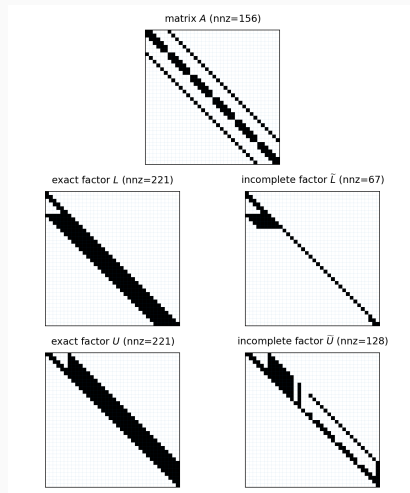
Для применения предобуславливателя используют оба фактора:

$$Ly = r, \quad Uz = y.$$

На рисунке показаны одна и та же разреженная матрица A , а также точные и неполные факторы L и U . Числа **nnz** – number of nonzeros – в заголовках показывают, что неполные факторы заметно реже точных.

Если число сохраняемых ненулевых элементов в строке ограничено, то построение и применение такого предобуславливателя имеют порядок $O(n)$.

Для плотной матрицы полное LU имеет стоимость $O(n^3)$.



MINRES: симметричная, но не положительно определенная матрица

Метод сопряженных градиентов требует

$$A = A^T > 0.$$

Если матрица симметрична, но не является положительно определенной, то функционал энергии

$$\Psi(x) = \frac{1}{2}(Ax, x) - (b, x)$$

может не иметь минимума.

В этом случае можно минимизировать не функционал энергии, а норму невязки:

$$\|Ax - b\|_2 \rightarrow \min.$$

Метод MINRES ищет приближение в виде

$$x^{(k)} = x^{(0)} + y_0 r^{(0)} + y_1 A r^{(0)} + \dots + y_{k-1} A^{k-1} r^{(0)},$$

то есть

$$x^{(k)} \in x^{(0)} + K_k(A, r^{(0)}), \quad r^{(0)} = b - Ax^{(0)}.$$

Коэффициенты $y_0, \dots, y_{k-1}$ выбирают так, чтобы норма невязки была минимальна.

Если матрица несимметрична, $A \neq A^T$, то методы CG и MINRES в их прежнем виде уже неприменимы: в них существенно использовалась симметрия матрицы.

Метод GMRES сохраняет общую идею крыловских методов: ищем приближение в виде

$$x^{(k)} \in x^{(0)} + K_k(A, r^{(0)}), \quad r^{(0)} = b - Ax^{(0)},$$

где

$$K_k(A, r^{(0)}) = \text{Lin}\{r^{(0)}, Ar^{(0)}, \dots, A^{k-1}r^{(0)}\}.$$

То есть после k шагов рассматриваются все векторы вида

$$x^{(k)} = x^{(0)} + y_0 r^{(0)} + y_1 Ar^{(0)} + \dots + y_{k-1} A^{k-1} r^{(0)}.$$

Из всех векторов выбирается тот, для которого норма невязки минимальна:

$$\|b - Ax^{(k)}\|_2 \rightarrow \min.$$

Иначе говоря, GMRES выбирает лучшую линейную комбинацию векторов

$$r^{(0)}, Ar^{(0)}, \dots, A^{k-1}r^{(0)}$$

по критерию минимальной нормы невязки.

GMRES: отличие от предыдущих методов

- **CG** применим к матрицам

$$A = A^T > 0$$

и использует специальную структуру квадратичной задачи.

- **MINRES** тоже требует симметрию

$$A = A^T,$$

но не требует положительной определенности.

- **GMRES** работает с общей несимметричной матрицей

$$A \neq A^T$$

и на каждом шаге выбирает в текущем крыловском подпространстве вектор с наименьшей нормой невязки.

- Так как

$$K_k(A, r^{(0)}) \subset K_{k+1}(A, r^{(0)}),$$

на каждом следующем шаге выбор делается из более широкого множества.

- Поэтому норма невязки в GMRES по шагам не возрастает.
- Цена за это — нужно хранить все уже построенные базисные векторы текущего крыловского подпространства.

Метод GMRES уже позволяет решать несимметричные системы

$$Ax = b.$$

Но это не единственный возможный путь.

В GMRES каждое новое приближение выбирается из все более широкого крыловского подпространства, поэтому по ходу работы приходится хранить все уже построенные векторы.

Для больших разреженных матриц интересны и другие методы, устроенные ближе по духу к CG: новое приближение в них строится по формулам, использующим лишь небольшое число текущих векторов, а не всю накопленную информацию.

Один из таких методов — метод бисопряженных градиентов, к которому мы сейчас и перейдем.

Метод бисопряженных градиентов: идея

Для несимметричной системы

$$Ax = b$$

метод бисопряженных градиентов (Biconjugate gradient method, BiCG) строит два связанных итерационных процесса: один для матрицы A , другой для матрицы A^T .

Начинаем с невязки

$$r^{(0)} = b - Ax^{(0)}$$

и произвольно выбираем вспомогательный вектор из критерия

$$\tilde{r}^{(0)} \neq 0, \quad (\tilde{r}^{(0)}, r^{(0)}) \neq 0.$$

Затем строятся две последовательности невязок и направлений:

$$r^{(k)}, p^{(k)} \quad \text{и} \quad \tilde{r}^{(k)}, \tilde{p}^{(k)}.$$

Они выбираются так, чтобы выполнялись условия

$$(\tilde{r}^{(i)}, r^{(j)}) = 0, \quad (\tilde{p}^{(i)}, Ap^{(j)}) = 0, \quad i \neq j.$$

Это называется **биортогональностью**: векторы из одной последовательности ортогональны векторам из другой

Метод бисопряженных градиентов: алгоритм

Пусть

$$\begin{aligned}r^{(0)} &= b - Ax^{(0)}, & \tilde{r}^{(0)} &\neq 0, & (\tilde{r}^{(0)}, r^{(0)}) &\neq 0, \\p^{(0)} &= r^{(0)}, & \tilde{p}^{(0)} &= \tilde{r}^{(0)}.\end{aligned}$$

Для $k = 0, 1, 2, \dots$ вычисляем коэффициент шага

$$\alpha_k = \frac{(\tilde{r}^{(k)}, r^{(k)})}{(\tilde{p}^{(k)}, A\tilde{p}^{(k)})}.$$

Он задает шаг вдоль направления $p^{(k)}$:

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

После этого обновляем невязки:

$$\begin{aligned}r^{(k+1)} &= r^{(k)} - \alpha_k A p^{(k)}, \\ \tilde{r}^{(k+1)} &= \tilde{r}^{(k)} - \alpha_k A^\top \tilde{p}^{(k)}.\end{aligned}$$

После обновления невязок вычисляем коэффициент

$$\beta_k = \frac{(\tilde{r}^{(k+1)}, r^{(k+1)})}{(\tilde{r}^{(k)}, r^{(k)})}.$$

Он показывает, какая часть предыдущего направления переносится в новое направление.

Затем строим новые направления:

$$p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)},$$

$$\tilde{p}^{(k+1)} = \tilde{r}^{(k+1)} + \beta_k \tilde{p}^{(k)}.$$

При этом сохраняются условия биортогональности:

$$(\tilde{r}^{(i)}, r^{(j)}) = 0, \quad (\tilde{p}^{(i)}, Ap^{(j)}) = 0, \quad i \neq j.$$

Метод BiCG можно понимать как несимметричный аналог CG.

- Для несимметричных систем он использует две последовательности невязок и направлений.
- В отличие от GMRES, метод не выбирает на каждом шаге вектор с наименьшей нормой невязки.
- Поэтому сходимость может быть нерегулярной.
- Если знаменатели в формулах для α_k или β_k оказываются нулевыми или слишком малыми, шаг невозможно или ненадежно вычислить.
- Требуется умножение не только на A , но и на A^T .

BiCGSTAB — стабилизированный вариант метода BiCG для несимметричных систем $Ax = b$.

В методе BiCG невязка может убывать нерегулярно: после хорошего шага возможен заметный всплеск.

В BiCGSTAB к шагу типа BiCG добавляется дополнительная коррекция.

Одна итерация устроена так:

- сначала строится направление $p^{(k)}$ и выполняется шаг, похожий на шаг BiCG;
- после этого вычисляется промежуточная невязка

$$s^{(k)};$$

- затем вдоль вектора $s^{(k)}$ делается еще одна коррекция с коэффициентом

$$\omega_k;$$

- эта коррекция подбирается так, чтобы сделать новую невязку как можно меньше.

В стандартной реализации метод не требует умножений на A^T .

Задаем начальное приближение $x^{(0)}$ и начальную невязку $r^{(0)} = b - Ax^{(0)}$.

Выбираем фиксированный вспомогательный вектор

$$\tilde{r}^{(0)}, \quad (\tilde{r}^{(0)}, r^{(0)}) \neq 0.$$

Начальная инициализация:

$$p^{(0)} = r^{(0)}, \quad \rho_0 = (\tilde{r}^{(0)}, r^{(0)}).$$

Для $k = 0, 1, 2, \dots$ выполняем:

$$v^{(k)} = Ap^{(k)}, \quad \alpha_k = \frac{\rho_k}{(\tilde{r}^{(0)}, v^{(k)})},$$

$$s^{(k)} = r^{(k)} - \alpha_k v^{(k)},$$

$$t^{(k)} = As^{(k)}, \quad \omega_k = \frac{(t^{(k)}, s^{(k)})}{(t^{(k)}, t^{(k)})},$$

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)} + \omega_k s^{(k)},$$

$$r^{(k+1)} = s^{(k)} - \omega_k t^{(k)}.$$

После обновления невязки вычисляем

$$\rho_{k+1} = (\tilde{r}^{(0)}, r^{(k+1)}).$$

Затем находим коэффициент

$$\beta_{k+1} = \frac{\rho_{k+1}}{\rho_k} \frac{\alpha_k}{\omega_k}$$

и обновляем направление:

$$p^{(k+1)} = r^{(k+1)} + \beta_{k+1}(p^{(k)} - \omega_k v^{(k)}).$$

Таким образом, одна итерация состоит из двух частей:

- шаг вдоль основного направления

$$p^{(k)};$$

- дополнительная коррекция вдоль промежуточной невязки

$$s^{(k)}.$$

Именно эта вторая часть обычно делает поведение невязки более гладким, чем в BiCG.

BiCGSTAB: как понимать эти формулы

В формулах предыдущего слайда:

- $p^{(k)}$ — основное направление шага;
- $v^{(k)} = Ap^{(k)}$ — образ этого направления под действием матрицы A ;
- α_k — коэффициент шага вдоль $p^{(k)}$;
- $s^{(k)}$ — невязка после этого первого шага;
- $t^{(k)} = As^{(k)}$ — образ промежуточной невязки;
- ω_k — коэффициент дополнительной коррекции вдоль $s^{(k)}$.

То есть одна итерация BiCGSTAB состоит из двух частей:

- шаг типа BiCG:

$$x^{(k)} \rightarrow x^{(k)} + \alpha_k p^{(k)};$$

- стабилизирующая коррекция:

$$x^{(k)} + \alpha_k p^{(k)} \rightarrow x^{(k)} + \alpha_k p^{(k)} + \omega_k s^{(k)}.$$

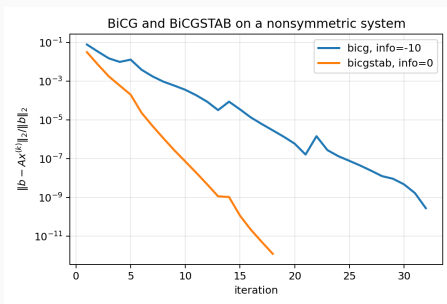
Именно эта вторая часть обычно делает поведение невязки более гладким.

BiCGSTAB: зачем нужна стабилизация

У BiCG невязка может вести себя нерегулярно: убывание может чередоваться со всплесками.

BiCGSTAB добавляет стабилизирующий шаг, поэтому часто дает более гладкую сходимость.

- BiCGSTAB обычно устойчивее BiCG на практике.
- Но универсальной гарантии быстрой сходимости нет.
- Результат сильно зависит от матрицы и предобуславливателя.

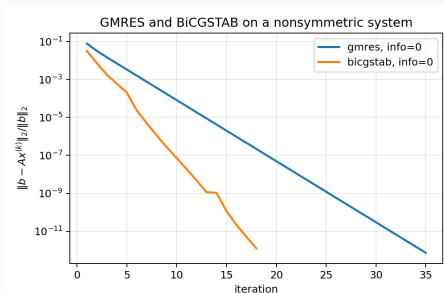


GMRES и BiCGSTAB: сравнение на примере

Оба метода применимы к несимметричным системам $Ax = b$.

- GMRES на каждом шаге минимизирует норму невязки, но требует хранить все уже построенные векторы текущего крыловского подпространства.
- BiCGSTAB не минимизирует невязку на каждом шаге, но использует более компактную итерационную схему.

На рисунке показан **один конкретный пример**, в котором BiCGSTAB сошелся быстрее GMRES. Это не означает, что BiCGSTAB вообще лучше: универсального лидера здесь нет.



Крыловские методы: общая картина

Многие нестационарные методы строят поправку к начальному приближению в крыловском подпространстве:

$$K_k(A, r^{(0)}) = \text{Lin}\{r^{(0)}, Ar^{(0)}, \dots, A^{k-1}r^{(0)}\}, \quad r^{(0)} = b - Ax^{(0)}.$$

То есть

$$x^{(k)} \in x^{(0)} + K_k(A, r^{(0)}).$$

Итого:

- CG минимизирует ошибку в энергетической норме для

$$A = A^T > 0;$$

- MINRES минимизирует невязку для симметричных матриц;
- GMRES минимизирует невязку для несимметричных матриц;
- BiCG использует две связанные последовательности для A и A^T ;
- BiCGSTAB строится на той же идее, но добавляет стабилизирующую коррекцию.

Сравнительная таблица с методами

Тип матрицы	Метод
$A = A^T > 0$	CG, PCG
$A = A^T$, не обязательно $A > 0$	MINRES
$A \neq A^T$	GMRES, BiCG, BiCGSTAB

- CG использует A -сопряженные направления.
- PCG ускоряет CG с помощью предобуславливателя.
- MINRES и GMRES минимизируют норму невязки.
- BiCG и BiCGSTAB применяются к несимметричным системам и строят новое приближение без хранения всей накопленной последовательности векторов.

Методы 2-го порядка: основная идея

До сих пор мы использовали методы 1-го порядка: они строят шаг по значению функции и её градиенту.

В **методах 2-го порядка** дополнительно используют матрицу вторых производных

$$H(x) = \nabla^2 f(x),$$

которую называют **гессианом**.

В точке $x^{(k)}$ функцию заменяют локальной квадратичной моделью:

$$f(x^{(k)} + p) \approx f(x^{(k)}) + (\nabla f(x^{(k)}), p) + \frac{1}{2}(H(x^{(k)})p, p).$$

Минимум этой квадратичной модели достигается в направлении $p^{(k)}$, которое находится из линейной системы

$$H(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

После этого делают шаг

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

То есть на каждом шаге метод 2-го порядка сводит задачу оптимизации к решению линейной системы.

Методы 2-го порядка: плюсы и минусы

Плюсы:

- обычно требуется меньше итераций;
- вблизи минимума сходимость часто намного быстрее, чем у методов 1-го порядка.

Минусы:

- нужно вычислять или приближать гессиан;
- на каждом шаге требуется решать линейную систему;
- одна итерация заметно дороже, особенно в больших размерностях.

Методы 2-го порядка: примеры

- **Метод Ньютона** — использует точный гессиан $H(x^{(k)})$ и на каждом шаге решает линейную систему $H(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)})$, после чего делает шаг

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

- **Демпфированный метод Ньютона / trust-region методы** — ограничивают шаг, чтобы сделать метод устойчивее вдали от минимума.
- **Квазиньютоновские методы (BFGS, L-BFGS)** — не вычисляют точный гессиан, а строят его приближение по информации о предыдущих шагах.
- **Метод Гаусса–Ньютона** — специальный метод для задач наименьших квадратов $\min \frac{1}{2} \|F(x)\|_2^2$, в котором гессиан приближенно заменяют матрицей

$$J(x)^T J(x),$$

где $J(x)$ — якобиан вектора невязок $F(x)$.

- **Метод Левенберга–Марквардта** — модификация для задач наименьших квадратов и подбора параметров модели по данным; сочетает идею Гаусса–Ньютона с демпфированием шага.

Задача 7

Решить краевую задачу

$$\begin{aligned}-u''(t) + (1+t)u(t) &= (\pi^2 + 1 + t)\sin(\pi t), \quad 0 < t < 1, \\ u(0) &= 0, \quad u(1) = 0.\end{aligned}$$

Точное решение:

$$u_*(t) = \sin(\pi t).$$

Для равномерной сетки

$$t_i = ih, \quad i = 0, 1, \dots, n+1, \quad h = \frac{1}{n+1},$$

построить разностный аналог задачи:

$$\begin{aligned}-\frac{x_{i-1} - 2x_i + x_{i+1}}{h^2} + (1+t_i)x_i &= (\pi^2 + 1 + t_i)\sin(\pi t_i), \quad i = 1, \dots, n, \\ x_0 &= 0, \quad x_{n+1} = 0.\end{aligned}$$

Полученную систему записать в виде

$$Ax = b.$$

Показать, что матрица A симметрична и положительно определена.

Задача 7

Для системы $Ax = b$ реализовать и сравнить следующие методы:

- метод градиентного спуска с постоянным шагом

$$x^{(k+1)} = x^{(k)} - \tau(Ax^{(k)} - b);$$

- метод наискорейшего градиентного спуска

$$x^{(k+1)} = x^{(k)} - \tau_k r^{(k)}, \quad r^{(k)} = Ax^{(k)} - b, \quad \tau_k = \frac{(r^{(k)}, r^{(k)})}{(Ar^{(k)}, r^{(k)})};$$

- метод сопряженных градиентов.

Остановить итерации при условии

$$\frac{\|Ax^{(k)} - b\|_2}{\|b\|_2} < 10^{-8}.$$

Для $n = 50, 100, 200, 400$ сравнить число итераций, время счета и ошибку

$$\max_i |x_i - u_*(t_i)|.$$

Для каждой сетки оценить число обусловленности

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$$

и объяснить, как рост $\kappa(A)$ влияет на число итераций.

Шарый С. П.: Курс ВЫЧИСЛИТЕЛЬНЫХ МЕТОДОВ

Wikipedia: Biconjugate gradient method

Netlib Templates

[Стохастический градиентный спуск](#)

Нелинейное преобразование Фурье

Чеховской Игорь Сергеевич

Линейное преобразование Фурье

Рассмотрим линейное волновое уравнение на бесконечной прямой:

$$u_{tt} = a^2 u_{xx}, \quad x \in \mathbb{R}.$$

Возьмём по пространственной переменной x прямое преобр-е Фурье:

$$\tilde{u}(\omega, t) = \mathcal{F}_x[u](\omega, t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{+\infty} u(x, t) e^{i\omega x} dx.$$

Используем свойство производной:

$$\mathcal{F}_x[u_x](\omega, t) = i\omega \tilde{u}(\omega, t).$$

Применяя $\mathcal{F}_x$ к уравнению, получим для каждого фиксированного ω обыкновенное дифференциальное уравнение по t :

$$\tilde{u}_{tt}(\omega, t) + a^2 \omega^2 \tilde{u}(\omega, t) = 0.$$

Его решение:

$$\tilde{u}(\omega, t) = C_1(\omega) \cos(a\omega t) + C_2(\omega) \frac{\sin(a\omega t)}{a\omega},$$

где $C_1(\omega)$, $C_2(\omega)$ задаются начальными условиями.

Исходное решение восстанавливается обратным преобразованием Фурье:

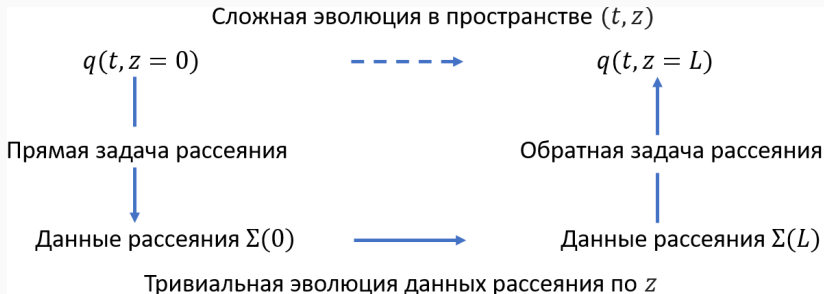
$$u(x, t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{+\infty} \tilde{u}(\omega, t) e^{-i\omega x} d\omega.$$

А что если уравнение **нелинейное**, например, $u_t - 6uu_x + u_{xxx} = 0$ — уравнение Кортевега – де Фриза?

Метод обратной задачи рассеяния

Метод обратной задачи рассеяния (МОЗР) – *аналитический* метод решения задачи Коши для некоторых нелинейных эволюционных уравнений (*интегрируемых*).

Метод представляет собой аналог метода Фурье решения линейных дифференциальных уравнений в частных производных.



Метод обратной задачи рассеяния

МОЗР был предложен в 1967 г. К. С. Гарднером, Дж. М. Грином, М. Д. Крускалом и Р. М. Миурой, применивших его к уравнению Кортевега – де Фриза (КдФ):

$$u_t - 6uu_x + u_{xxx} = 0.$$

Это уравнение было выведено в конце XIX века для описания волн на мелкой воде. Тогда же были получены некоторые его точные решения – солитоны. Интерес к солитонам возобновился в связи с исследованиями по физике плазмы в 60-х годах XX века. В 1965 году М. Д. Крускал и Н. Забуски обнаружили путем численного моделирования, что солитоны уравнения Кортевега – де Фриза сталкиваются упруго, т.е. после взаимодействия они сохраняют свою форму и скорость, как будто просто прошли друг сквозь друга (эффект, совершенно не характерный для линейных волн). Этот результат дал толчок к новым исследованиям, которые в результате привели к возникновению МОЗР.

В 1971 г. В. Е. Захаров и А. Б. Шабат применили МОЗР к **нелинейному уравнению Шредингера** (НУШ).

В данный момент метод также известен под названием **нелинейное преобразование Фурье** (nonlinear Fourier transform – NFT).

Задача Захарова-Шабата (ЗШ)

НУШ с затухающими гран. условиями:

$$\begin{aligned} i \frac{\partial q}{\partial z} + \frac{\sigma}{2} \frac{\partial^2 q}{\partial t^2} + |q|^2 q &= 0. \\ q(z, t) &\rightarrow 0, \quad t \rightarrow \pm\infty \end{aligned} \quad (1)$$

является условием совместности (представление/уравнения Лакса)

$$\frac{\partial \hat{L}}{\partial z} = [\hat{M}, \hat{L}] = \hat{M}\hat{L} - \hat{L}\hat{M} \quad (2)$$

двух линейных уравнений

$$\hat{L}\Psi = \zeta\Psi, \quad \frac{\partial \Psi}{\partial z} = \hat{M}\Psi,$$

где операторы $\hat{L}$, $\hat{M}$ – пара Лакса:

$$\hat{L} = i \begin{pmatrix} \partial_t & -q(z, t) \\ -\sigma q^*(z, t) & -\partial_t \end{pmatrix} \quad (3)$$

$$\hat{M} = \begin{pmatrix} -2i\zeta^2 + i|q(z, t)|^2 & 2q(z, t)\zeta + iq_t(z, t) \\ -2q^*(z, t)\zeta + iq_t^*(z, t) & 2i\zeta^2 - i|q(z, t)|^2 \end{pmatrix} \quad (4)$$

Здесь $\zeta = \xi + i\eta$ – комплексный спектральный параметр.

Прямая задача ЗШ – нахождение спектра $\{\zeta\}$ оператора $\hat{L}$:

$$\hat{L}\Phi = \zeta\Phi, \quad \Phi = \begin{pmatrix} \phi_1 \\ \phi_2 \end{pmatrix} \quad (5)$$

или

$$\begin{cases} \frac{d\phi_1}{dt} = q(z, t)\phi_2 - i\zeta\phi_1 \\ \frac{d\phi_2}{dt} = -\sigma q^*(z, t)\phi_1 + i\zeta\phi_2 \end{cases} \quad (6)$$

Частные решения для прямой задачи ЗШ:

$$\phi_1(t, \zeta) = e^{-i\zeta t} + O(1), \quad \phi_2(t, \zeta) = O(1), \quad t \rightarrow -\infty; \quad (7)$$

$$\psi_1(t, \zeta) = O(1), \quad \psi_2(t, \zeta) = e^{i\zeta t} + O(1), \quad t \rightarrow \infty.$$

Прямая задача

Указанные функции решают прямую задачу ЗШ, и все эти различные решения оказываются линейно зависимыми:

$$\begin{pmatrix} \phi_1 \\ \phi_2 \end{pmatrix} = a(\zeta) \begin{pmatrix} \tilde{\psi}_1 \\ \tilde{\psi}_2 \end{pmatrix} + b(\zeta) \begin{pmatrix} \psi_1 \\ \psi_2 \end{pmatrix},$$
$$\begin{pmatrix} \tilde{\phi}_1 \\ \tilde{\phi}_2 \end{pmatrix} = -\tilde{a}(\zeta) \begin{pmatrix} \tilde{\psi}_1 \\ \tilde{\psi}_2 \end{pmatrix} + \tilde{b}(\zeta) \begin{pmatrix} \psi_1 \\ \psi_2 \end{pmatrix} \quad (8)$$

Здесь по определению

$$\begin{pmatrix} \tilde{\psi}_1 \\ \tilde{\psi}_2 \end{pmatrix} = \begin{pmatrix} \psi_2^*(t, \zeta^*) \\ -\psi_1^*(t, \zeta^*) \end{pmatrix}$$

В (8) функции $a(\zeta)$, $b(\zeta)$ называются **функциями рассеяния Йоста**. Из граничных условий (7) вытекает, что

$$a(\zeta) = \lim_{t \rightarrow +\infty} \phi_1(t, \zeta) e^{i\zeta t}, \quad (9)$$

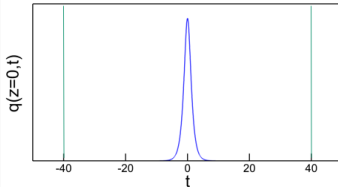
$$b(\zeta) = \lim_{t \rightarrow +\infty} \phi_2(t, \zeta) e^{-i\zeta t}. \quad (10)$$

Свойство коэффициентов рассеяния Йоста:

$$|a(\xi)|^2 + \sigma |b(\xi)|^2 = 1.$$

Прямая задача

$$\begin{cases} \frac{d\phi_1}{dt} = q(z_0, t)\phi_2 - i\zeta\phi_1 \\ \frac{d\phi_2}{dt} = -\sigma q^*(z_0, t)\phi_1 + i\zeta\phi_2 \end{cases}$$



$$\begin{array}{ccc} \phi_1(-T) = e^{i\zeta T} & \xrightarrow{\hspace{1cm}} & \phi_1(T) \\ \phi_2(-T) = 0 & & \phi_2(T) \end{array}$$



Коэффициенты Йоста:

$$a(\zeta) = \phi_1(T, \zeta) e^{i\zeta T}, \quad b(\zeta) = \phi_2(T, \zeta) e^{-i\zeta T}$$

Физический смысл $a(\zeta)$ и $b(\zeta)$: если слева приходит волна единичной амплитуды, то $a(\zeta)$ – это коэффициент прохождения, а $b(\zeta)$ – коэффициент отражения (аналогия с квантовой механикой).

1. Линейная алгебра (матрицы).

- Есть матрица $A \in \mathbb{C}^{n \times n}$.
- Собственные значения $\lambda_1, \dots, \lambda_n$ образуют **дискретный спектр**.
- Собственные векторы образуют базис в $\mathbb{C}^n$.

2. Оператор на бесконечной прямой.

- Рассмотрим оператор типа $-\partial_{xx}$ на $\mathbb{R}$.
- Собственные функции пропорциональны плоским волнам $e^{i\xi x}$, параметр ξ пробегает *всю* прямую.
- Вместо суммы по λ_k получаем интеграл по непрерывному параметру ξ :

$$u(x) = \sum_k c_k \phi_k(x) \quad \longrightarrow \quad u(x) = \int_{-\infty}^{+\infty} \hat{u}(\xi) e^{i\xi x} d\xi.$$

Для задачи ЗШ:

- **дискретный спектр** $\zeta_n \Rightarrow$ солитоны (локализованные «частицы»);
- **непрерывный спектр** $\xi \in \mathbb{R} \Rightarrow$ «радиация» / фон, аналог обычных волн.

Спектр оператора $\hat{L}$ состоит из двух частей:

1) **дискретный спектр** из дискретных собственных значений (д. с. з.) $\zeta_n = \xi_n + i\eta_n$, являющихся собственными значениями задачи ЗШ с положительной мнимой частью $\eta_n > 0$. Каждому с. з. соответствует левая и правая **нормировочная константа**, определяемая вычетами

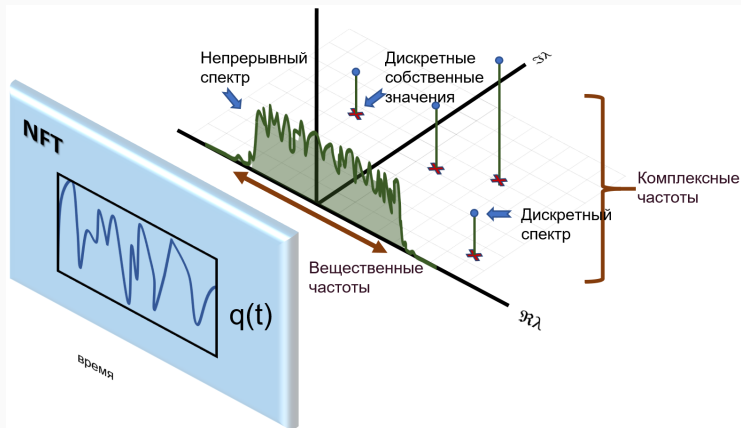
$$l_n = \frac{1}{b(\zeta_n) a'(\zeta_n)}, \quad r_n = \frac{b(\zeta_n)}{a'(\zeta_n)}.$$

Здесь штрих обозначает производную по отношению к комплексному параметру ζ .

2) **непрерывный спектр**, задающийся через левый и правый коэффициенты отражения:

$$l(\xi) = \frac{b^*(\xi)}{a(\xi)}, \quad r(\xi) = \frac{b(\xi)}{a(\xi)}, \quad \xi \in \mathbb{R}.$$

Прямая задача



Прямая задача

В случае аномальной дисперсии ($\sigma = 1$, фокусирующий НУШ) существует дискретный спектр, а отдельному д. с. з. соответствует солитон

$$q(\tau, z = 0) = A \operatorname{sech}[A \cdot (\tau - T)] e^{-i\Omega\tau + i\varphi}.$$

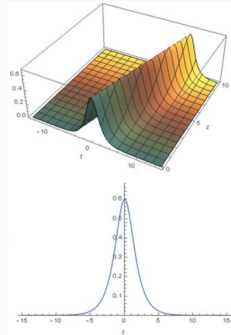
Его параметры определяются следующим образом через д. с. з. $\zeta_0 = \xi_0 + i\eta_0$ и правую нормировочную константу r_0 :

$A = 2\eta_0$ – амплитуда,

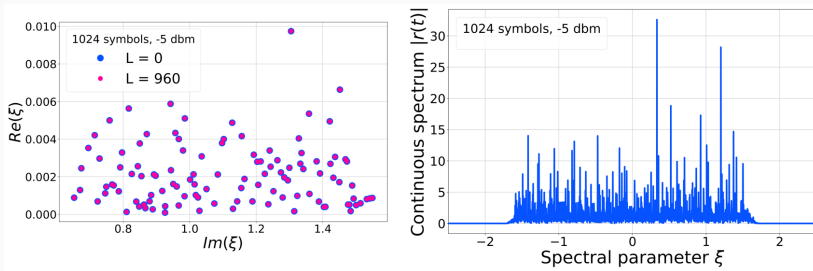
$\Omega = -2\xi_0$ – частота (“скорость”),

$T = -\frac{1}{2\eta_0} \log \frac{|r_0|}{2\eta_0}$ – положение,

$\varphi = -\arg [ir_0]$ – фаза.

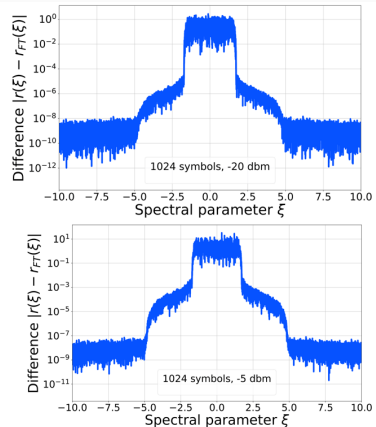
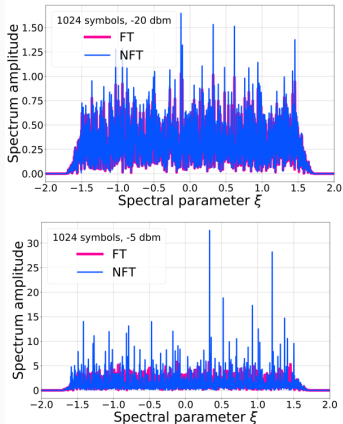


Прямая задача. Пример нелинейного спектра для телекоммуникационного сигнала



Пример дискретного (слева) и непрерывного (справа) нелинейного спектра Фурье для сигнала с 1024 символами и средней мощностью -5 dBm.

Прямая задача. Связь между линейным и нелинейным спектрами



Пример непрерывного нелинейного спектра Фурье и обычного спектра Фурье (слева) и абсолютное значение разности (справа).

Спектры NFT и FT слабо различаются при малой мощности и сильно отличаются при большой мощности!

Полный нелинейный спектр сигнала $q(z_0, t)$ выглядит следующим образом:

$$\sum_l = \left\{ I(\xi), [\zeta_n, I_n]_{n=1}^N \right\}, \quad \sum_r = \left\{ r(\xi), [\zeta_n, r_n]_{n=1}^N \right\}, \quad (11)$$

где N – общее число солитонов в сигнале.

В случае нормальной дисперсии ($\sigma = -1$, дефокусирующий НУШ) нелинейный спектр не имеет дискретной части и для его характеристики достаточно знать левый $I(\xi)$ или правый $r(\xi)$ коэффициенты отражения.

Собственные значения ζ_n не зависят от z :

$$\zeta_n(z) \equiv \zeta_n(z_0) = \text{const}$$

Для остальных величин имеем следующую зависимость от длины распространения z :

$$\begin{aligned} I(\xi, z) &= I(\xi, z_0) e^{-2i\xi^2(z-z_0)}, & I_n(z) &= I_n(z_0) e^{-2i\zeta_n^2(z-z_0)}, \\ r(\xi, z) &= r(\xi, z_0) e^{2i\xi^2(z-z_0)}, & r_n(z) &= r_n(z_0) e^{2i\zeta_n^2(z-z_0)}. \end{aligned}$$

Обратная задача

Фиксируем $z = z_0$ и рассматриваем потенциал $q(t) = q(z_0, t)$.

Из левого коэффициента отражения $I(\zeta)$, дискретных с. з. ζ_j и нормировочных констант r_j может быть составлена следующая функция:

$$\Omega_L(\theta) = \int_{-\infty}^{\infty} \frac{d\xi}{2\pi} e^{-i\xi\theta} I(\xi) + (-i) \sum_j r_j e^{-i\zeta_j\theta}$$

называемая ядром уравнений Гельфанда – Левитана – Марченко (УГЛМ) на функции $A_{1,2}(x, y)$:

$$\begin{aligned} A_1^*(x, y) + \int_{-\infty}^x dz A_2(x, z) \Omega_L(y + z) &= 0 \\ -A_2^*(x, y) + \Omega_L(x + y) + \int_{-\infty}^x dz A_1(x, z) \Omega_L(y + z) &= 0 \end{aligned}$$

для $-x \leq y \leq x$ и $0 \leq x$. Решение НУШ определяется как

$$q(t) = -2A_2(t, t - 0).$$

Ядро Ω_L и коэффициент $I(\xi)$ при отсутствующем дискретном спектре связаны соотношением:

$$I(\xi) = \int_0^{\infty} d\theta \Omega_L(\theta) e^{i\xi\theta}.$$

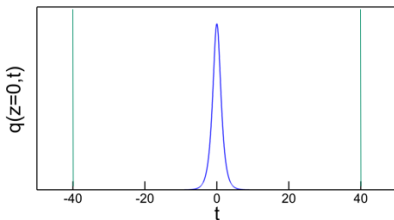
Основные этапы:

- нахождение непрерывного спектра (в каждой точке вещественной оси)
- нахождение дискретных с. з. (в верхней комплексной полуплоскости)
- нахождения нормирующих констант для каждого д. с. з.

Будем считать, что сигнал задан в точках дискретизации $m = 1, 2, \dots, M$ с шагом дискретизации по времени τ : $q_m = q(z_0, T_1 + \tau(m - \frac{1}{2}))$.

Шаг по временной переменной определяется как $\tau = (T_2 - T_1) / (M - 1)$, где T_1, T_2 – соответственно левая и правая границы временного интервала.

$$\begin{cases} \frac{d\phi_1}{dt} = q(z_0, t)\phi_2 - i\zeta\phi_1 \\ \frac{d\phi_2}{dt} = -\sigma q^*(z_0, t)\phi_1 + i\zeta\phi_2 \end{cases}$$



$$\begin{aligned} \phi_1(-T) &= e^{i\zeta T} \\ \phi_2(-T) &= 0 \end{aligned}$$

$$\xrightarrow{\hspace{1cm}} \begin{aligned} \phi_1(T) \\ \phi_2(T) \end{aligned}$$



Коэффициенты Йоста:

$$a(\zeta) = \phi_1(T, \zeta) e^{i\zeta T}, \quad b(\zeta) = \phi_2(T, \zeta) e^{-i\zeta T}$$

Перепишем уравнение (6) в виде

$$\frac{d\Phi}{dt} = Q(\zeta, q)\Phi, \quad Q(\zeta, q) = \begin{pmatrix} -i\zeta & q(z_0, t) \\ -\sigma q^*(z_0, t) & i\zeta \end{pmatrix}, \quad \Phi = \begin{pmatrix} \phi_1 \\ \phi_2 \end{pmatrix} \quad (12)$$

Пусть $q(z_0, t)$ есть кусочно-постоянная функция ($q_m = \text{const}$) на интервале $t \in [T_1 + (m - 0.5)\tau, T_1 + (m + 0.5)\tau]$. Тогда уравнение (12) можно проинтегрировать на этом интервале:

$$\begin{bmatrix} \phi_1(T_1 + (m + 0.5)\tau, \zeta) \\ \phi_2(T_1 + (m + 0.5)\tau, \zeta) \end{bmatrix} = T_m \begin{bmatrix} \phi_1(T_1 + (m - 0.5)\tau, \zeta) \\ \phi_2(T_1 + (m - 0.5)\tau, \zeta) \end{bmatrix}.$$

Для перехода со слоя m на слой $m + 1$ формально имеем уравнение

$$\Phi_{m+1/2} = T_m \Phi_{m-1/2}, \quad T_m = \exp(\tau Q_m). \quad (13)$$

При прямом вычислении матричной экспоненты получается **схема Боффетты-Осборна** (здесь $k^2 = -|q_m|^2 - \zeta^2$):

$$\begin{aligned} T_m^{(BO)}(\zeta) &= \exp(\tau Q_m(\zeta)) = \exp\left(\tau \begin{pmatrix} -i\zeta & q_m \\ -\sigma q_m^* & i\zeta \end{pmatrix}\right) \\ &= \begin{pmatrix} \cosh(k\tau) - \frac{i\zeta}{k} \sinh(k\tau) & \frac{q_m}{k} \sinh(k\tau) \\ -\sigma \frac{q_m^*}{k} \sinh(k\tau) & \cosh(k\tau) + \frac{i\zeta}{k} \sinh(k\tau) \end{pmatrix}. \end{aligned} \quad (14)$$

Общий вид передаточных методов:

$$\Phi(t_m + \tau) = T(q_m) \Phi(t_m) \quad (15)$$

Среди них также выделяют (улучшенный) метод Абловица-Ладика:

$$T_m(\zeta) = \frac{1}{\sqrt{1 + \sigma\tau^2 |q_m|^2}} \begin{pmatrix} Z & \tau q_m \\ -\sigma\tau q_m^* & Z^{-1} \end{pmatrix}, \quad Z = e^{i\zeta\tau} \quad (16)$$

Оба метода обладают 2-м глобальным порядком точности.

Численные методы для прямой задачи ЗШ. Быстрое вычисление – FNFT

Вычислительная сложность традиционного подхода составляет $O(N^2)$!

Быстрое нелинейное преобразование Фурье (fast nonlinear Fourier transform – FNFT) уменьшает вычислительную сложность до $O(N \log^2 N)$.

<https://github.com/FastNFT/FNFT>

Рассмотрим метод Абловица-Ладика:

$$\begin{pmatrix} \psi_1 \\ \psi_2 \end{pmatrix}_{n+1} = \frac{1}{\sqrt{1 + \tau^2 |q_n|^2}} \begin{pmatrix} e^{-i\tau\zeta} & \tau q_n \\ -\tau q_n^* & e^{i\tau\zeta} \end{pmatrix} \begin{pmatrix} \psi_1 \\ \psi_2 \end{pmatrix}_n.$$

Обозначим $Z = e^{-i\tau\zeta}$. Тогда получаем переходную матрицу $T_n(Z)$:

$$T_n(Z) = \frac{S_n(Z)}{d_n(Z)}, \quad d_n(Z) = Z \sqrt{1 + \tau^2 |q_n|^2}, \quad S_n(Z) = \begin{pmatrix} Z^2 & \tau q_n Z \\ -\tau q_n^* Z & 1 \end{pmatrix}.$$

Идея метода – быстрое перемножение матричных полиномов

$$S(Z) = S_N(Z) \times S_{N-1}(Z) \times \cdots \times S_2(Z) \times S_1(Z)$$

для различных значений переменной Z .

Итоговая вычислительная схема для коэффициентов Йоста:

$$\begin{pmatrix} a(\zeta) \\ b(\zeta) \end{pmatrix} \approx e^{i\zeta(T_2+0.5\tau)} \frac{S(Z)}{d(Z)} \begin{pmatrix} e^{i\zeta(T_1-0.5\tau)} \\ 0 \end{pmatrix}, \quad Z = e^{-i\tau\zeta},$$

то есть

$$\begin{pmatrix} a(\zeta) \\ b(\zeta) \end{pmatrix} \approx \frac{e^{i\zeta(T_2-T_1+\tau)}}{d(Z)} \begin{pmatrix} S_{1,1}(Z) \\ S_{1,2}(Z) \end{pmatrix}.$$

Здесь $d(Z) = \prod_{n=1}^N d_n(Z)$, а $S_{1,1}(Z)$ и $S_{1,2}(Z)$ – элементы первой строки результирующей матрицы $S(Z)$.

Метод поиска нулей функции на комплексной плоскости (Delves & Lyness, 1967).

Пусть $f(z)$ – аналитическая функция в области $\mathcal{D}$, и $\forall z \in \mathcal{D} \ |f(z)| < \infty$.
Разложим $f(z)$ и производную $f'(z)$ в ряд в точке $z_0 \in \mathcal{D}$:

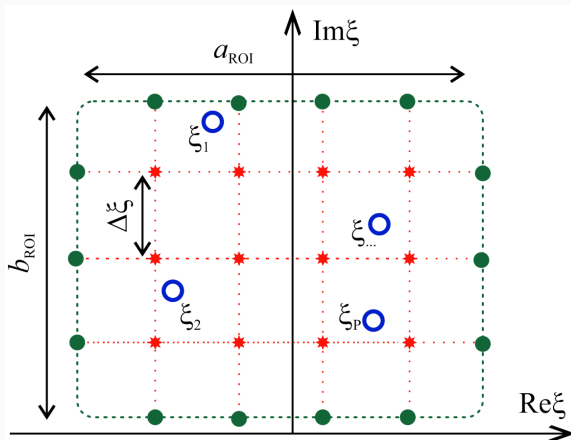
$$f(z) = \sum_{k=0}^{\infty} \frac{c_k}{k!} (z - z_0)^k, \quad f'(z) = \sum_{k=1}^{\infty} \frac{c_k}{(k-1)!} (z - z_0)^{k-1}, \quad c_k = f^{(k)}(z_0)$$

Если z_0 – нуль функции f , то $c_0 = 0$ и $f'/f = (z - z_0)^{-1} + \mathcal{O}(|z - z_0|^0)$.

Если z_0 нуль кратности n , то

$$f(z) = \sum_{k=n}^{\infty} \frac{c_k}{k!} (z - z_0)^k, \quad \frac{f'(z)}{f(z)} = \frac{n}{z - z_0} + \mathcal{O}(|z - z_0|^0).$$

Метод контурных интегралов



Используя принцип аргумента, имеем:

$$\oint_{\gamma} \frac{f'(z)}{f(z)} dz = 2\pi i N$$

С точностью до множителя $2\pi i$ получили количество нулей функции $f(z)$ внутри контура γ с учетом их кратности.

Обобщенный принцип аргумента:

$$\frac{1}{2\pi i} \oint_{\gamma} z^p \frac{f'(z)}{f(z)} dz = \sum z_i^p$$

Вычисляя интегралы в левой части, при $p = 0$ получим число нулей N внутри контура γ . Вычисляя для $p = 1 \dots N$, получим систему уравнений:

$$s_p = \sum z_i^p, \quad p = 1 \dots N.$$

Решать ее напрямую не нужно.

Метод контурных интегралов

Вместо этого, используя величины s_p , будем искать так называемые тождества Ньютона σ_p из следующей системы уравнений:

$$\sigma_1 = -\sum z_i : \quad s_1 + \sigma_1 = 0$$

$$\sigma_2 = z_1 z_2 + z_2 z_3 + \dots + z_{n-1} z_N : \quad s_2 + s_1 \sigma_1 + 2\sigma_2 = 0$$

...

$$\sigma_N = (-1)^N z_1 z_2 \dots z_N : \quad s_N + s_{N-1} \sigma_1 + \dots + s_1 \sigma_{N-1} + N \sigma_N = 0$$

Эта система решается рекуррентно:

$$\sigma_p = -\frac{1}{p} \left(\sum_{j=1}^{p-1} s_j \sigma_{p-j} + s_p \right)$$

Далее, используя σ_p , необходимо построить полином

$$P(z) = z^N + \sigma_1 z^{N-1} + \sigma_2 z^{N-2} + \dots + \sigma_{N-1} z + \sigma_N,$$

корни которого в точности совпадают с корнями изначальной функции $f(z)$.

Используя метод контурных интегралов, найти корни некоторой комплексной функции внутри заданной прямоугольной области (пример выбрать самостоятельно). Для вычисления интегралов в комплексной плоскости использовать формулу трапеций. Корни полинома $P(z)$ найти с помощью построения соответствующей сопровождающей матрицы и нахождения ее собственных значений. Собственные значения найти QR алгоритмом, самостоятельно его реализовав.

Статьи по теме

<https://core.ac.uk/download/pdf/161258182.pdf>

<http://npl.nsu.ru/website/photonics/upload/custom/files/optica-4-3-307.pdf>

Метод контурных интегралов для прямой задачи ЗШ

[https:](https://publications.aston.ac.uk/id/eprint/37290/1/optlett_v14.pdf)

[//publications.aston.ac.uk/id/eprint/37290/1/optlett_v14.pdf](https://publications.aston.ac.uk/id/eprint/37290/1/optlett_v14.pdf)

https://www.researchgate.net/publication/326656562_Contour_integrals_for_numerical_computation_of_discrete_eigenvalues_in_the_Zakharov-Shabat_problem

<https://arxiv.org/abs/1708.01144>